AIIT
AI Interview Training

코드 열람 가이드

파일·함수·메소드 사전과 찾아 읽는 법

AI 1차 채용면접 자동화 플랫폼 · Team AIIT · 2026-11-09 프로그램 기준

이 문서는 `huge/` 프로그램 안에 **어떤 파일이 있고, 그 안의 함수가 무슨 일을 하고, 누가 그 함수를 부르는지**를 쉽게 풀어 쓴 안내서입니다. 코드를 처음 보는 사람도 따라올 수 있게 **비유 → 실제 이름** 순서로 설명합니다. (2026-11-10 코드 기준 — 질문 글 검사·질문 적용 범위·공고 기반 질문 후보·AI 공고 초안 그래프 반영)

> **이렇게 읽으세요** 1장에서 '함수'가 무엇인지 알고 → 2장에서 가게 지도를 보고 → 3장에서 이야기 하나를 따라가고 → 4·5장에서 궁금한 파일을 찾아보세요. 맨 뒤 **부록**에는 함수 전체 목록이 있습니다. 사전처럼 찾아볼 때만 쓰면 됩니다.

---

1. 코드 읽기 첫걸음

1-1. 함수는 '자판기'예요

자판기에 **동전과 버튼(넣는 것)** 을 주면 **음료(나오는 것)** 가 나오죠. 함수도 똑같습니다.



그림. 함수는 자판기

```
def hashPassword(password):        # 이름: hashPassword, 넣는 것: password
    return hasher.hash(password)   # 나오는 것: 알아볼 수 없게 갈아 놓은 글
```

- `def` 는 "지금부터 자판기를 만든다"는 뜻이에요.
- 괄호 안 `password` 를 **인자**라고 불러요. 자판기에 넣는 재료예요.
- `return` 다음이 **반환값**, 자판기에서 나오는 결과예요.
- 다른 곳에서 `hashPassword("abc123")` 처럼 쓰면 자판기를 **부른다(호출한다)** 고 해요.
- `hashPassword` 는 🏠 우리가 만든 자판기지만, 그 안의 `hasher.hash` 는 📦 남이 만든 공구(argon2 라이브러리)예요. 누구 것인지 가리는 법은 1-6절에 있어요.

> **원래 뜻:** 함수(function)는 이름이 붙은 코드 묶음으로, 인자를 받아 정해진 일을 하고 반환값을 돌려준다.

> 🔧 **전문 설명** — 함수 정의는 `def 이름(매개변수) -> 반환형식:` 꼴이며, 이 프로젝트는 타입 힌트를 붙여 `def hashPassword(password: str) -> str` 처럼 씁니다. 호출 시 넘기는 실제 값이 인자(argument), 정의의 이름이 매개변수(parameter)입니다. Python 함수는 일급 객체라 📦

`Depends(requireAdmin)`, `graph.add_node("ask", ask)` 처럼 **이름만 넘겨 다른 코드가 나중에 부르게** 할 수 있습니다 — 그래서 일부 함수는 코드 안에 직접 부르는 곳이 없습니다.

1-2. 클래스는 '붕어빵 틀', 객체는 '붕어빵'



그림. 클래스는 틀, 객체는 그 틀로 만든 하나하나

- **클래스**는 틀이에요. `LiveSession` 은 "면접 한 건을 진행하는 직원" 틀입니다.
- 지원자 민지가 면접을 시작하면 이 틀로 **민지 전용 직원(객체)** 이 한 명 만들어져요. 지수가 시작하면 지수 전용 직원이 또 생기고요.
- 그 직원이 할 줄 아는 일이 **메소드**예요. `send`(화면에 말 전하기), `watchdog`(연결 지켜보기) 같은 것들이죠.
- 메소드 안의 `self` 는 "나 자신"이에요. `self.send(...)` = "내가 화면에 전한다".

> **원래 뜻:** 클래스는 데이터(속성)와 그 데이터를 다루는 함수(메소드)를 묶은 설계도이고, 객체는 그 설계도로 만든 실제 하나다.

> 🔧 **전문 설명** — `class LiveSession:` 의 `__init__` 이 생성자이고, `LiveSession(ws, applicationId, sessionId)` 호출이 인스턴스를 만듭니다. 인스턴스 속성(`self.state` , `self.deadline`)은 객체마다 따로 있습니다. 이 프로젝트는 상속을 거의 쓰지 않고, 쓰는 곳은 틀이 요구할 때뿐입니다: `Base(DeclarativeBase)`(SQLAlchemy 모델), `BaseModel`(Pydantic 스키마), `TypeDecorator`(암호화 칸), `Protocol`(`graph.Runner` — 구현을 강제하지 않는 구조적 인터페이스).

1-3. 그 밖에 자주 보이는 것

보이는 모양	쉬운 뜻	원래 뜻
`async def` / `await`	"라면 물 끓는 동안 다른 일 하기". AI 대답처럼 오래 기다릴 때 쉬어 가며 다른 일을 해요	비동기 함수. `await` 지점에서 실행을 양보한다
`@router.post("/auth/login")`	자판기에 붙인 **주소 이름표**. 이 주소로 요청이 오면 이 함수가 일해요	장식자(decorator). 함수를 등록하거나 감싼다
함수 안의 함수	큰 기계 안의 작은 부품	중첩 함수(nested function)
`raise ApiError(...)`	"이건 안 돼요!" 하고 멈추며 이유를 알려 줘요	예외를 발생시켜 실행을 중단한다
대문자로 시작하는 화면 함수 `Modal`	레고 블록 같은 **화면 조각**	React 컴포넌트

보이는 모양	쉬운 뜻	원래 뜻
`use` 로 시작하는 `useApp`	블록에 꽂는 **센서**. 로그인 정보 같은 걸 꺼내 줘요	React 훅(hook)

> 🔧 **전문 설명** — FastAPI 는 `async def` 핸들러를 이벤트 루프에서, 일반 `def` 핸들러를 스레드 풀에서 실행합니다. WebSocket 면접(`LiveSession`)은 한 연결 안에서 수신·heartbeat 감시·자막 중계를 🐍 `asyncio` 작업 여러 개로 동시에 돌립니다. React 쪽은 Next.js App Router 의 클라이언트 컴포넌트 (`"use client"`)이고, 상태는 📦 `useState` ·📦 `useEffect` , 전역 상태는 Context(`Providers`)로 나눕니다.

1-4. 이름만 봐도 하는 일이 보여요

이 프로젝트는 이름 짓는 규칙이 있어서, 이름 끝만 봐도 대충 알 수 있어요.

이름 모양	하는 일	예
`require…`	조건을 검사해서 아니면 막아요 (문지기)	`requireAdmin` — 관리자가 아니면 막음
`…Or404`	찾아보고 없으면 "없어요(404)"	`getPostingOr404`
`…View`	장부(DB) 내용을 화면에 보여 줄 모양으로 바꿔요	`postingView`
`…Task`	뒷주방(작업자)이 하는 오래 걸리는 일	`evaluateTask` — 채점
`is…` / `has…`	예/아니오로 대답해요	`isEmptyAnswer` — 답변이 비었나?
`UPPER_SNAKE`	바뀌지 않는 숫자·글 (상수)	`DEVICE_END_GRACE_SEC = 3`

> 🔧 **전문 설명** — 명명 규칙은 CLAUDE.md 5.1: Python 함수·변수 camelCase(PEP 8 의 snake_case 대신 프로젝트 규칙), 클래스 PascalCase, 상수 UPPER_SNAKE_CASE. DB 칸·API JSON 키는 TRD/API 명세서를 따라 snake_case 이며, SQLAlchemy 의 `process_bind_param` 처럼 라이브러리가 정한 이름은 예외입니다.

1-5. 함수를 찾는 법 (VS Code)

하고 싶은 일	누를 키
파일 열기	`Ctrl+P` 누르고 파일 이름
지금 파일의 함수 목록 보기	`Ctrl+Shift+O`
프로젝트 전체에서 함수 이름으로 찾기	`Ctrl+T`
이 함수가 **어디 있는지** 가기	이름 위에서 `F12`
이 함수를 **누가 부르는지** 보기	이름 위에서 `Shift+F12`
원래 있던 곳으로 돌아오기	`Alt+←`

> **팁:** 화면 주소와 파일은 짝이 있어요. 주소 `/mypage` → 파일 `frontend/app/mypage/page.tsx` . 주소 `/apply/123/consent` → `frontend/app/apply/[applicationId]/consent/page.tsx` (`[ ]` 는 '바뀌는 자리').

> 🔧 **전문 설명** — VS Code 의 `F12` / `Shift+F12` 는 Pylance(Python)·TypeScript 언어 서버의 정적 분석을 씁니다. 문자열로 등록된 Celery 작업(`"huge.sweep"`)이나 그래프 노드처럼 이름으로 연결된 곳은 참조

찾기에 안 잡히므로 `Ctrl+Shift+F` 문자열 검색을 함께 쓰세요.

1-6. 이 함수는 누가 만들었을까?

코드에 나오는 함수가 전부 우리가 만든 건 아니에요. 요리할 때 **내 손**, **집에 원래 있던 공구**, **가게에서 사 온 공구**, **내가 직접 만든 공구**를 섞어 쓰는 것과 같아요.

표시	누구 것?	비유	이 프로젝트의 예	설명은 어디서 찾나
	**내장** — 언어에 처음부터 붙어 있음	내 손	`len` , `print` , `open` , `KeyError` · (JS) `JSON.parse` , `Math.round`	Python / JavaScript 공식 문서
	**표준 라이브러리** — Python 을 깔면 같이 오는 공구함	집 공구함	`secrets` , `hashlib` , `datetime` , `asyncio` , `decimal`	Python 공식 문서 '표준 라이브러리'
	**브라우저 기능** — 브라우저가 화면 코드에 빌려줌	건물에 달린 전기·수도	`fetch` , `getUserMedia` , `MediaRecorder` , `sendBeacon`	MDN 웹 문서
	**외부 라이브러리** — 따로 설치한 남의 공구	가게에서 사 온 공구	FastAPI, SQLAlchemy, LangGraph, argon2 · React, Next.js	그 라이브러리의 문서
	**우리가 만든 함수** — `huge/` 안에 `def` · `function` 이 있음	직접 만든 공구	`hashPassword` , `ownedApplication` , `api`	이 가이드, `F12`

구분하는 법 — 파일 맨 위 `import` 줄을 보세요.

```
import secrets                          # 🐍 설치 안 했는데 있음 → 표준
from argon2 import PasswordHasher       # 📦 requirements.txt 에 있음 → 외부
from app.models import User             # 🏠 app 으로 시작 → 우리 것
from ..timeutil import now              # 🏠 점(.)으로 시작 → 우리 것 (옆 파일)

len(password)                           # 🧰 import 없이 바로 씀 → 내장
```

- `import` 없이 바로 쓰는데 이 파일에 `def` 도 없다? → 🧰 **내장**이에요.
- 이름이 `app.` 이나 `.` 으로 시작한다? → 🏠 **우리 것**이에요.
- 나머지는 `backend/requirements.txt`(화면은 `frontend/package.json`)에 있으면 📦 **외부**, 없으면 🐍 **표준**이에요.
- 화면 코드(`.tsx`)는 `@/` 나 `./` 로 시작하면 🏠, `react` · `next/...` 처럼 이름으로 시작하면 📦, `import` 없이 쓰는 `fetch` · `navigator` 는 🌐 예요.

이 가이드에서는: 부록 표에 적힌 함수는 **모두 🏠 우리가 만든 것**이에요. 바깥 것은 부록의 **"가져다 쓰는 모듈"** 줄과, 4·5장 🔧 전문 설명에서 이름 앞에 붙은 표시(🧰 🐍 🌐 📦)로 알 수 있어요. 표시가 없는 이름은 우리 것이에요.

원래 뜻: 내장 함수(built-in)는 import 없이 쓰는 함수, 표준 라이브러리(standard library)는 언어와 함께 배포되는 모듈 모음, 외부 라이브러리(third-party)는 따로 설치하는 패키지다.

> 🥢 **전문 설명** — Python 내장은 `builtins` 모듈(`dir(__builtins__)`)이고, 표준 라이브러리 목록은 `sys.stdlib_module_names` (3.10+)로 확인할 수 있습니다. 국내 교재(『점프 투 파이썬』 등)는 표준 라이브러리를 **'외장 함수'** 라고 부르기도 해서, pip 패키지를 뜻하는 '외부 라이브러리'와 헷갈리기 쉽습니다 — 이 가이드는 '외장'이라는 말 대신 **표준 / 외부**로 나눕니다. import 이름과 설치 이름이 다른 경우가 있습니다: `fitz` = pymupdf, `argon2` = argon2-cffi, `langchain_openai` = langchain-openai. 외부 라이브러리 함수는 버전에 따라 동작이 바뀔 수 있어 `requirements.txt` 에 버전을 고정(`fastapi==0.118.0`)합니다. FastAPI 의 `Depends` , SQLAlchemy 의 `TypeDecorator` 처럼 **외부 틀이 우리 함수를 대신 불러 주는** 경우가 많아, 부록의 '부르는 곳 `—` ' 은 대개 이 📦 틀이 부르는 함수입니다.

---

2. 가게 지도 — 파일들이 사는 곳

이 프로그램을 **AI 면접관이 있는 가게**라고 생각해 보세요. 손님(지원자)이 오면 여러 사람이 차례로 일을 넘겨줍니다.



그림. 요청 하나가 지나가는 층

가게 자리	하는 일	폴더
🪟 **홀** (손님이 보는 곳)	화면, 버튼, 입력 칸	`frontend/app` , `frontend/components`
🛎️ **카운터** (주문 받기)	주소별 요청 받기, 로그인·권한 확인	`backend/app/routers`
🍳 **주방** (요리하기)	질문 만들기, 이력서 읽기, 채점 같은 실제 일	`backend/app/services`
🎙️ **면접실**	면접을 실시간으로 진행	`backend/app/interview`
📏 **규칙판**	상태 이동·점수 계산 규칙 (DB도 AI도 안 씀)	`backend/app/domain`
📒 **장부**	어떤 표에 무엇을 적는지	`backend/app/models.py`
🌙 **뒷주방**	오래 걸리는 일·시간표 일	`backend/app/worker.py` , `jobs.py`

제일 중요한 약속: 홀은 카운터에게만 말하고, 카운터는 주방에 일을 넘기고, 주방은 규칙판과 장부를 봐요. 서로 자기 일만 하니까 고장 났을 때 어디를 고칠지 바로 알 수 있어요.

> 🔧 **전문 설명** — 계층형 구조(Layered Architecture): **표현(Next.js) → API(FastAPI 라우터) → 응용 서비스 → 도메인 → 영속성(SQLAlchemy/PostgreSQL)**. 의존은 한 방향(위 → 아래)이고, `domain/` 은 DB·네트워크를 import 하지 않는 순수 함수라 단위 테스트가 빠릅니다. 오래 걸리는 일은 메시지 큐(Redis 브로커)를 거쳐 Celery 작업자 프로세스(worker · worker-eval · worker-sandbox)가 처리해 API 응답 시간을 지킵니다.

---

3. 이야기로 따라가기 — 민지의 하루

지원자 **민지**가 지원하고 면접을 보고 결과를 받기까지, 어떤 함수들이 차례로 일하는지 따라가 봐요. `→` 는 "다음 사람에게 넘긴다(부른다)"는 뜻이에요.

📝 장면 1. 민지가 이력서를 제출해요

1. 민지가 확인 화면에서 **[제출하기]** 를 눌러요. → 화면 `Confirm` 이 `api()` 로 카운터에 주문을 보내요.
2. 카운터 `submitResume` 가 받아요. "작성 중" 상태에서 "준비됨"으로 바꿔도 되는지 규칙판 `transition` 에게 물어봐요.
3. 괜찮으면 `scheduleCompose` 에게 "민지 질문 10개 준비해 줘" 하고 주방에 맡겨요. 민지는 기다리지 않고 다음 화면으로 가요.
4. 주방의 `compose` 가 질문 10칸을 하나씩 채워요.
   - `similar` 가 비슷한 기존 질문 3개를 찾아 **예시**로 펼쳐 놓고 (RAG),
   - AI(`structured`)가 민지 이력서에 맞는 새 질문을 만들고,
   - `checkGenerated` 가 "질문 모양이 맞나? 이력서에 근거가 있나? 예시를 베끼지 않았나?" 검사해요.
   - 글 검사관 `textProblem` 도 봐요: 이상한 글자가 붙었나? "간단히 답해 주세요" 같은 **답하는 방법 지시**가 있나? **한 번에 두 가지**를 묻나? 말끝이 "~느꼈는가요"처럼 어색하나? 걸리면 "이게 문제였어" 하고 알려 주며 다시 만들게 해요.
   - 3번 실패하면 `bankFallback` 이 **질문은행**에서 골라 와요. 그것도 없으면 예비 질문, 마지막엔 고정 문장이에요.
   - 질문은행에서 고를 때 `candidates` 는 **이 공고에 쓸 수 있는 질문**만 봐요. 다른 공고 전용 회사 질문이나 다른 직무 질문은 빼요(적용 범위).
5. 다 끝나면 장부에 "질문 준비 완료(ready)"라고 적어요.

> 🔧 **전문 설명** — `POST /applications/{id}/resume` → `submitResume` 가 `transition(drafting → ready)` 후 커밋하고 `scheduleCompose` 로 `asyncio` 작업을 띄웁니다(응답은 즉시). `compose` 는 LangGraph `StateGraph` 로 10칸(`SLOT_ORDER`)을 채우며, RAG 단계에서 BGE-M3 1024차원 임베딩으로 pgvector 코사인 검색을 하고, LLM 출력은 JSON 스키마(`RagQuestion`)로 강제·Pydantic 재검증합니다. 글 검사는 `domain/question_text.py` 의 정규식 규칙(찌꺼기 `junk_text` , 답변 방식 지시 `meta_phrase` , 두 가지 묻기 `compound_question` , 어색한 말끝 `awkward_ending` 등)이고, 실패 코드를 `PROBLEM_TEXT` 문장으로 바꿔 이전 질문과 함께 다음 프롬프트에 넣습니다(자기 수정 루프). 질문은행 후보는

`posting_id · job_code` 가 비었거나 이 공고·이 직무와 같은 행만 고릅니다(C-17). 검증(`checkGenerated`) 실패가 `question_regen_limit` 번이거나 시간 예산(60초)을 넘기면 질문은행 → 예비 질문 → 고정 문장 순으로 내려가는 **단계적 대체(graceful degradation)** 구조입니다.

🎙️ 장면 2. 민지가 면접을 봐요

1. 민지가 **[면접 시작]** → 카운터 `startInterview` 가 받아요.
   - `requirePrecheck` : 환경 점검을 15분 안에 통과했나?
   - `tryAcquire` : 지금 면접 자리가 남았나? (놀이기구 번호표)
2. 화면 `Exam` 이 **WebSocket**(끊지 않는 전화선)을 열어요. → `interviewSocket` → 민지 전용 직원 `LiveSession` 이 생겨요.
3. 직원은 순서도(`buildInterviewGraph`)대로 움직여요.
   - `sendQuestion` : 질문 음성과 자막을 보내요.
   - 민지가 말하면 `onAudio` 가 소리를 모으고,
   - `transcribeWithRetry` 가 소리를 글로 바꾸고(STT), `saveAnswer` 가 장부에 적어요.
   - `buildFollowupGraph` : 답이 애매하면 꼬리질문을 하나 더 만들어요.
   - "~한 적이 있나요?"처럼 **예/아니오로 답하는 질문**에 민지가 "네" 하고 짧게만 답하면, `yesNoFollowup` 이 AI 에게 묻지도 않고 바로 "그렇게 답하신 이유를 구체적으로 말씀해 주세요."라고 물어요.
4. 그동안 `watchdog` 은 1초마다 "전화선이 살아 있나?" 지켜봐요.
5. 10문제가 끝나면 `finishInterview` → (코딩테스트가 있으면) `enterCoding` , (없으면) `endWith(completed)` → `finish` .

🔧 **전문 설명** — `startInterview` 는 한 트랜잭션 안에서 점검 유효성(`precheck_valid_until`) 확인, Redis Lua 원자 연산으로 동시 면접 자리 확보(`tryAcquire`), 상태 전이를 처리합니다. 이후 WebSocket(`/ws/interview/{sessionId}`)에서 `LiveSession` 이 LangGraph 체크포인트(PostgreSQL saver, `thread_id = 지원 ID`)와 함께 그래프를 돌려, 재연결 시 같은 지점부터 이어집니다. 답변 음성은 16kHz mono PCM16 바이너리 프레임으로 받고, STT 실패는 `stt_retry_limit` 만큼 재시도합니다. 꼬리질문 판단 전 `yesNoFollowup` 이 `isYesNo(본질문)` 이고 답이 `YES_NO_SHORT_ANSWER` (30자)보다 짧으면 LLM 호출 없이 고정 꼬리질문(reason_code `vague`)을 정하고, 길면 `followupPrompt` 가 '이유가 없으면 이유를 물어라'를 덧붙입니다. 꼬리질문 글도 `validateFollowup` 에서 본질문과 같은 `textProblem` 검사를 거칩니다.

⚡ 장면 3. 갑자기 인터넷이 끊기면?

1. `watchdog` 이 15초 동안 "살아 있어" 신호를 못 받아요.
2. 규칙판 `judgeDisconnect` 에게 물어요: "민지가 나가기 직전에 **'나가요' 쪽지**를 남겼나?"
   - 쪽지 없음 → 인터넷 문제(장애) → `pauseForFault` → `markInterrupted` . 답변은 저장되고, 나중에 이어서 볼 수 있어요.
   - 쪽지 있음 → 민지가 창을 닫은 것 → `endWith(ended)` . 응시가 끝나요.

> 🔧 **전문 설명** — 끊김 판정은 `heartbeat_interval_sec × heartbeat_miss_limit` (5초 × 3 = 15초) 무응답, 또는 장치 종료 + `DEVICE_END_GRACE_SEC` (3초) 대기 뒤 수행합니다. 브라우저는 `pagehide` / `beforeunload` 에서 🌐 `navigator.sendBeacon` 으로 unload 이벤트를 남기고, `judgeDisconnect` 가 `unload_lookback_sec` 안의 이벤트 유무로 `fault` / `not_fault` 를 반환합니다. 판정 근거가 없으면 fault — **불확실성은 지원자에게 유리하게** 라는 설계 원칙입니다.

🧮 장면 4. 채점하고 결과를 알려 줘요

1. `finish` → `queueEvaluation` → `enqueueEvaluation`: 뒷주방에 "채점해 줘" 쪽지를 넣어요.
2. 뒷주방의 `evaluateTask` → `evaluate` 가 시작해요.
    - `redact`: AI가 보기 전에 이름·연락처를 지워요.
    - `scoreQuestion`: 질문마다 AI(4B)가 항목별 0~5점과 **근거 문장**을 내요.
    - `matchQuote`: 그 근거가 **민지 답변에 정말 있는지** 코드가 확인해요 (90% 이상 같으면 인정).
    - `computeScores`: 점수 계산은 **AI가 아니라 코드**가 해요. 총점 ≥ 합격 기준이면 1차 합격.
    - `writeReason`: AI(8B)가 "왜 이 결과인지" 이유 글을 써요. 결과 자체는 못 바꿔요.
    - 이상한 점이 있으면 `NeedsReview` → 관리자가 직접 보는 **평가 지연**으로 가요.
3. 5분마다 `sweepTask` → `releaseDue`: 공개 시각(마감 다음 날 10:00)이 지난 결과를 공개하고 알림을 만들어요. → `queueMail` 로 메일도 보내요.

> 🔧 **전문 설명** — 채점은 커밋 뒤 큐잉(`enqueueEvaluation`)하고, 브로커 장애 시 5분 주기 `sweepTask` 가 누락분(`findUnscored`)을 재투입하는 **at-least-once + 멱등 잠금(`eval:lock:{id}`)** 구조입니다. T5(문항 채점, Qwen3 4B, temperature 0) 결과의 근거 인용은 `matchQuote` 로 원문 대조(정규화 후 `QUOTE_MATCH_RATIO = 0.9`)하고, 점수는 🐍 `Decimal` · 🐍 `ROUND_HALF_UP` 으로 코드가 계산합니다. T10(결과 이유, Qwen3 8B)은 이미 정해진 판정을 입력으로 받아 설명만 생성하며, 검증 실패 2회면 `NeedsReview` (평가 지연)로 사람에게 넘깁니다.

---

4. 백엔드 파일 소개 — "이 파일은 누구?"

각 파일을 **가게의 한 사람**으로 소개합니다. 중요한 함수만 골랐고, 전체 목록은 부록에 있어요.

🚪 입구와 공통 도구

`main.py` — **가게 문 여는 사람** 가게(서버)를 켤 때 필요한 걸 다 연결해요.

- `lifespan` — 켤 때: 장부 표 만들기, 직무 목록 넣기, 1분 정리 일 시작. 끌 때: 정리.
- `recoverAfterRestart` — 껐다 켰을 때, 하다 만 질문 준비를 다시 시작하고 자리 수를 맞춰요.
- `warmUp` — 첫 손님이 기다리지 않게 AI를 미리 깨워요 (첫 호출은 45초 걸렸어요).
- `UploadSizeGuard` — 너무 큰 이력서 파일은 문 앞에서 돌려보내요.

> 🔧 **전문 설명** — FastAPI 앱 팩토리. `lifespan` (async context manager)에서 🟫 `create_all` + `migrate.addColumns`, 체크포인트 saver 열기, `jobs.runForever` 백그라운드 작업 등록. `UploadSizeGuard` 는 ASGI 미들웨어로 `Content-Length` 를 보고 본문을 읽기 전에 411/413 으로 거절합니다(메모리 보호). 예외 처리기(`apiErrorHandler` 등)와 8개 라우터를 🟫 `include_router` 로 붙입니다.

`config.py` — 설정 메모장

- `loadEnv` — 컴퓨터 환경 변수(DB 주소, 암호 열쇠 등)를 읽어요.
- `SETTING_DEFAULTS` — 이어가기 3번, heartbeat 5초 같은 기본값 모음이에요.

> 🔧 **전문 설명** — `Env` dataclass 에 환경 변수를 한 번 읽어 둡니다(12-factor 설정). `SETTING_DEFAULTS` 는 `{키: (운영값, 개발값)}` 이고 실제 값은 DB `app_settings` 가 우선합니다 (`settings_store.getSetting`).

`deps.py` — 문지기들

- `currentUser` — 로그인 안 했으면 "로그인하세요(401)".
- `requireAdmin` / `requireApplicant` — 관리자만, 지원자만 들어오게 해요.
- `ownedApplication` — **내 지원서만** 볼 수 있어요. 남의 것이면 "없어요(404)"라고 해서 있는지조차 숨겨요.

> 🔧 **전문 설명** — FastAPI 의존성 주입(🟫 `Depends`) 함수 사슬: `optionalUser → currentUser → activeUser → requireAdmin/requireApplicant` . `ownedApplication` 은 소유자 불일치 시 403 이 아닌 404 를 돌려 **자원 존재 여부 노출(IDOR 탐색)** 을 막고, PostgreSQL 에서는 `SELECT … FOR UPDATE` 행 잠금을 겁니다.

`security.py` — 금고지기

- `hashPassword` / `verifyPassword` — 비밀번호를 되돌릴 수 없게 갈아서(Argon2) 저장하고, 로그인할 때 맞는지만 확인해요.
- `createSession` / `resolveSession` — 로그인하면 **출입 팔찌(쿠키)** 를 주고, 다음 요청 때 팔찌로 누군지 알아봐요.

> 🔧 **전문 설명** — 비밀번호는 `argon2-cffi` 의 🟫 `PasswordHasher` (Argon2id, 메모리 하드 KDF). 세션 토큰은 🐍 `secrets.token_urlsafe(32)` 로 만들고 DB 에는 **SHA-256 해시만** 저장해 DB 유출 시 세션 탈취를 막습니다. 쿠키는 HttpOnly(스크립트 접근 차단), 만료는 `session_idle_hours` 슬라이딩. `safeNext` 는 로그인 뒤 이동 경로를 허용 목록으로 제한해 오픈 리다이렉트를 막습니다.

`crypto.py` — 자물쇠 장인

- `encryptText` / `decryptText` — 답변·이력서 같은 민감한 글을 열쇠로 잠그고 풀어요 (AES-256-GCM).
- `EncryptedText` — 장부 칸에 붙이면 **저장할 때 자동으로 잠그고, 읽을 때 자동으로 풀어** 줘요.

> 🔗 **전문 설명** — AES-256-GCM(AEAD): 96비트 무작위 nonce + 암호문 + 인증 태그를 base64 로 저장해 **기밀성과 무결성(변조 탐지)** 을 함께 보장합니다. `EncryptedText` / `EncryptedJson` 은 SQLAlchemy 🟫 `TypeDecorator` 로 ORM 경계에서 투명하게 암복호화합니다. 키(`APP_ENCRYPTION_KEY` , 32바이트)가 없으면 기동을 거부(fail-closed)합니다. `subjectHash` 는 동의자 식별에 HMAC 을 써 이메일 사전 대입으로 되돌리지 못하게 합니다.

`errors.py` — 안내 방송

- `ApiError` — "왜 안 되는지"를 `{code, message}` 한 가지 모양으로 화면에 알려 줘요.

> 🔗 **전문 설명** — 도메인 예외 `ApiError(status, code, message)` 를 전역 예외 처리기가 `{"code", "message"}` JSON 으로 바꿉니다(TRD 4-3). Pydantic 검증 오류는 422 `VALIDATION_ERROR` 한 줄로, 예상 못 한 예외는 내부 정보를 숨긴 500 으로 응답합니다.

`timeutil.py` — 시계

- `now` — 지금 한국 시각. `releaseAtFor` — 결과 공개 시각(마감 다음 날 10:00).

> 🔗 **전문 설명** — 모든 시각은 `Asia/Seoul` tz-aware 🐍 `datetime` . SQLite 는 시간대를 저장하지 않아 `asKst` 가 naive 값을 KST 로 해석해 테스트(SQLite)와 운영(PostgreSQL `timestamptz`)의 비교 결과를 맞춥니다.

`settings_store.py` — 운영 설정 서랍

- `getSetting` / `setSetting` — 관리자가 바꾼 설정을 읽고 써요. 없으면 기본값.

> 🔗 **전문 설명** — 키-값 설정 테이블(`app_settings` , 값은 `{"v": …}` JSON). `setSetting` 은 변경 감사 로그를 함께 남깁니다. 알 수 없는 키는 🧰 `KeyError` 로 거부합니다.

`models.py` — 장부 양식 DB에 어떤 표가 있는지 정한 파일이에요. `User` (회원), `JobPosting` (공고), `Application` (지원서), `Answer` (답변), `Result` (결과) 등 28개 표가 있어요. 부록에서 표 이름과 칸을 볼 수 있어요.

> 🔗 **전문 설명** — SQLAlchemy 2.0 선언형 매핑(🟫 `Mapped[...]` , 🟫 `mapped_column`). 기본 키는 UUID 문자열(`newId`), 민감 칸은 `EncryptedText` . 무결성은 DB 제약으로도 지킵니다: `CHECK` (상태 값 목록, `checkName`), `UNIQUE(posting_id, user_id)` (같은 공고 중복 지원 차단, `uq_applications_posting_user`).

> 칸 이름은 테이블 정의서를 따라 snake_case.

`schemas.py` — **주문서 양식** 화면이 보내는 요청이 **올바른 모양인지** 검사해요. 예: `SignupRequest` 는 이름·이메일·비밀번호·약관 동의가 다 있어야 통과.

> 🔧 **전문 설명** — Pydantic v2 모델. 공통 부모 `Strict` 가 정의되지 않은 필드를 거부(`extra="forbid"`) 해 **대량 할당(mass assignment)** 을 막고, 🟫 `Field(pattern=…, max_length=…)` 로 입력 길이·형식을 경계에서 검증합니다. 예: `SignupRequest.phone` 은 `^01[0-9]-\d{3,4}-\d{4}$` .

📏 규칙판 (`domain/`)

`domain/status.py` — 보드게임 말판

- `transition` — "작성 중 → 준비됨"처럼 **허락된 칸으로만** 말을 옮겨요. 아니면 오류.
- `judgeDisconnect` — 끊겼을 때 '나가요' 쪽지를 보고 장애인지 본인이 나간 건지 정해요. 모르면 **지원자에게 유리하게 장애**.
- `nextAction` — 마이페이지 카드에 어떤 버튼(이어 쓰기·점검·이어가기…)을 보여 줄지 정해요.

> 🔧 **전문 설명** — 유한 상태 기계(FSM): `ALLOWED: dict[상태, set[다음 상태]]` 를 단일 진실 원천으로 두고 `transition()` 만 상태를 바꿉니다. 불법 전이는 `TransitionError` . `judgeDisconnect` 는 최근 이벤트 집합만 받는 순수 함수라 DB 없이 테스트됩니다.

`domain/scoring.py` — 계산기

- `itemScores` → `interviewScore` → `totalScore` — 항목 점수를 모아 면접 점수, 총점을 계산해요. 0.1+0.2 오차가 없는 🐍 `Decimal` 로 계산해요.
- `matchQuote` — AI가 가져온 근거가 답변에 진짜 있는지 찾아요 (띄어쓰기·문장부호 차이는 봐줘요).

> 🔧 **전문 설명** — 항목 점수 = 그 항목을 본 문항 점수 평균 ÷ 5 × 100, 면접 점수 = Σ(항목 점수 × 가중치), 총점 = 코딩 사용 시 비율 가중 합(EVL-03·04). 모두 🐍 `Decimal` + 🐍 `quantize(0.01, ROUND_HALF_UP)` 로 이진 부동소수점 오차(69.999…)를 없앱니다. `matchQuote` 는 정규화(공백·문장부호 제거) 후 정확 일치 → 없으면 `closestSpan` 으로 유사도 0.9 이상 구간을 찾아 **답변 원문 구절**을 돌려줍니다(E17).

`domain/question_text.py` — 질문 글 검사관 (2026-11-10)

- `textProblem` — AI 가 만든 질문 글을 보고 문제를 찾아요: 이상한 글자(이모지·영어 조각), "1문장으로"·"간단히" 같은 답하는 방법 지시, 한 번에 두 가지 묻기, 질문으로 안 끝남, "~는가요" 같은 어색한 말끝, 100자 넘음.
- `cleanQuestion` — 질문 끝에 묻어 나온 따옴표·괄호 부스러기를 털어 내요.
- `isYesNo` — "예/아니오"로 끝나 버리는 질문인지 알아봐요("무엇·어떻게·왜"가 없고 "설명해 주세요"도 아닌 질문).

> 🔧 **전문 설명** — DB·AI 없는 순수 함수 모음이라 단위 테스트(`test_question_text.py`)로 빠르게 검증됩니다. 문제 코드는 우선순위 순서로 하나만 돌려줍니다(foreign_script → junk_text → meta_phrase → compound_question → not_question → awkward_ending → too_long). 본질문(`question_compose.checkGenerated`)과 꼬리질문(`interview/graph.validateFollowup`)이 같은 함수를 써서 기준이 어긋나지 않습니다. 엑셀 질문은행·예비 질문에는 적용하지 않습니다(사용자 결정, C-16).

`domain/password_policy.py` — 비밀번호 규칙표

- `checkPassword` — 8~64자, 영문·숫자·특수문자 섞기 같은 규칙을 어기면 무엇을 어겼는지 알려 줘요.

> 🔧 **전문 설명** — 규칙: 길이 8~64, 영문·숫자·특수문자 각 1자 이상, 이메일 로컬 파트 포함 금지, 같은 문자 4회 연속 금지. 가입·변경·재설정이 같은 함수를 쓰고, 프론트 `PasswordChecklist` 도 같은 규칙을 보여 줍니다(서버가 최종 판단).

🛎 카운터 (`routers/`) — 주소별 창구

`routers/auth.py` — 회원 창구

- `signup` (회원가입), `login` (로그인 — 5번 틀리면 잠시 막힘), `logout` , `passwordReset` (비밀번호 찾기 메일), `withdrawMe` (탈퇴).

> 🔧 **전문 설명** — 로그인 실패는 계정이 아닌 **IP 단위**로 세어 `login_fail_limit` (기본 5) 초과 시 429 `LOGIN_BLOCKED` (계정 잠금 공격으로 남의 계정을 막지 못하게). 비밀번호 재설정은 계정 존재 여부와 상관없이 같은 응답(202)을 줘 이메일 열거를 막습니다. 토큰은 Redis 에 SHA-256 해시 키로 30분 TTL 저장, 사용 시 `DELETE` 결과로 1회만 통과(동시 클릭 경쟁 방지), 재설정 뒤 그 사용자의 모든 세션을 지웁니다. 로그인 쿠키는 `HttpOnly; SameSite=Lax` .

`routers/postings.py` — 공고 게시판 (지원자용)

- `listPostings` / `getPosting` — 공고 목록과 상세.

> 🔧 **전문 설명** — 인증 없이 열리는 읽기 전용 API. 게시(`published`) 공고만 노출하고 `postingView` 로 내부 칸(평가 기준 세부 등)을 걸러 냅니다.

`routers/applications.py` — 지원 창구

- `createApplication` — [지원하기]. 공고 하나에 한 번만.
- `consent` — 동의 5가지 받기.
- `uploadResume` → `acceptUpload` — 이력서 파일을 받아 크기·형식을 보고 주방(추출)에 넘겨요.
- `submitResume` — 이력서 확정 + 질문 준비 시작 (3장 장면 1).

> 🔗 **전문 설명** — `createApplication` 은 🟫 `with_for_update` 직렬화 + DB `UNIQUE` 2중 방어. 업로드는 🐍 `asyncio.Semaphore` 로 동시 2건, `detectType` 으로 매직 넘버(파일 앞 바이트) 검사, 상한(`upload_max_mb`) 초과는 413. 추출은 비동기로 돌고 화면은 `extract-status` 를 폴링합니다. 제출 뒤 수정은 409 `RESUME_LOCKED`.

`routers/interview.py` — 면접 창구

- `precheck` — 환경 점검 결과 받기.
- `startInterview` / `resumeInterview` — 면접 시작, 이어가기.
- `interviewSocket` — 면접 전화선(WebSocket) 연결 → `LiveSession` 에게 넘김.
- `codingRun` / `codingAutosave` / `codingSubmit` — 코딩테스트 실행·자동 저장·제출.

> 🔗 **전문 설명** — REST(점검·시작·이어가기·코딩) + WebSocket(진행) 혼합. `interviewSocket` 은 `Origin` 헤더가 `PUBLIC_ORIGIN` 과 같을 때만 받고(교차 사이트 WebSocket 하이재킹 방지), `registry.claimSession` 으로 세션 ID 당 연결 1개만 허용합니다. `beaconEvent` 는 🌐 `sendBeacon` 이 Content-Type 을 고를 수 없어 본문을 직접 파싱합니다. 코딩 API 는 남은 시간이 0이면 서버가 자동 제출(`autoSubmitIfOver`)합니다.

`routers/me.py` — 마이페이지 창구

- `myApplications` (지원 내역), `myResult` (결과, 공개 뒤에만), `notifications` (🔔 알림).

> 🔗 **전문 설명** — 결과는 `release_status == released` 이고 공개 시각이 지났으며 `purge_at` 전일 때만 반환(`isReleased`). 지원자용 결과에는 근거 인용·합불 사유 원문이 아닌 요약·강점·개선점만 담습니다(RPT-03).

`routers/admin.py`, `admin_ops.py`, `admin_questions.py` — 관리자 창구

- 공고: `createPosting`, `patchPosting`, `draftAi` (AI 초안), `draftAiRevise` (AI 에게 한 칸 다시 쓰기), `publish` (게시 — 게시하면 그 공고로 질문 후보 만들기도 시작), `close` (마감).
- 지원자: `applicants` (목록), `applicantDetail` (상세), `compare` (비교), `passed` (1차 합격자).
- 평가 지연: `reprocess` (다시 돌리기), `manualDecision` (직접 결정).
- 운영: `stats` (통계), `opsStatus` (운영 현황), `putSetting` (동시 면접 수), `releaseLoginBlock` (로그인 차단 풀기).
- 질문: `listQuestions` (출처와 적용 범위 `scopeView` 를 같이 보여 줌), `approve` ([추가]), `reject` ([제외]·[빼기]).

> 🔗 **전문 설명** — 모든 엔드포인트가 🟫 `Depends(requireAdmin)`. 변경 행위(게시·마감·직접 결정·설정 변경·차단 해제)는 `AuditLog` 에 남기되, 지원자 평가 내용(메모 본문·결정 사유)은 감사 로그에 넣지 않아 개인정보 노출면을 줄입니다. 지원자 이메일은 운영 현황에서 `maskEmail` 로 가립니다.

🍳 주방 (`services/`)

`question_compose.py` — 질문 요리사 (3장 장면 1)

- `compose` — 질문 10개 만들기 전체. `ensureComposed` — 면접 시작 때 아직이면 조금 기다리고, 넘으면 AI 없이 채워요.
- `checkGenerated` — 만든 질문 검사. `bankFallback` — 질문은행에서 고르기. `lastResort` — 최후의 고정 문장.
- `synthesizeAll` — 질문 음성(TTS)을 미리 만들어 둬요.
- `scopeFor` — 질문에 **어디서 쓸 수 있는지** 꼬리표를 붙여요: 회사 질문은 "이 공고만", 직무 질문은 "이 직무만", 인성 질문은 "어디서나".
- `savePending` — 면접 때 AI 가 만든 좋은 질문을 꼬리표와 함께 '검토 대기'로 모아 둬요(관리자가 [추가]해야 씀).

> 🔧 **전문 설명** — LangGraph 📦 `StateGraph(SlotState)` : `pick → examples → generate → check → (accept | regenerate | bank)` 조건부 간선. 중복 방지는 3중: 정규화 문자열 비교(`sameText`), 임베딩 코사인 0.9(`maxSimilarity`), 엑셀 `duplicate_group` . 사용 횟수는 `INSERT … ON CONFLICT` 로 경쟁 없이 +1. LLM 입력에는 `resumeText` 가 이름·연락처를 뺀 요약만 넣습니다.

`rag.py` — 비슷한 질문 찾는 사서

- `embed` — 문장의 '뜻'을 숫자 목록(벡터)으로 바꿔요.
- `similar` — 뜻이 가장 가까운 기존 질문을 찾아요. `cosine` — 두 뜻이 얼마나 가까운지 숫자 하나로 (1에 가까울수록 비슷).

> 🔧 **전문 설명** — 임베딩 서버(BGE-M3, 1024차원) HTTP 호출 → pgvector `vector` 칸에 저장, 검색은 코사인 거리. 응답은 개수·차원을 검사(`parseResponse`)하고 실패는 `EmbedUnavailable` 로 올려 상위가 질문은행으로 대체합니다. SQLite(테스트)에서는 `supported() == False` 로 벡터 기능을 끕니다.

`resume_extract.py` — 이력서 번역가

- `extractUpload` — 읽기 → (스캔본이면) OCR → AI 정리 → 저장, 전체 흐름.
- `detectType` — 이름만 `.pdf` 로 바꾼 가짜 파일을 걸러요.
- `structure` — AI가 글을 학력·경력·기술 칸에 나눠 담아요.
- `lowConfidenceFields` — 흐릿하게 읽힌 칸을 표시해서 지원자가 확인하게 해요.

> 🔧 **전문 설명** — 파이프라인: 형식 검사 → 격리 프로세스(`pdf_box`)로 텍스트층 추출(PyMuPDF → pdfplumber) → `isReadable` 이 실패한 쪽만 렌더링 후 OCR(PaddleOCR) → `repairDateRanges` · `splitCoverLetter` 전처리 → LLM 구조화(`StructuredResume` 스키마) → `fillBlanks` 병합. OCR 신뢰도 낮은 줄에서 온 칸 경로(`career[0].company`)를 돌려 화면이 강조합니다. 동시 추출은 세마포어로 제한.

`pdf_box.py` — **유리방 PDF 읽개** PDF는 **따로 떨어진 작은 프로세스**에서 읽어요. 이상한 PDF가 터져도 서버는 멀쩡해요.

- `readPages` — 쪽마다 글 뽑기. `renderPage` — OCR용 그림 만들기.

> 🔧 **전문 설명** — 신뢰할 수 없는 파일 파싱을 **프로세스 격리**합니다: `subprocess` 로 stdin 에 PDF 를 넣고, 리눅스에서는 `resource.setrlimit` 으로 메모리·CPU 시간을 제한하며 부모가 타임아웃을 겁니다. 파서 크래시·폭탄 PDF 가 API 프로세스에 영향을 주지 않습니다.

`evaluation.py` — **채점 선생님** (3장 장면 4)

- `evaluate` — 채점 전체. `scoreQuestion` — 질문 하나 채점. `computeScores` — 점수 계산.
- `redact` — 이름·연락처 지우기. `fence` / `instructionHits` — 답변 속 "만점 줘" 같은 **몰래 명령** (프롬프트 인젝션) 막기.
- `writeReason` / `writeFeedback` — 결과 이유, 강점·개선점 글.
- `decideManually` — 관리자 직접 결정을 저장해요.

> 🔧 **전문 설명** — 입력 준비(`loadInput` : 지원 시 잠근 기준 버전만) → `redact` (PII 제거) → `fence` (꺾쇠 치환으로 구분 태그 탈출 방지) → `instructionHits` (채점 조작 문구 탐지 → 관리자 확인) → T5 문항 채점(`T5Out` 스키마, 근거 `max_length` 제한) → `checkT5` → `computeScores` → T10 이유(`checkT10` 검증) → T6/T7/T8 요약·피드백·코드 리뷰 → `save` (행 잠금 단일 트랜잭션). 실패 유형을 `AiUnavailable` (재시도)· `EvalFailed` · `NeedsReview` (사람 검토)로 나눠 처리합니다.

`llm.py` — **AI 전화기**

- `structured` — AI에게 묻고, 대답이 **정해진 모양(JSON)** 인지 검사해요. 시간 초과·모양 틀림은 오류.
- `laneUrl` — 빠른 4B 서버로 갈지, 꼼꼼한 8B 서버로 갈지.

> 🔧 **전문 설명** — llama.cpp 서버(OpenAI 호환 API, Q4_K_M 양자화 GGUF 모델)를 LangChain `ChatOpenAI` 로 호출. `structured` 는 `with_structured_output(schema, method='json_schema')` 로 JSON 스키마에 맞춘 출력을 받고 Pydantic 으로 다시 검증해 **형식을 두 번 확인**합니다. lane `rt` =Qwen3 4B(GPU, 실시간), `eval` =Qwen3 8B(CPU, 결과 이유).

`speech.py` — **귀와 입**

- `transcribe` — 소리 → 글 (STT). `synthesize` — 글 → 소리 (TTS).

> 🔧 **전문 설명** — 내부 STT(Whisper 계열, GPU)·TTS(CPU) 서비스 HTTP 클라이언트. STT 입력은 16kHz mono PCM16, 출력은 글·단어 타임스탬프·처리 시간. 실패는 `SpeechError` 로 올려 엔진이 재시도/장애 처리를 정합니다.

`admission.py` — 놀이기구 번호표

- `tryAcquire` — 자리 하나 잡기. Redis 안에서 **끼어들 틈 없이 한 번에**(Lua) 세요.
- `release` — 자리 돌려주기. `reconcile` — 1분마다 숫자를 장부와 맞춰요.

> 🔗 **전문 설명** — 동시 면접 상한 N 을 Redis 카운터 `interview:active` 로 관리. `ACQUIRE_LUA` 가 `GET → 비교 → INCR` 을 서버 측에서 원자적으로 실행해 **TOCTOU 경쟁**을 없앱니다. 커밋 전 입장은 `interview:pending:{id}` (TTL 60초)로 표시해 1분 주기 `reconcile` 이 놓치지 않습니다(E10).

`coding.py` · `coding_bank.py` · `sandbox_runner.py` — 코딩테스트 담당

- `pickProblems` — 지원자마다 기초 1 · 응용 1 문제를 골라요.
- `runExamples` — [실행] 버튼: 예시 입력으로 돌려 보기.
- `finishCoding` — 제출(또는 시간 끝) → 숨겨진 테스트 채점 예약.
- `run` — 지원자 코드를 **유리 상자**(인터넷 끊김, 메모리 256MB, 10초) 안에서 돌려요.

> 🔗 **전문 설명** — 문제 배정은 지원 ID 기반 결정적 선택(같은 지원 = 같은 문제). 코드 실행은 api 가 직접 하지 않고 Celery `sandboxRunTask` 로 worker-sandbox 에 위임(api 컨테이너에 Docker 소켓 없음). 숨겨진 테스트 채점은 5분 중복 억제 키로 멱등 처리합니다.

`postings.py` · `posting_draft.py` — 공고 담당

- `validateForPublish` — 게시 전에 빈칸이 없나 확인. `updatePosting` — 고치기(게시 뒤엔 잠긴 칸 있음). `closePosting` — 마감.
- `start` / `generate` — AI 공고 초안. `biasWarnings` — 나이·성별·종교 같은 차별 표현 찾기.
- `buildDraftGraph` — AI 공고 초안을 **순서도**로 만들어요: ① 같은 직무의 예전 공고를 참고하고 → ② 초안을 쓰고 → ③ 칸마다 검사(차별 표현? 예전 공고 베끼기?) → 문제 있는 칸만 다시 써요.
- `startRevision` / `revise` — 관리자가 "이 칸만 다시 써 줘" 하면 그 칸만 고쳐 화면에 돌려줘요. 저장은 관리자가 [임시저장]을 눌러야 돼요.

> 🔗 **전문 설명** — 평가 기준은 버전 행(`EvaluationCriteria`)으로 쌓고 첫 응시 시작 시 잠급니다 (`locked`) — 이미 응시한 사람과 다른 기준으로 채점되지 않게. 게시 뒤 직무·기간 같은 칸은 수정 불가(409), 본문 수정은 `PostingRevision` 이력과 지원자용 요약(`revisionSummary`)을 남깁니다. AI 초안은 LangGraph `StateGraph(DraftState)` (retrieve → draft → check → rewrite ↺ → save / deliver / fail)로, 체크포인트 thread `posting-draft:{공고 id}` 에 처음 입력·참고 공고를 남겨 두었다가 한 칸 다시 쓰기 때 같은 thread 로 다시 실행합니다(C-15). 상태에는 지원자 개인정보가 없습니다.

`posting_questions.py` — 공고를 보고 질문 미리 만들어 두기 (2026-11-10)

- `schedule` → `generateForPosting` — 공고를 **게시하면** 뒤에서 질문 후보를 만들어요. 공고의 업무·자격 줄마다 직무 질문 1개(최대 6개), 회사 질문 3개(지원 동기 · 회사가 하는 일 · 조직 문화로 초점을 나눠서).
- `makeOne` — 질문 1개를 만들고 면접 질문과 같은 검사를 해요. 이미 비슷한 질문이 있으면 버려요.

- `loadWebCandidates` — 인터넷 면접 질문 자료를 참고해 새로 쓴 후보 파일을 불러와요(명령 `load_web_questions`).
- 모두 '검토 대기'로만 들어가요. 관리자가 질문 관리 화면에서 [추가]해야 면접에 쓰여요.

> 🔗 **전문 설명** — 게시 API(`publish`)가 커밋한 뒤 🐍 `asyncio` 작업으로 띄우고(`tasks` 집합에 참조 보관), 예외는 잡아서 로그만 남깁니다 — 부가 기능이 게시를 막지 않게. 프롬프트·검사는 `question_compose.ragPrompt` · `checkGenerated` 를 재사용하고, 이력서 칸은 '(없음)'이라 지원자 개인정보가 들어가지 않습니다. 중복은 정규화 글(`duplicate_group`) 일치와 임베딩 코사인 `BANK_DUPLICATE_SIMILARITY`(0.8)로 막습니다. 감사 기록 `question.posting_candidates` · `question.web_candidates`.

`login_block.py` — 경비원

- `recordFailure` — 로그인 실패 세기. 같은 곳에서 5번이면 차단. `release` — 관리자가 풀어 주기.

> 🔗 **전문 설명** — 키 = 클라이언트 IP 의 해시(`guardKey`). `X-Forwarded-For` 는 신뢰 프록시에서 온 요청일 때만 믿어 IP 위조를 막고(`clientIp`), 예외 대역(`loginExemptIps`)을 지원합니다. 집계 창 안 실패가 상한을 넘으면 `blocked_until` 까지 429, 관리자는 문의 코드로 해제(감사 기록).

`notify.py` — 우체부

- `create` — 🔔 알림 만들기. `sendNow` — 메일 1통 보내기. `retryOrFail` — 실패하면 다시, 그래도 안 되면 실패 기록.

> 🔗 **전문 설명** — 알림 행은 업무 트랜잭션 안에서 만들고, 메일은 **커밋 뒤** 큐에 넣는(transactional outbox 와 비슷한) 순서라 롤백된 알림이 메일로 나가지 않습니다. SMTP 실패는 `RETRY_DELAYS_SEC` 간격 재시도 후 `failed` 로 남겨 대시보드에서 재처리합니다.

`storage.py` · `recording.py` — 파일 창고와 녹화 담당

- `putObject` / `getObject` — 파일을 잠가서 넣고, 풀어서 꺼내요.
- `saveChunk` — 10초짜리 녹화 조각을 차례로 저장해요.

> 🔗 **전문 설명** — S3 호환(RustFS) 📦 `boto3` 클라이언트. 서버 측 암호화에 기대지 않고 **애플리케이션 계층에서 AES-GCM 암호화**하며, 객체 키를 AAD(추가 인증 데이터)로 묶어 다른 경로로 바꿔치기한 객체는 복호화 단계에서 거부됩니다.

`retention.py` · `withdrawal.py` — 청소부

- `purgeDue` — 보관 기간(결과 공개 +7일) 지난 데이터 지우기. `purgeRecordings` — 녹화는 3일.
- `archiveStats` — 지우기 전에 **통계 숫자만** 남겨 둬요.
- `withdrawApplication` / `withdrawAccount` — 동의 철회·탈퇴하면 바로 지워요.

> 🥖 **전문 설명** — 보관 기한(`purge_at` = 결과 공개 + 7일, 녹화 = 응시 끝 + 3일) 지난 데이터를 매일 03:00 삭제. 파일 먼저, DB 나중 순서로 고아 파일을 막고, 건별 실패는 다음 날 재시도합니다. 삭제 전 `archiveStats` 로 익명 집계만 보존하고 동의 기록은 증빙으로 남깁니다.

`question_bank.py` — 질문은행 사서

- `loadAll` — 엑셀 질문들을 장부에 넣어요. `dedupeExcel` — 뜻이 겹치는 질문을 정리해요.

> 🥖 **전문 설명** — 📦 `openpyxl` read-only 모드로 엑셀을 스트리밍 읽기. (시트, 행 번호) 키로 중복 적재를 막고, 다른 회사 이름·무관한 질문은 정규식으로 제외합니다. `dedupeExcel` 은 임베딩 생성 뒤 의미 유사 중복을 `excluded` 로 바꿉니다.

🎙️ 면접실 (`interview/`)

`interview/engine.py` — 면접 진행 직원 `LiveSession` (3장 장면 2·3)

- `serve` — 일 시작. `send` — 화면에 말 전하기.
- `onText` — 화면에서 온 쪽지(심장 박동, 화면 이탈, 답변 완료…)를 종류별로 처리.
- `onAway` — 화면 이탈: 1번째 경고, 2번째 취소. 3초 안에 온 신호는 한 번으로 셈.
- `watchdog` — 전화선 지키기. `judgeDeviceEnded` — 장치가 꺼지면 3초 기다렸다 판정.
- `pauseForFault` — 장애 중단. `endWith` — 응시 끝내기.

> 🥖 **전문 설명** — 연결당 1개의 `LiveSession` 이 상태(`idle → preparing → answering → processing → coding → ended`)를 가지며, `send` 는 🐍 `asyncio.Lock` 으로 동시 송신을 직렬화합니다. 화면 이탈은 `AWAY_MERGE_SEC = 3` 안의 신호를 1회로 병합(탭 숨김 + 포커스 잃음 이중 신호), 장치 종료는 `DEVICE_END_GRACE_SEC = 3` 뒤 판정해 unload 비콘이 먼저 도착할 시간을 줍니다(E18).

`interview/graph.py` — 면접 순서도

- `buildInterviewGraph` — 준비 → 질문 → 답변 → 꼬리질문 → 다음… 순서를 그림처럼 정해 둬요 (LangGraph).
- `buildFollowupGraph` — 꼬리질문 순서도. `validateFollowup` — 꼬리질문 검사.

> 🥖 **전문 설명** — 면접 진행을 LangGraph 그래프로 선언하고, 실제 부작용(DB·WebSocket)은 `Runner` 프로토콜 구현체(`LiveSession`)에 위임합니다 — 그래프 로직을 가짜 러너로 단위 테스트할 수 있습니다. 답변 대기는 📦 `interrupt()` 로 멈추고 체크포인트에서 재개, `RECURSION_LIMIT` 으로 무한 루프를 막습니다.

`interview/lifecycle.py` — 면접 기록원

- `logEvent` — 끊김·이탈 같은 일을 시각과 함께 적어요.
- `resumesLeft` — 남은 이어가기 횟수 (3번 – 지원자 쪽 끊김 수).

- `markInterrupted` · `finish` · `enterCoding` — 장애 중단, 끝, 코딩으로 넘어가기.

> 🔧 **전문 설명** — 응시 수명 주기의 부작용 모음: 이벤트 기록(`session_events`), 장애 중단 (`markInterrupted` — 지원자 원인 끊김이 `resume_limit` 에 닿으면 종료, 서버 원인 `SERVER_SIDE_REASONS` 는 세지 않음), 종료(`finish` — 자리 반환 + 채점 큐잉), 코딩 타이머 일시정지·재개.

`interview/registry.py` — 지금 일하는 직원 명단

- `register` / `get` — 지원서마다 지금 면접 중인 직원을 찾아요. `terminateMany` — 공고 마감 때 한꺼번에 끝내요.

> 🔧 **전문 설명** — 프로세스 메모리의 `dict[지원 ID, LiveSession]` . 같은 지원 건에 새 연결이 오면 이전 세션을 `replaced` 로 끝냅니다. api 프로세스 하나를 전제로 한 설계라, api 를 여러 개로 늘리려면 프로세스 사이에 공유되는 명부가 따로 필요합니다(현재 범위 밖).

`interview/checkpoint.py` — 저장 포인트 게임 세이브처럼, 면접 순서도가 **어디까지 했는지** 저장해 둬요.

> 🔧 **전문 설명** — 운영은 📦 `AsyncPostgresSaver` (psycopg 연결 풀), 테스트는 📦 `InMemorySaver` . `thread_id` = 지원 ID 라 재연결·서버 재시작 뒤에도 같은 그래프 상태에서 재개합니다. 응시가 끝나면 `deleteThread` 로 지웁니다.

🌙 뒷주방 (`worker.py` , `jobs.py`)

`worker.py` — 밤에도 일하는 작업자 (Celery) | 작업 | 언제 | 하는 일 | |---|---|---| | `evaluateTask` | 응시가 끝나면 | 채점 | | `sweepTask` | 5분마다 | 빠진 채점 다시 넣기, 결과 공개(`releaseDue`), 평가 지연 정리 | | `sendMailTask` | 알림이 생기면 | 메일 보내기 | | `remindersTask` | 매시 정각 | 마감 임박 안내 | | `purgeTask` | 매일 03:00 | 오래된 데이터 지우기 | | `purgeNoticeTask` | 매일 09:00 | 삭제 2일 전 관리자 안내 | | `purgeRecordingsTask` | 매시 30분 | 기한 지난 녹화 지우기 | | `sandboxRunTask` · `gradeCodingTask` | 코딩 실행·채점 때 | 유리 상자에서 코드 돌리기 |

> 🔧 **전문 설명** — Celery 앱(브로커·결과 백엔드 = Redis). 큐를 나눠 `evaluate` (채점, `-c 1`), `default` (메일·점검, `-c 2`), `sandbox` (코드 실행, `-c 2` , 이 작업자만 Docker 소켓 보유)로 격리합니다. 주기 작업은 `beat_schedule` (`sweep` 300초, `purge` 03:00 등). 채점은 Redis 잠금 키로 같은 건 중복 실행을 막고, AI 서버 다운은 실패로 세지 않고 `reschedule` 로 미룹니다.

`jobs.py` — 1분 알람

- `runForever` — 서버 안에서 1분마다: `sweepOrphans` (주인 없는 면접 정리), `releaseHeldSeats` (끊긴 지 10분 지난 자리 돌려주기), `closeExpiredPostings` (마감 시각 지난 공고 닫기), `reconcile` (자리 숫자 맞추기), `regradePending` (못 끝낸 코딩 채점 다시).

> 🔧 **전문 설명** — Celery 와 별개로 api 프로세스 안 🐍 `asyncio` 루프에서 60초마다 도는 가벼운 정리 작업. 각 주기를 try/except 로 감싸 한 번의 실패가 루프를 멈추지 않게 합니다. 진행 중 WebSocket 세션을 끊어야 하는 일(공고 자동 마감)이 있어 같은 프로세스에 둡니다.

🔧 관리 도구

`manage.py` — 설치할 때 쓰는 명령들. `init` (첫 설치·관리자 계정), `loadQuestions` (질문 엑셀 넣기), `embedQuestions` (질문 뜻을 숫자로).

> 🔧 **전문 설명** — `python -m app.manage <명령>` CLI. `init` 은 멱등(이미 있으면 건너뜀)이라 여러 번 돌려도 안전합니다. 관리자 초기 비밀번호는 `must_change_password` 가 켜진 채 만들어져 첫 로그인 때 변경을 강제합니다(`activeUser`).

`migrate.py` — 장부에 새 칸·규칙(UNIQUE 등)을 덧붙여요. **`devtools.py`** — 개발자가 잠긴 글을 서버 안에서만 풀어 보는 도구예요.

---

5. 프론트엔드 파일 소개 — 화면들

5-1. 화면 파일 (`frontend/app/`) — 주소 = 파일

주소	파일	무슨 화면?	중요한 조각
`/jobs`	`app/jobs/page.tsx`	공고 목록	`JobList` (검색·거르기)
`/jobs/[postingId]`	`app/jobs/[postingId]/page.tsx`	공고 상세	`JobDetail` ([지원하기])
`/login` , `/signup`	`app/login` , `app/signup`	로그인, 회원가입	`LoginForm` , `SignupForm`
`/apply/…/consent`	`app/apply/[applicationId]/consent`	① 동의	`Consent`
`/apply/…/resume`	`…/resume` , `…/resume/confirm`	② 이력서 고르기·확인	`ResumeMethod` , `Confirm`
`/apply/…/precheck`	`…/precheck`	③ 환경 점검	`PrecheckStep` → `Precheck`
`/apply/…/rules`	`…/rules`	④ 시작 안내	`Rules` ([면접 시작])
`/exam/[applicationId]`	`app/exam/[applicationId]/page.tsx`	**면접 화면**	`Exam` (가장 큰 화면, 600줄)
`/exam/…/resume`	`app/exam/[applicationId]/resume`	이어가기	`Resume`
`/mypage`	`app/mypage/page.tsx`	마이페이지	`Applications` , `Results` , `Account`
`/mypage/results/…`	`app/mypage/results/[applicationId]`	결과 상세	`ResultDetail`
`/admin/postings`	`app/admin/postings/…`	공고 관리·만들기	`PostingEditor` , AI 초안 `Chips` , 한 칸 다시 쓰기 `AiRewrite`

주소	파일	무슨 화면?	중요한 조각
`/admin/applicants`	`app/admin/applicants/…`	지원자 목록·상세	`Applicants`, `ApplicantDetail`, `Report`
`/admin/compare`	`app/admin/compare`	지원자 비교	`Radar` (레이더 그림)
`/admin/dashboard`	`app/admin/dashboard`	대시보드	`Bars` (막대그래프)
`/admin/questions`	`app/admin/questions/page.tsx`	질문 관리 — AI 가 만든 질문 검토 (출처·적용 범위 표시)	`Questions`
`/admin/settings`, `/admin/next-stage`		운영 설정, 다음 전형	

> 🔧 **전문 설명** — Next.js App Router 의 파일 시스템 라우팅: `app/**/page.tsx` 가 라우트, `[param]` 은 동적 세그먼트(📦 `useParams`). 페이지 기본 내보내기(`…Page`)는 📦 `<Suspense>` 경계로 📦 `useSearchParams` 를 쓰는 본문 컴포넌트를 감쌉니다. API 는 같은 출처 `/api/*` 를 `next.config` rewrites 로 FastAPI 에 넘겨 쿠키(HttpOnly, SameSite)와 CORS 문제를 피합니다.

`page.tsx` 안의 `…Page` 함수는 "이 주소로 오면 여기서 시작"하는 **입구**예요. 실제 내용은 같은 파일의 다른 조각(`Exam`, `Consent` …)에 있어요.

5-2. 같이 쓰는 조각 (`frontend/components/`)

파일	별명	중요한 것
`Providers.tsx`	가게 전체 방송실	`useApp` — 지금 로그인한 사람, 토스트 알림. 로그인이 끝나면(401) 알려 줘요
`Shell.tsx`	건물 뼈대	`Gnb` (위 메뉴), `useGuard` (역할 문지기), `MobileBanner` (모바일 안내)
`AdminShell.tsx`	관리자 사무실 뼈대	왼쪽 메뉴, 관리자만
`ApplyFlow.tsx`	지원 단계 안내원	`Stepper` (①②③④ 표시), 상태가 안 맞으면 맞는 단계로 돌려보내기
`Precheck.tsx`	환경 점검 기사	마이크·카메라·스피커·네트워크·AI 서버 점검. 요청을 **줄 세워 차례로** 보내요
`exam/Overlays.tsx`	면접 중 덮개 창	경고, 장애 중단(남은 이어가기 횟수), 종료 안내
`exam/Coding.tsx`	코딩테스트 화면	30초마다 자동 저장, [실행], [제출]
`PostingEditor.tsx`	공고 작성 3단계	`weightError` (가중치 합 100%), `criteriaError` (합격 기준 빈칸), `AiRewrite` (AI 에게 한 칸 다시 쓰기)
`ResumeForm.tsx`	이력서 입력 폼	흐릿하게 읽힌 칸 강조
`Ui.tsx`	레고 상자	`Button`, `Modal`, `Field`, `Spinner`, `ErrorBox`
`NotificationBell.tsx`	🔔 종	알림 목록·읽음·숨기기

> 🔧 **전문 설명** — 공통 컴포넌트는 Context(`Providers` 의 `useApp`, `ApplyFlow` 의 `useApplication`)로 데이터를 내려 prop drilling 을 줄입니다. 접근성: `Modal` 은 포커스 트랩· `aria-labelledby`, 응시 중 덮개는 ESC 로 닫히지 않음, 그래프(`Bars`, `Radar`)는 같은 값을 표로도 제공합니다. `Precheck` 는 서버 점검 요청을 Promise 체인 큐(`checkQueue`)로 직렬화해 늦게 온 응답이 통과 표시를 덮어쓰는 경쟁을 없앴습니다.

5-3. 도우미 (`frontend/lib/`)

파일	하는 일
`api.ts`	**모든 서버 요청의 문**. `api("/auth/login", {body})` 처럼 써요. 오류는 `{code, message}` 로 알려 줘요
`media.ts`	카메라·마이크·스피커. `openStream` (켜기), `routeToSpeaker` (고른 스피커로 소리 내기), `measureMbps` (인터넷 속도)
`pcm.ts`	답변 녹음기 `PcmRecorder` — 말하는 동안만 소리 조각을 서버로 보내요
`recording.ts`	카메라 녹화 — 10초 조각으로 잘라 올려요
`exam.ts`	면접 정보 보관, 창을 닫을 때 **'나가요' 쪽지**(`sendUnloadBeacon`) 보내기
`format.ts`	시각을 한국 시간으로 예쁘게 (`longDateTime` , `mmss` = 01:30)
`status.ts`	상태 이름표와 색 (`interviewing` → "면접 진행 중", `badge-blue`)

🔧 **전문 설명** — `api.ts` 는 🌐 `fetch` 래퍼(`credentials: "same-origin"` , `cache: "no-store"`)로 오류를 `{code, message}` 예외로 통일하고, `/auth/` 밖의 401 은 `UNAUTHORIZED_EVENT` 로 방송해 전역 로그아웃 처리를 합니다. 미디어는 🌐 `getUserMedia` (deviceId `ideal` 제약 — 장치를 빼도 다른 장치로 열림), 🌐 `AudioWorkletNode` (`/pcm-worklet.js`) 기반 PCM 캡처, 🌐 `MediaRecorder` (webm, 10초 timeslice), 스피커 지정은 🌐 `HTMLMediaElement.setSinkId` 를 씁니다.

---

6. 테스트 — 함수의 '사용 설명서'

함수가 맞게 움직이는지 궁금하면 **테스트**를 보세요. 테스트는 "이 함수는 이렇게 동작해야 해"라고 적어 둔 시험지예요.

- 백엔드: `backend/tests/test_….py` — 예: `test_scoring.py` 는 점수 계산, `test_device_end.py` 는 창 닫기 판정(E18), `test_question_text.py` 는 질문 글 검사, `test_posting_questions.py` 는 질문 적용 범위와 게시 때 질문 후보를 시험해요.
- 화면 전체: `e2e/tests/….spec.ts` — 진짜 브라우저로 지원부터 결과까지 눌러 봐요.
- 찾는 법: `Ctrl+Shift+F` 로 함수 이름을 `tests` 폴더에서 찾으면, 그 함수를 어떻게 쓰는지 예시가 나와요.

전체 테스트 파일 목록은 부록 C에 있어요.

🔧 **전문 설명** — 백엔드 테스트는 pytest + SQLite 메모리 DB + fakeredis 로 외부 의존 없이 돌고, LLM·STT 는 가짜 객체로 바꿉니다(TDD: 실패하는 테스트 먼저). E2E 는 Playwright(Chromium)가 실제 스택(docker compose)에 가짜 카메라·마이크(`--use-fake-device-for-media-stream`)로 접속해 사용자 흐름 전체를 검증합니다.

부록 — 전문가용 전체 목록

여기부터는 **모든 함수**를 빠짐없이 적은 사전입니다. 이름·줄 번호·인자·부르는 곳은 프로그램(Python `ast`)이 코드에서 자동으로 뽑았고, '하는 일'은 코드의 설명 주석 또는 함수 본문을 읽고 적었습니다. 부르는 곳은 import 를 따라가 **그 파일의 함수임이 확인된 호출만** 셉니다.

부록 표 읽는 법

칸	뜻
이름	`클래스.메소드` 또는 `바깥함수.안쪽함수` 로 적었습니다
종류	🌐 API = 주소가 붙은 요청 처리 함수 · ⏱ 작업 = Celery 작업 · 함수 · async 함수 · 메소드 · 안쪽 함수 · 클래스
줄	파일 안 줄 번호 (`Ctrl+G` 로 이동)
인자 → 반환	`(a, b) -> bool` 은 a, b 를 받아 참/거짓을 돌려준다는 뜻
부르는 곳	`모듈:함수` 줄임 표기. `r.` = routers, `s.` = services, `i.` = interview. 🧪 = 그 함수를 직접 부르는 테스트 파일 수
가져다 쓰는 모듈	파일마다 맨 위에 적은 줄. 그 파일의 `import` 를 프로그램이 읽어 🏠 우리 / 🐍 표준 / 📦 외부로 나눴습니다(1-6절). `` `fitz`(pymupdf) `` 는 코드에 쓰는 이름(설치 이름)입니다. 🧰 내장과 🌐 브라우저 기능은 import 가 없어 이 줄에 나오지 않습니다
부르는 곳이 `–`	우리 코드가 직접 부르지 않고 **틀(프레임워크)이 부르는** 경우입니다: FastAPI(API 함수), Celery(작업), LangGraph(그래프 노드), SQLAlchemy(`process_...`), 또는 이름 글자로 부르는 경우. API 함수는 그 주소를 쓰는 **화면 파일**을 함께 적었고, 그래도 `–` 이면 주소를 서버 응답으로 받아 쓰거나(음성·녹화 주소) 주소 일부를 변수로 만드는 경우입니다

부록 A. 백엔드 함수 전체 표 (huge/backend/app)

A-0. 폴더 지도

파일	줄	함수·클래스	역할
`config.py`	103	3	환경 변수와 운영 설정 기본값(SETTING_DEFAULTS)
`crypto.py`	70	11	민감 정보 암호화(AES-256-GCM)와 암호화 DB 칸 형식
`db.py`	28	3	DB 연결과 세션
`deps.py`	48	6	로그인·권한 검사(FastAPI Depends)
`devtools.py`	144	9	개발용 점검 명령(암호문 풀어 보기 등)
`domain/password_policy.py`	21	1	비밀번호 규칙
`domain/question_text.py`	70	3	AI 질문 글 검사(찌꺼기·두 가지 묻기·말끝·예/아니오)
`domain/scoring.py`	134	11	점수 계산 규칙(Decimal)
`domain/status.py`	65	5	응시 상태 말판과 끊김 판정
`errors.py`	31	6	API 오류 모양과 처리기
`interview/checkpoint.py`	78	5	LangGraph 체크포인트 저장소
`interview/engine.py`	794	54	면접 진행 엔진(WebSocket 한 건 = LiveSession)
`interview/graph.py`	195	33	면접·꼬리질문 순서도(LangGraph)

파일	줄	함수·클래스	역할
interview/lifecycle.py	132	13	응시 기록·이어가기 횟수·코딩 타이머
interview/registry.py	37	6	살아 있는 면접 세션 명부
jobs.py	84	5	api 안에서 1분마다 도는 정리 작업
main.py	124	10	FastAPI 앱 입구(라우터 연결, 시작/종료, 업로드 크기 검사)
manage.py	96	6	관리 명령(python -m app.manage init 등)
migrate.py	107	3	칸·제약 덧붙이기
models.py	528	34	DB 표 정의(SQLAlchemy)
routers/admin.py	459	27	관리자 API: 공고·지원자·메모·평가 지연
routers/admin_ops.py	209	7	관리자 API: 대시보드·운영 설정·메일 재처리·차단 해제
routers/admin_questions.py	66	6	관리자 API: 질문 관리
routers/applications.py	260	19	지원 API: 지원·동의·이력서
routers/auth.py	200	14	로그인·회원가입·비밀번호 API
routers/interview.py	321	22	면접 API: 환경 점검·시작·WebSocket·코딩·녹화
routers/me.py	122	8	마이페이지 API
routers/postings.py	59	3	공고 조회 API(지원자용)
schemas.py	217	32	요청 본문 모양(Pydantic)
security.py	72	7	비밀번호 해시와 로그인 세션
services/admission.py	87	6	동시 면접 자리(Redis Lua)
services/coding.py	100	6	코딩테스트 진행
services/coding_bank.py	158	2	코딩 문제 목록
services/evaluation.py	715	47	자동 채점 전체
services/llm.py	72	5	LLM 호출 공통
services/login_block.py	142	13	로그인 실패 IP 차단
services/notify.py	126	8	알림과 메일
services/pdf_box.py	113	6	PDF 읽기 격리 프로세스
services/posting_draft.py	506	43	AI 공고 초안
services/posting_questions.py	139	7	공고 기반·인터넷 참고 질문 후보
services/postings.py	216	14	공고 규칙
services/question_bank.py	219	11	질문은행 엑셀 읽기
services/question_compose.py	700	47	면접 질문 10개 만들기(RAG)
services/rag.py	165	18	임베딩과 벡터 검색
services/recording.py	93	7	카메라 녹화 조각 저장
services/resume_extract.py	435	34	이력서 읽기(PDF·OCR·LLM)
services/retention.py	173	6	보관 기간 지난 데이터 삭제
services/sandbox_runner.py	170	6	코드 격리 실행
services/speech.py	41	4	STT·TTS 호출
services/storage.py	72	6	파일 저장소(RustFS, 암호화)
services/withdrawal.py	36	2	동의 철회·회원 탈퇴
settings_store.py	29	3	운영 설정 읽기·쓰기
timeutil.py	30	4	한국 시각 도우미

파일	줄	함수·클래스	역할
`worker.py`	446	31	Celery 작업자(채점·메일·시간표)

A-1. 입구·설정·공통

app/config.py

역할: 환경 변수와 운영 설정 기본값(SETTING_DEFAULTS) — 환경 설정(.env)과 재시작 없이 바꾸는 설정값(app_settings)의 기본값.

가져다 쓰는 모듈 | 표준 `dataclasses` `os`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`Env`	클래스	11	``	환경 변수에서 읽은 실행 설정 묶음 (DB·Redis 주소, 암호화 키, AI 서버 주소 등)	config:loadEnv
`loadEnv`	함수	33	`() -> Env`	환경 변수를 읽어 Env 를 만든다. 값이 없으면 개발용 기본값(SQLite 등)	—
`defaultSetting`	함수	101	`(key)`	운영 설정 기본값을 돌려준다. 개발(dev)·운영(prod) 값이 다를 수 있다	settings_store:getSetting

app/crypto.py

역할: 민감 정보 암호화(AES-256-GCM)와 암호화 DB 칸 형식 — DB 민감 칸 암호화 (AES-256-GCM, 앱 레벨) — TRD 4-2 · 11장, 기술 아키텍처 8-3.

가져다 쓰는 모듈 | 우리 `config` | 표준 `base64` `hashlib` `hmac` `json` `os` | 외부 `cryptography` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`subjectHash`	함수	17	`(email) -> str`	동의 기록의 동의자 표시 (테이블 정의서 4-9). 이메일 목록 대입으로 되돌리지 못하게 서버 키로 HMAC.	migrate:fillConsentSubjects · r.applications:consent · 🧪 테스트 1개 파일
`ipHash`	함수	23	`(ip) -> str`	로그인 차단 표의 IP 표시. IPv4 는 경우의 수가 적어 일반 해시로는 되돌릴 수 있으므로 키를 섞는다.	s.login_block:guardKey · 🧪 테스트 1개 파일
`loadKey`	함수	29	`() -> bytes`	APP_ENCRYPTION_KEY 를 읽어 32바이트 키로 바꾼다. 없거나 길이가 틀리면 앱을 켜지 않는다	crypto:ipHash · crypto:subjectHash · s.storage · 🧪 테스트 1개 파일
`encryptText`	함수	41	`(plain) -> str`	글을 AES-256-GCM 으로 암호화하고 nonce 를 앞에 붙여 base64 로 돌려준다	crypto:process_bind_param · 🧪 테스트 1개 파일
`decryptText`	함수	46	`(token) -> str`	encryptText 의 반대. 앞 12바이트 nonce 를 떼어 복호화한다	crypto:process_result_value · devtools:decryptValue
`EncryptedText`	클래스	51	`(TypeDecorator)`	DB 칸 형식: 저장할 때 암호화, 읽을 때 복호화하는 글자 칸	devtools:encryptedModels · models

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`EncryptedText.process_bind_param`	메소드	55	`(self, value, dialect)`	DB에 쓰기 직전 암호화 (SQLAlchemy 가 부름)	—
`EncryptedText.process_result_value`	메소드	58	`(self, value, dialect)`	DB에서 읽은 직후 복호화 (SQLAlchemy 가 부름)	—
`EncryptedJson`	클래스	62	`(TypeDecorator)`	DB 칸 형식: JSON 을 글로 바꿔 암호화해 저장하는 칸	devtools:encryptedModels · models
`EncryptedJson.process_bind_param`	메소드	66	`(self, value, dialect)`	JSON → 글 → 암호화	—
`EncryptedJson.process_result_value`	메소드	69	`(self, value, dialect)`	복호화 → 글 → JSON	—

app/db.py

역할: DB 연결과 세션

가져다 쓰는 모듈 | 우리 `config` | 표준 `collections` | 외부 `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`Base`	클래스	9	`(DeclarativeBase)`	모든 DB 표 클래스(models.py)의 부모	devtools:encryptedModels · devtools:fieldText · main:lifespan · manage:init · 외 1곳 · 테스트 1개 파일
`makeEngine`	함수	13	`(url)`	DB 연결 엔진을 만든다. SQLite 면 스레드 검사 끄기, PostgreSQL 이면 연결 풀 10+10	—
`getDb`	함수	23	`() -> Iterator[Session]`	요청 하나 동안 쓸 DB 세션을 열고, 끝나면 닫는다 (FastAPI Depends 용)	deps:optionalUser · r.admin:answerAudio · r.admin:applicantDetail · r.admin:applicants · 외 66곳 · 테스트 2개 파일

app/deps.py

역할: 로그인·권한 검사(FastAPI Depends) — 인증·인가 의존성. 남의 데이터는 404 로 숨긴다 (ACC-06, 기술 아키텍처 10장 require_owner).

가져다 쓰는 모듈 | 우리 `db` `errors` `models` `security` | 외부 `fastapi` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`optionalUser`	함수	12	`(request, db) -> User | None`	쿠키의 로그인 토큰으로 사용자를 찾는다. 없으면 None	deps:currentUser · r.postings:getPosting · r.postings:listPostings
`currentUser`	함수	16	`(user) -> User`	로그인 필수. 없으면 401 LOGIN_REQUIRED	deps:activeUser · r.auth:me · r.auth:passwordChange

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`activeUser`	함수	22	`(user) -> User`	관리자 초기 비밀번호를 바꾸기 전에는 비밀번호 변경·로그아웃·내 정보 말고는 막는다.	deps:requireAdmin · deps:requireApplicant · r.me:hideNotification · r.me:hideReadNotifications · 외 2곳
`requireApplicant`	함수	29	`(user) -> User`	지원자만 통과. 아니면 403	r.applications:cancelUpload · r.applications:consent · r.applications:createApplication · r.applications:deleteSavedResume · 외 23곳
`requireAdmin`	함수	35	`(user) -> User`	관리자만 통과. 아니면 403	r.admin:answerAudio · r.admin:applicantDetail · r.admin:applicants · r.admin:close · 외 25곳
`ownedApplication`	함수	41	`(db, user, applicationId, forUpdate) -> Application`	내 지원 건만 꺼낸다. 남의 것이면 404. forUpdate 면 행을 잠근다(PostgreSQL)	r.applications:cancelUpload · r.applications:consent · r.applications:extractStatus · r.applications:getApplication · 외 10곳

app/devtools.py

역할: 개발용 점검 명령(암호문 풀어 보기 등) — 개발자 확인 명령. DB 의 암호화 칸(답변·이력서·판정 근거 등)을 서버 안에서만 풀어 본다.

가져다 쓰는 모듈 · 우리 `config` `crypto` `db` `models` · 표준 `binascii` `json` `sys` · 외부 `cryptography` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`pretty`	함수	26	`(value) -> str`	값을 사람이 읽기 좋게 바꾼다(JSON 들여쓰기, 빈 값 표시)	devtools:fieldText
`fieldText`	함수	34	`(db, table, column, rowId) -> str`	표 이름과 ID 로 한 줄을 읽는다. 암호화 칸은 앱의 칸 타입이 읽을 때 자동으로 푼다.	🧪 테스트 1개 파일
`decryptValue`	함수	47	`(token) -> str`	암호문 하나를 풀어 보여 준다. 모양이 틀리면 안내 문장	🧪 테스트 1개 파일
`answersText`	함수	60	`(db, applicationId) -> str`	한 지원 건의 질문·답변을 순서대로 풀어 글로 만든다	🧪 테스트 1개 파일
`encryptedModels`	함수	77	`() -> list`	암호화 칸을 가진 표 목록을 찾는다	devtools:plainRows
`asText`	함수	83	`(value) -> str | None`	값을 글로 바꾼다(dict/list 는 JSON)	devtools:plainRows
`plainRows`	함수	91	`(db) -> dict[str, tuple[list[str], list[list]]]`	암호화 칸이 있는 표마다 (칸 이름들, 줄들). 앱의 칸 타입이 읽을 때 풀어 주므로 모든 값이 평문이다.	devtools:makePlain · 🧪 테스트 1개 파일
`makePlain`	함수	102	`(db) -> str`	dev_plain 스키마에 평문 사본 표를 새로 만든다. 지금 시점의 사본이라 원본이 바뀌면 다시 돌린다.	🧪 테스트 1개 파일

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`dropPlain`	함수	121	`() -> str`	개발용으로 만든 평문 사본 스키마를 지운다	—

app/errors.py

역할: API 오류 모양과 처리기 — 오류 응답은 {"code", "message"} 한 가지 모양 (TRD 4-3, API 명세서 2장).

가져다 쓰는 모듈 | 외부 `fastapi`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`ApiError`	클래스	8	`(Exception)`	화면에 보낼 오류(상태 코드·code·message)를 담는 예외	deps:activeUser · deps:currentUser · deps:requireAdmin · deps:requireApplicant · 외 42곳 · 테스트 1개 파일
`ApiError.__init__`	메소드	9	`(self, status, code, message)`	상태 코드·code·message 저장	—
`notFound`	함수	15	`() -> ApiError`	404 NOT_FOUND 오류를 만든다	deps:ownedApplication · r.admin:answerAudio · r.admin:applicantDetail · r.admin:createMemo · 외 24곳
`apiErrorHandler`	async 함수	19	`(_request, exc)`	ApiError 를 {code, message} JSON 응답으로 바꾼다	main · 테스트 1개 파일
`unexpectedErrorHandler`	async 함수	23	`(_request, _exc)`	원인은 응답에 넣지 않는다. 예외는 이 뒤에 서버 로그로 남는다 (Starlette 가 다시 던짐)	main
`validationErrorHandler`	async 함수	28	`(_request, exc)`	입력 검사 실패를 422 VALIDATION_ERROR 한 줄 메시지로 바꾼다	main · 테스트 1개 파일

app/main.py

역할: FastAPI 앱 입구(라우터 연결, 시작/종료, 업로드 크기 검사) — Huge API — Team AIIT. 화면은 Next.js 가 /api/* 를 이 서버로 넘겨 같은 출처로 부른다 (TRD 3장).

가져다 쓰는 모듈
우리 `db` `domain` `errors` `interview` `models` `routers` `services.postings` `services` `settings_store` `timeutil`
표준 `asyncio` `contextlib` `logging` `re` | 외부 `fastapi` `pydantic`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`warmUp`	async 함수	29	`() -> None`	첫 LLM 호출은 프롬프트 처리에 45초가 걸렸다(2026-10-30 실측). 첫 지원자가 기다리지 않게 미리 부른다.	main:lifespan
`warmUp.Ping`	클래스	33	`(BaseModel)`	AI 서버 예열 응답 모양	—

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`recoverAfterRestart`	함수	43	`() -> None`	재시작 전 '질문 구성 중'이던 지원은 다시 구성하고, 끊긴 이력서 추출은 실패로 두고, Redis 자리 수를 DB 에 맞춘다.	main:lifespan · 🧪 테스트 1개 파일
`spawn`	함수	62	`(coro) -> None`	백그라운드 작업을 띄우고 끝날 때까지 참조를 잡아 둔다	main:lifespan
`lifespan`	async 함수	69	`(_app)`	앱 시작 때: 표 만들기·칸 추가·직무 코드 넣기·정리 반복 시작 / 종료 때 정리	—
`uploadLimitBytes`	함수	89	`() -> int`	업로드 상한(설정 MB + 여유) 바이트	main:**call**
`UploadSizeGuard`	클래스	94	``	이력서 업로드 요청이 상한보다 크면 본문을 읽기 전에 거절하는 미들웨어	🧪 테스트 1개 파일
`UploadSizeGuard.__init__`	메소드	100	`(self, inner)`	감쌀 앱 저장	—
`UploadSizeGuard.__call__`	메소드	103	`(self, scope, receive, send)`	Content-Length 를 보고 없거나 크면 411/413 으로 거절	—
`health`	🌐 API	123	`()`	`GET /health`	—

app/manage.py

역할: 관리 명령(python -m app.manage init 등) — 운영 명령. python -m app.manage <명령>

가져다 쓰는 모듈

- 🏠 **우리** `config` `db` `domain.scoring` `migrate` `models` `security` `services.posting_questions` `services.postings` `services.question_bank` `services.rag` `timeutil`
- 🐍 **표준** `datetime` `os` `sys`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`init`	함수	27	`(db) -> None`	처음 설치: 표 만들기, 직무 코드, 관리자 계정 1개	—
`loadQuestions`	함수	42	`(db) -> None`	엑셀 질문은행을 DB 에 넣는다	—
`samplePosting`	함수	46	`(db) -> None`	시험용 예시 공고를 만든다	—
`embedQuestions`	함수	64	`(db) -> None`	질문은행 문장을 임베딩해 벡터를 저장한다	—
`dedupeQuestions`	함수	71	`(db) -> None`	엑셀 질문 중 같은 문장을 묶는다	—
`loadWebQuestions`	함수	75	`(db, postingId) -> None`	명령 load_web_questions <공고 id>: 인터넷 참고 후보 파일(WEB_QUESTIONS_PATH)을 검토 대기로 넣는다	—

app/migrate.py

역할: 칸·제약 덧붙이기 — 기존 표에 새 칸 넣기. create_all 은 이미 있는 표를 바꾸지 않으므로 PostgreSQL 에서만 직접 더한다.

가져다 쓰는 모듈 표준 `logging` 외부 `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`fillConsentSubjects`	함수	62	`(engine) -> None`	동의자 표시(subject_hash)가 빈 행을 채운다. 해시는 서버 비밀키 HMAC 이라 이메일 목록으로 되돌릴 수 없다.	migrate:addColumns
`addChecks`	함수	73	`(engine) -> None`	상태 칸 CHECK 제약(models.CHECK_VALUES). 이미 있으면 건너뛴다. 기존 값이 어긋나면 그 제약만 빼고 경고한다 (서비스는 뜬다).	migrate:addColumns
`addColumns`	함수	88	`(engine) -> None`	PostgreSQL 에 새 칸·제약을 덧붙인다 (이미 있으면 건너뜀)	main:lifespan · manage:init

app/security.py

역할: 비밀번호 해시와 로그인 세션 — 비밀번호 해시(Argon2id)와 로그인 세션(HttpOnly 쿠키, 2시간 미끄러짐 만료) — TRD 11장.

가져다 쓰는 모듈 우리 `models` `settings_store` `timeutil` 표준 `datetime` `hashlib` `secrets` 외부 `argon2` (argon2-cffi) `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`hashPassword`	함수	19	`(password) -> str`	비밀번호를 Argon2 로 해시	manage:init · r.auth · r.auth:passwordChange · r.auth:passwordResetConfirm · 외 1곳 · 테스트 1개 파일
`verifyPassword`	함수	23	`(passwordHash, password) -> bool`	비밀번호가 해시와 맞는지(틀리거나 형식 오류면 False)	r.auth:login · r.auth:passwordChange · r.auth:withdrawMe
`tokenHash`	함수	30	`(token) -> str`	로그인 토큰의 SHA-256 (DB 에는 토큰 대신 이것만 저장)	r.auth:passwordChange · r.auth:passwordResetConfirm · r.auth:sendResetLink · security:createSession · 외 2곳 · 테스트 1개 파일
`createSession`	함수	34	`(db, user) -> str`	새 로그인 토큰을 만들고 만료 시각과 함께 저장	r.auth:login
`resolveSession`	함수	42	`(db, token, touch) -> User | None`	토큰으로 사용자를 찾는다. 만료면 None, touch 면 만료 연장	deps:optionalUser · r.interview:beaconEvent · r.interview:interviewSocket
`dropSession`	함수	54	`(db, token) -> None`	로그아웃: 토큰 행 삭제	r.auth:logout
`safeNext`	함수	66	`(nextPath) -> str | None`	로그인 복귀 경로는 내부 경로 허용 목록만 받는다 (ACC-04). 화면 라우트 기준 (화면정의서 Tech-16).	r.auth:login

app/settings_store.py

역할: 운영 설정 읽기·쓰기 — app_settings: 재시작 없이 바꾸는 설정값 (TRD 14장). 행이 없으면 프로필 기본값.

가져다 쓰는 모듈 | 우리 `config` `models` | 외부 `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`getSetting`	함수	9	`(db, key)`	운영 설정 값(DB 에 없으면 기본값)	i.engine:**init** · i.engine:sendCodingPhase · i.lifecycle:codingStarted · i.lifecycle:enterCoding · 외 19곳 · 테스트 1개 파일
`setSetting`	함수	16	`(db, key, value, actorId) -> None`	운영 설정을 바꾸고 변경 기록을 남긴다	r.admin:putSetting
`allSettings`	함수	28	`(db) -> dict`	모든 운영 설정	r.admin:getSettings

app/timeutil.py

역할: 한국 시각 도우미 — 모든 시각 계산은 Asia/Seoul (기술 아키텍처 11장).

가져다 쓰는 모듈 | 표준 `datetime` `zoneinfo`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`now`	함수	9	`() -> datetime`	지금 한국 시각(KST)	i.engine:beginQuestion · i.engine:onAudio · i.engine:onWarnAck · i.engine:saveAnswer · 외 75곳 · 테스트 17개 파일
`asKst`	함수	13	`(value) -> datetime | None`	시각을 KST 로 맞춘다(SQLite 처럼 시간대 없는 값은 KST 로 본다)	i.engine:watchdog · i.lifecycle:codingRemaining · i.lifecycle:pauseCodingTimer · i.lifecycle:recentEventTypes · 외 34곳 · 테스트 1개 파일
`iso`	함수	21	`(value) -> str | None`	시각을 ISO 글로(초까지)	i.engine:judgeConnectionLoss · i.engine:pauseForFault · r.admin:applicantDetail · r.admin:applicants · 외 23곳
`releaseAtFor`	함수	26	`(closeAt, releaseHour) -> datetime`	결과 공개 시각 = 마감일 다음 날 10:00 (RPT-02).	manage:samplePosting · r.admin:publish · s.evaluation:releaseSchedule · s.postings:closePosting · 외 1곳 · 테스트 1개 파일

A-2. domain — 규칙표 (DB·AI 없음)

app/domain/password_policy.py

역할: 비밀번호 규칙 — 비밀번호 규칙 (PRD 5-1, TRD 11장). 가입·변경·재설정이 같은 함수를 쓴다.

가져다 쓰는 모듈 표준 `re`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`checkPassword`	함수	9	`(password, email) -> list[str]`	어긴 규칙의 안내 문구 목록. 비어 있으면 통과.	r.auth:policyError · 🧪 테스트 1개 파일

app/domain/question_text.py

역할: AI 질문 글 검사 — 본질문(T2·T3)과 꼬리질문(T4)이 같이 쓰는 규칙. 찌꺼기·답변 방식 지시·두 가지 묻기·질문 아님·어색한 말끝·100자, 예/아니오 질문 판별 (CONFLICTS C-16·C-17)

가져다 쓰는 모듈 표준 `re`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`isYesNo`	함수	45	`(text) -> bool`	예/아니오로 답할 수 있는 질문인가 (의문사가 없고 '설명해 주세요' 같은 부탁 꼴도 아님)	i.engine:followupPrompt · i.engine:yesNoFollowup · 🧪 테스트 1개 파일
`cleanQuestion`	함수	50	`(text) -> str`	질문 끝에 새어 나온 따옴표·괄호·JSON 조각을 떼고 앞뒤 빈칸을 지운다	i.graph:decide · i.graph:validateFollowup · s.posting_questions:makeOne · s.question_compose:check · 외 1곳 · 🧪 테스트 1개 파일
`textProblem`	함수	54	`(text) -> str | None`	질문 글의 문제 코드를 돌려준다 (foreign_script · junk_text · meta_phrase · compound_question · not_question · awkward_ending · too_long). 괜찮으면 None	i.graph:validateFollowup · s.question_compose:checkGenerated · 🧪 테스트 1개 파일

app/domain/scoring.py

역할: 점수 계산 규칙(Decimal) — 점수 계산 (PRD 5-7 EVL-02~04, COD-06). api 는 조회용, 2단계 worker 는 확정용으로 같은 코드를 쓴다.

가져다 쓰는 모듈 표준 `decimal` `difflib` `re`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`roundScore`	함수	25	`(value) -> Decimal`	점수를 소수 둘째 자리로 반올림 (ROUND_HALF_UP)	domain.scoring:codingScore · domain.scoring:interviewScore · domain.scoring:itemScores · domain.scoring:totalScore
`validateWeights`	함수	29	`(weights) -> str | None`	틀린 이유를 돌려주고, 맞으면 None. 5 단위 · 항목마다 5 이상 · 합계 100 (JOB-04).	s.postings:applyCriteria · 🧪 테스트 1개 파일
`validateCodingRatio`	함수	42	`(ratio) -> bool`	코딩 반영 비율이 10~90 사이 10 단위인지 검사	s.postings:applyCriteria · 🧪 테스트 1개 파일

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`itemScores`	함수	46	`(questionScores) -> dict[str, Decimal]`	항목 점수 = 그 항목을 본 질문들의 점수 평균 ÷ 5 × 100 (EVL-03).	s.evaluation:computeScores · 🧪 테스트 1개 파일
`interviewScore`	함수	61	`(items, weights) -> Decimal`	AI 면접 점수 = Σ(항목 점수 × 항목 비율).	s.evaluation:computeScores · 🧪 테스트 1개 파일
`codingScore`	함수	66	`(problems) -> Decimal`	코딩테스트 점수 = 문제별 (통과 수 ÷ 전체 수)의 평균 × 100. 2문제 같은 배점 (COD-06).	s.evaluation:computeScores · 🧪 테스트 1개 파일
`totalScore`	함수	74	`(interview, coding, codingEnabled, codingRatio) -> Decimal`	총점 (EVL-04). 코딩테스트를 안 쓰면 AI 면접 점수가 곧 총점.	s.evaluation:computeScores · 🧪 테스트 1개 파일
`normalizeForQuote`	함수	86	`(text) -> str`	근거 대조용: 띄어쓰기·문장부호를 지운 글	domain.scoring:matchQuote · domain.scoring:quoteInAnswer · i.engine:isEmptyAnswer · s.evaluation:checkT5 · 외 4곳
`quoteInAnswer`	함수	90	`(quote, answer) -> bool`	근거 문장이 실제 답변에 있는지 (EVL-02). 띄어쓰기·문장부호 차이는 무시한다.	🧪 테스트 1개 파일
`matchQuote`	함수	96	`(quote, answer, minRatio) -> str | None`	AI 가 따온 근거와 맞는 답변 속 구절을 답변 글 그대로 돌려준다. 없으면 None.	s.evaluation:checkT5 · 🧪 테스트 1개 파일
`closestSpan`	함수	118	`(q, norm, minRatio) -> tuple[int, int] | None`	가장 긴 공통 부분을 기준으로 앞뒤 길이를 조금씩 바꿔 가장 비슷한 구간을 찾는다.	domain.scoring:matchQuote

app/domain/status.py

역할: 응시 상태 말판과 끊김 판정 — 응시 상태 전이 (TRD 7-1, 기술 아키텍처 6-4 상태도). 상태 원장은 applications.attempt_status.

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`TransitionError`	클래스	26	`(Exception)`	허용되지 않은 상태 이동일 때 나는 오류	domain.status:transition · 🧪 테스트 1개 파일
`transition`	함수	30	`(current, target) -> str`	ALLOWED 표에 있는 이동이면 새 상태를, 없으면 TransitionError	i.lifecycle:enterCoding · i.lifecycle:finish · i.lifecycle:markInterrupted · r.applications:submitResume · 외 2곳 · 🧪 테스트 2개 파일
`statusOnClose`	함수	36	`(current) -> str | None`	공고 마감 시각의 처리 (INT-10, D-04). 바꿀 필요가 없으면 None.	s.postings:closePosting · 🧪 테스트 1개 파일
`nextAction`	함수	45	`(status, released) -> str | None`	마이페이지 카드 버튼 (API 명세서 9장 next_action, 화면정의서 §3.4).	r.me:myApplications · 🧪 테스트 1개 파일
`judgeDisconnect`	함수	58	`(recentEventTypes) -> str`	끊기기 직전 unload_lookback_sec 안의 기록으로 판단 (TRD 7-2). 원인 모름은 지원자에게 유리하게 '장애'.	i.engine:judgeConnectionLoss · i.engine:judgeDeviceEnded · 🧪 테스트 1개 파일

A-3. models · schemas — 표와 요청 모양

app/models.py

역할: DB 표 정의(SQLAlchemy) — DB 테이블 (1단계 범위). 칸 이름은 API 명세서·TRD 4-1 의 이름을 따른다(snake_case, DB 규칙 예외).

가져다 쓰는 모듈 | 우리 `crypto` `db` `timeutil` | 표준 `datetime` `uuid` | 외부 `sqlalchemy`

클래스	줄	DB 표	칸(일부)
`User`	18	`users`	id, email, password_hash, name, phone, role, must_change_password, failed_logins, locked_until …
`LoginSession`	35	`login_sessions`	token_hash, user_id, expires_at, created_at
`LoginBlock`	43	`login_blocks`	ip_hash, fail_count, window_started_at, code, blocked_at, blocked_until
`JobCode`	54	`job_codes`	code, name, status, sort_order
`JobPosting`	62	`job_postings`	id, status, source, title, job_code, company_name, external_id, company_intro, duties …
`EvaluationCriteria`	91	`evaluation_criteria`	id, posting_id, version, locked, locked_at, weights, coding_enabled, coding_ratio, pass_score …
`PostingRevision`	113	`posting_revisions`	id, posting_id, revised_at, field, before, after, summary
`Application`	124	`applications`	id, user_id, posting_id, attempt_status, analysis_status, release_status, criteria_version, resume_method, resume_snapshot …
`Consent`	178	`consents`	id, application_id, subject_hash, consent_version, items, agreed_at, withdrawn_at
`ResumeUpload`	190	`resume_uploads`	id, user_id, application_id, file_key, file_name, content_type, stage, method, pages_text …
`SavedResume`	209	`saved_resumes`	user_id, resume, name, updated_at
`Question`	217	`questions`	id, source, kind, category, text, sheet, row_no, duplicate_group, status …
`QuestionUsage`	237	`question_usage`	posting_id, question_id, use_count
`InterviewQuestion`	244	`interview_questions`	id, application_id, seq, kind, answer_order, qtype, text, source_question_id, source …
`Answer`	267	`answers`	id, application_id, interview_question_id, transcript, words, audio_key, audio_sec, stt_elapsed_sec, started_at …
`InterviewSession`	283	`interview_sessions`	id, application_id, user_id, recording_segment, created_at, ended_at
`Recording`	293	`recordings`	id, application_id, segment_no, file_key, last_chunk_no, size_bytes, started_at, last_chunk_at, expires_at
`SessionEvent`	311	`session_events`	id, application_id, session_id, type, detail, judged, seq, at
`CodingSubmission`	323	`coding_submissions`	id, application_id, problem_no, problem_key, code, saved_at, paste_blocked, final, passed …
`AppSetting`	340	`app_settings`	key, value, updated_by, updated_at

클래스	줄	DB 표	칸(일부)
`DeletionLog`	348	`deletion_logs`	id, application_id, user_id, cause, scope, deleted_items, deleted_at
`StatsArchive`	360	`stats_archive`	id, posting_id, posting_title, month, applications, taken, decided, passed, total_sum …
`AuditLog`	383	`audit_logs`	id, actor_id, action, target, detail, at
`Result`	397	`results`	id, application_id, criteria_id, item_points, interview_score, coding_score, total_score, pass_score, ai_decision …
`ItemScore`	430	`item_scores`	id, application_id, interview_question_id, eval_item, score, quote, quote_verified, regenerated, scored_at
`Highlight`	445	`highlights`	id, application_id, interview_question_id, kind, eval_item, start_ms, end_ms, quote
`Memo`	458	`memos`	id, application_id, author_id, body, created_at, updated_at
`Notification`	469	`notifications`	id, user_id, application_id, kind, title, link, read_at, hidden_at, email_status …

함수	줄	인자 → 반환	하는 일	부르는 곳
`newId`	14	`() -> str`		—
`checkName`	515	`(table, column) -> str`		migrate:addChecks
`checkSql`	519	`(column, values) -> str`		migrate:addChecks

app/schemas.py

역할: 요청 본문 모양(Pydantic) — 요청 형식. 모델에 없는 칸이 오면 거절한다 (extra="forbid", TRD 4-3).

가져다 쓰는 모듈 | 표준 `datetime` `typing` | 외부 `pydantic`

클래스	줄	받는 칸
`Strict`	9	
`SignupAgreements`	13	age_14_plus, terms, privacy
`SignupRequest`	19	name, email, password, phone, agreements
`LoginRequest`	27	email, password, next
`PasswordChangeRequest`	33	current_password, new_password
`PasswordResetRequest`	38	email
`PasswordResetConfirm`	42	token, new_password
`WithdrawRequest`	47	password, confirm_text
`MePatch`	52	name, phone
`ManualDecisionIn`	57	decision, reason
`MemoIn`	62	body
`CriteriaIn`	66	weights, coding_enabled, coding_ratio, pass_score

클래스	줄	받는 칸
`PostingIn`	73	title, job_code, company_name, company_intro, duties, requirements, preferred, employment_type, location, headcount …
`DraftReviseRequest`	90	section, note, current
`DraftAiRequest`	97	job_code, career_level, career_years, employment_type, location, headcount, open_at, close_at, duties, skills …
`ApplicationCreate`	121	posting_id
`ConsentItems`	125	data_collection, ai_decision, share_with_admin, retention, exam_rules
`ConsentRequest`	133	consent_version, items
`EducationIn`	138	school, major, degree, start, end
`CareerIn`	146	company, role, start, end, description
`ProjectIn`	154	name, period, description, skills
`CertificateIn`	161	name, acquired
`ResumeIn`	166	name, education, career, projects, skills, certificates, cover_letter
`ResumeSubmit`	176	method, upload_id, resume, edited_fields
`Browser`	183	name, version
`Devices`	188	mic, camera, speaker
`PrecheckRequest`	194	browser, devices, network_mbps
`CodeRun`	200	problem_no, code
`CodeAutosave`	205	problem_no, code, paste_blocked
`CodeItem`	211	problem_no, code
`CodeSubmit`	216	codes

A-4. routers — 카운터 (API 주소)

app/routers/admin.py

역할: 관리자 API: 공고·지원자·메모·평가 지연 — 관리자 API (API 명세서 4장 관리용 · 10장). 결과·판정을 바꾸는 주소는 두지 않는다 (AC-14).

가져다 쓰는 모듈

우리 `config` `db` `deps` `errors` `interview.checkpoint` `interview.registry` `models` `schemas` `services.evaluation` `services.posting_draft` `services.posting_questions` `services.postings` `services.recording` `services.storage` `settings_store` `timeutil` `worker`

외부 `fastapi` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`getPostingOr404`	함수	40	`(db, postingId) -> JobPosting`	공고를 찾고 없으면 404	r.admin:applicants · r.admin:close · r.admin:deletePosting · r.admin:draftAi · 외 5곳
`listPostings`	API	48	`(status, db, _admin)`	`GET /admin/postings`	화면 admin/applicants · 화면 admin/next-stage · 화면 admin/postings/ai · 화면 admin/postings · 외 5곳
`createPosting`	API	65	`(body, db, admin)`	`POST /admin/postings`	화면 admin/applicants · 화면 admin/next-stage · 화면 admin/postings/ai · 화면 admin/postings · 외 5곳
`getPosting`	API	72	`(postingId, db, _admin)`	`GET /admin/postings/{postingId}`	화면 admin/postings/[postingId]/edit · 화면 admin/postings/ai · 화면 admin/postings/new · 화면 admin/postings · 외 1곳
`patchPosting`	API	82	`(postingId, body, db, _admin)`	`PATCH /admin/postings/{postingId}`	화면 admin/postings/[postingId]/edit · 화면 admin/postings/ai · 화면 admin/postings/new · 화면 admin/postings · 외 1곳
`draftAi`	API	89	`(postingId, body, db, admin)`	`POST /admin/postings/{postingId}/draft-ai` — AI 공고 초안 (S-R02). 정해진 값은 바로 복사하고, 문장은 뒤에서 AI 가 채운다 (G-06).	화면 admin/postings/ai
`draftAiStatus`	API	120	`(postingId, db, _admin)`	`GET /admin/postings/{postingId}/draft-ai/status`	화면 admin/postings/ai · 화면 components/PostingEditor.tsx
`draftAiRevise`	API	128	`(postingId, body, db, admin)`	`POST /admin/postings/{postingId}/draft-ai/revise` — AI 초안의 한 칸을 관리자 요청대로 다시 쓴다. 결과는 status 의 revision 으로 주고 DB 에는 쓰지 않는다 (2026-11-10).	화면 components/PostingEditor.tsx
`publish`	API	146	`(postingId, db, admin)`	`POST /admin/postings/{postingId}/publish` — 공고를 게시하고, 커밋 뒤 공고 기반 질문 후보 만들기를 뒤에서 시작한다 (posting_questions.schedule)	화면 components/PostingEditor.tsx

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`close`	🌐 API	162	`(postingId, db, admin)`	`POST /admin/postings/{postingId}/close`	화면 admin/postings
`deletePosting`	🌐 API	174	`(postingId, db, _admin)`	`DELETE /admin/postings/{postingId}`	화면 admin/postings/[postingId]/edit · 화면 admin/postings/ai · 화면 admin/postings/new · 화면 admin/postings · 외 1곳
`score`	함수	186	`(value) -> str | None`	점수를 '00.00' 글로(없으면 None)	r.admin:applicants · r.admin:resultView
`applicants`	🌐 API	194	`(postingId, page, size, result, analysis_status, sort, db, _`	`GET /admin/postings/{postingId}/applicants` — 지원자 목록 (ADM-01). 분석 전 행은 점수·decision 이 null. 결정을 바꾸는 입력은 없다.	화면 admin/applicants
`resultView`	함수	233	`(r) -> dict | None`	평가 결과를 화면용 dict 로	r.admin:applicantDetail
`applicantDetail`	🌐 API	246	`(applicationId, db, admin)`	`GET /admin/applicants/{applicationId}`	화면 admin/applicants/[applicationId] · 화면 admin/applicants · 화면 admin/compare · 화면 admin/dashboard · 외 1곳
`memoView`	함수	306	`(m) -> dict`	메모를 화면용 dict 로	r.admin:applicantDetail · r.admin:createMemo · r.admin:updateMemo
`memoBody`	함수	310	`(body) -> str`	메모 글 앞뒤 공백 제거, 비면 400	r.admin:createMemo · r.admin:updateMemo
`getMemoOr404`	함수	317	`(db, memoId) -> Memo`	메모를 찾고 없으면 404	r.admin:deleteMemo · r.admin:updateMemo
`createMemo`	🌐 API	326	`(applicationId, body, db, admin)`	`POST /admin/applicants/{applicationId}/memo` — 메모 본문은 감사 로그에 넣지 않는다 (지원자 개인 평가 내용)	화면 admin/applicants/[applicationId]
`updateMemo`	🌐 API	339	`(memoId, body, db, admin)`	`PATCH /admin/memos/{memoId}`	화면 admin/applicants/[applicationId]
`deleteMemo`	🌐 API	349	`(memoId, db, admin)`	`DELETE /admin/memos/{memoId}`	화면 admin/applicants/[applicationId]
`reprocess`	🌐 API	357	`(applicationId, db, admin)`	`POST /admin/analysis/{applicationId}/reprocess` — '확인 필요' 건을 같은 응시 자료로 처음부터 다시 돌린다. 결과를 사람이 고치지는 않는다 (EVL-05·08).	화면 admin/applicants/[applicationId] · 화면 admin/dashboard
`manualDecision`	🌐 API	380	`(applicationId, body, db, admin)`	`POST /admin/analysis/{applicationId}/decision` — 자동 채점을 끝내지 못한 건만 관리자가 합격·불합격과 사유를 정한다. 자동 채점 결과는 바꿀 수 없다 (CONFLICTS U8).	화면 admin/applicants/[applicationId]

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`getSettings`	API	402	`(db, _admin)`	`GET /admin/settings` — 현재 N · 지금 자리를 잡은 면접 수(AI 면접 진행 + 끊긴 지 10분 이내 장애 중단) · 최근 변경 (API 명세서 10장).	화면 admin/settings · 화면 components/AdminShell.tsx
`putSetting`	API	416	`(key, body, db, admin)`	`PUT /admin/settings/{key}`	화면 admin/settings
`answerAudio`	API	438	`(answerId, db, admin)`	`GET /admin/media/answers/{answerId}`	—
`recordingVideo`	API	449	`(applicationId, segmentNo, db, admin)`	`GET /admin/media/recordings/{applicationId}/{segmentNo}` — 녹화 다시 보기. 보관 기한(응시 끝 + 3일)이 지났으면 410. 재생할 때마다 감사 로그 (API 명세서 2-6).	—

app/routers/admin_ops.py

역할: 관리자 API: 대시보드·운영 설정·메일 재처리·차단 해제 — 관리자 2단계-B API: 지원자 비교 · 다음 전형 대상자 · 통계 · 운영 현황 (API 명세서 10장, 화면정의서 S-R07~S-R09).

가져다 쓰는 모듈 · 우리 `db` `deps` `domain` `errors` `models` `services` `timeutil` · 표준 `collections` `datetime` `decimal` `re` · 외부 `fastapi` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`score`	함수	31	`(value) -> str | None`	점수를 '00.00' 글로	r.admin_ops:compare · r.admin_ops:passed · r.admin_ops:stats
`maskEmail`	함수	38	`(email) -> str`	이메일 앞 2글자만 보이게 가린다	r.admin_ops:opsStatus
`compare`	API	44	`(ids, db, admin)`	`GET /admin/applicants/compare` — 2~3명 비교 (ADM-03). 참고용이며 결과는 바뀌지 않는다.	화면 admin/compare
`passed`	API	69	`(postingId, q, db, admin)`	`GET /admin/postings/{postingId}/passed` — 다음 전형 대상자 = 1차 합격자와 연락처 (ADM-06). 삭제 예정일을 함께 알려 그 전에 연락하게 한다.	화면 admin/next-stage
`stats`	API	90	`(posting_id, db, _admin)`	`GET /admin/stats` — 집계 숫자만 (ADM-07). 이름·연락처 없음. 보관 기간이 지나 지운 건은 익명 집계(StatsArchive)에서 더한다.	화면 admin/dashboard
`opsStatus`	API	174	`(db, _admin)`	`GET /admin/ops-status` — 운영 현황: 확인 필요 건 · 최근 7일 메일 최종 실패 · 문항 수 (화면정의서 S-R09).	화면 admin/dashboard
`releaseLoginBlock`	API	206	`(code, db, admin)`	`POST /admin/login-blocks/{code}/release`	화면 admin/dashboard

app/routers/admin_questions.py

역할: 관리자 API: 질문 관리 — 관리자 질문 관리: AI 가 만든 질문(검토 대기)을 승인하면 질문은행에서 쓴다.

가져다 쓰는 모듈 | 우리 `db` `deps` `errors` `models` | 외부 `fastapi` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`scopeView`	함수	20	`(db, q) -> dict`	질문의 적용 범위를 화면용으로: 이 공고만(공고 제목) · 이 직무만(직무 이름) · 공통	r.admin_questions:view
`view`	함수	30	`(db, q) -> dict`	질문을 화면용 dict 로 (종류·글·상태·출처 category·적용 범위 scope)	r.admin_questions:listQuestions · r.admin_questions:setStatus
`listQuestions`	API	35	`(status, kind, db, _admin)`	`GET /admin/questions`	화면 admin/questions · 화면 components/AdminShell.tsx
`setStatus`	함수	47	`(db, admin, questionId, status) -> dict`	AI 가 만든 질문의 상태(사용/제외)를 바꾸고 감사 기록을 남긴다	r.admin_questions:approve · r.admin_questions:reject
`approve`	API	60	`(questionId, db, admin)`	`POST /admin/questions/{questionId}/approve`	—
`reject`	API	65	`(questionId, db, admin)`	`POST /admin/questions/{questionId}/reject`	—

app/routers/applications.py

역할: 지원 API: 지원·동의·이력서 — 지원·동의·이력서 API (API 명세서 5장).

가져다 쓰는 모듈

우리 `crypto` `db` `deps` `domain` `errors` `models` `schemas` `services.postings` `services` `settings_store` `timeutil`

표준 `asyncio` | 외부 `fastapi` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`spawn`	함수	27	`(coro) -> None`	백그라운드 작업을 띄우고 참조를 잡아 둔다	r.applications:acceptUpload
`nextStep`	함수	33	`(db, app) -> str`	동의를 했으면 resume, 아니면 consent 단계	r.applications:createApplication · r.applications:getApplication · r.applications:submitResume

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`requireDrafting`	함수	37	`(app, code) -> None`	작성 중인 지원서만 고칠 수 있다. 아니면 409(code 는 부르는 쪽이 정함)	r.applications:cancelUploaded · r.applications:consent · r.applications:submitResume · r.applications:uploadResume
`createApplication`	🌐 API	43	`(body, db, user)`	`POST /applications`	화면 jobs/[postingId]
`getApplication`	🌐 API	65	`(applicationId, db, user)`	`GET /applications/{applicationId}` — 지원 절차 화면의 공통 정보와 상태별 가드 (화면정의서 §2.3).	화면 exam/[applicationId] · 화면 components/ApplyFlow.tsx
`withdrawConsent`	🌐 API	83	`(applicationId, db, user)`	`POST /applications/{applicationId}/consent/withdraw` — 동의 철회 (APP-03): 진행 중 응시는 취소하고 이 지원의 데이터를 바로 지운다. 되돌릴 수 없다.	화면 mypage
`consent`	🌐 API	92	`(applicationId, body, db, user)`	`POST /applications/{applicationId}/consent`	화면 apply/[applicationId]/consent
`acceptUpload`	async 함수	106	`(db, user, file, applicationId) -> ResumeUpload`	이력서 파일을 받아 크기·형식 검사 후 저장하고 추출을 예약한다(동시 2건)	r.applications:uploadResume
`uploadAccepted`	함수	133	`(upload) -> dict`	업로드 접수 응답(쪽수·예상 시간)	r.applications:uploadResume
`uploadStatus`	함수	138	`(upload) -> dict`	추출 진행 상태 응답(단계, 경과, 결과, OCR 신뢰도)	r.applications:extractStatus
`uploadResume`	🌐 API	149	`(applicationId, file, db, user)`	`POST /applications/{applicationId}/resume/upload`	화면 apply/[applicationId]/resume
`extractStatus`	🌐 API	159	`(applicationId, db, user)`	`GET /applications/{applicationId}/resume/extract-status`	화면 apply/[applicationId]/resume
`resumeFile`	🌐 API	168	`(applicationId, db, user)`	`GET /applications/{applicationId}/resume/file`	—
`cancelUpload`	🌐 API	180	`(applicationId, db, user)`	`DELETE /applications/{applicationId}/resume/upload`	화면 apply/[applicationId]/resume
`checkRequired`	함수	191	`(resume) -> None`	필수: 성명과 학력 또는 경력 1건 이상 (APP-05).	r.applications:submitResume
`submitResume`	🌐 API	205	`(applicationId, body, db, user)`	`POST /applications/{applicationId}/resume`	화면 apply/[applicationId]/resume/confirm
`getSavedResume`	🌐 API	234	`(db, user)`	`GET /me/resume`	화면 apply/[applicationId]/resume · 화면 mypage
`putSavedResume`	🌐 API	242	`(body, db, user)`	`PUT /me/resume`	화면 apply/[applicationId]/resume · 화면 mypage
`deleteSavedResume`	🌐 API	256	`(db, user)`	`DELETE /me/resume`	화면 apply/[applicationId]/resume · 화면 mypage

app/routers/auth.py

역할: 로그인·회원가입·비밀번호 API — 인증·계정 API (API 명세서 3장).

가져다 쓰는 모듈

- 우리 `config` `db` `deps` `domain.password_policy` `errors` `models` `schemas` `security` `services` `settings_store` `timeutil`
- 표준 `datetime` `secrets`
- 외부 `fastapi` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`policyError`	함수	26	`(password, email) -> None`	비밀번호 규칙을 어기면 400 PASSWORD_POLICY	r.auth:passwordChange · r.auth:passwordResetConfirm · r.auth:signup
`userView`	함수	32	`(user) -> dict`	사용자 정보를 화면용 dict 로	r.auth:me · r.auth:patchMe
`signup`	API	44	`(body, db)`	`POST /auth/signup`	화면 signup
`countAdminFailure`	함수	65	`(db, user) -> None`	관리자 계정만 잠근다. 뚫렸을 때 피해가 크고, 일부러 잠가도 10분 불편에 그친다.	r.auth:login
`login`	API	74	`(body, request, response, db)`	`POST /auth/login`	화면 login
`logout`	API	105	`(request, response, db)`	`POST /auth/logout`	화면 components/AdminShell.tsx · 화면 components/Shell.tsx
`me`	API	111	`(user)`	`GET /auth/me`	화면 components/Providers.tsx
`passwordChange`	API	116	`(body, request, user, db)`	`POST /auth/password-change`	화면 password/change
`sendResetLink`	함수	134	`(email) -> None`	응답을 보낸 뒤에 돈다. 그래서 가입 여부에 따라 응답 시간이 달라지지 않는다.	r.auth:passwordReset
`passwordReset`	API	154	`(body, background)`	`POST /auth/password-reset` — 결과와 상관없이 202. 가입 여부를 드러내지 않는다 (S-C03 ①).	화면 password/forgot
`expiredToken`	함수	160	`() -> ApiError`	재설정 링크 만료(30분) 오류	r.auth:passwordResetConfirm
`passwordResetConfirm`	API	165	`(body, db)`	`POST /auth/password-reset/confirm`	화면 password/reset
`withdrawMe`	API	183	`(body, response, user, db)`	`POST /me/withdraw` — 탈퇴: 진행 중 응시 취소, 응시 데이터·계정 즉시 삭제 (API 명세서 3장). 같은 이메일로 다시 가입할 수 있다.	화면 mypage
`patchMe`	API	194	`(body, user, db)`	`PATCH /me`	화면 mypage

app/routers/interview.py

역할: 면접 API: 환경 점검·시작·WebSocket·코딩·녹화 — 환경 점검 · 면접 시작/이어가기 · 이탈 신호 · 코딩테스트 · 미디어 (API 명세서 6장 · 8장).

가져다 쓰는 모듈

우리 `config` `db` `deps` `domain` `errors` `interview.engine` `interview` `models` `schemas` `security` `services` `settings_store` `timeutil`

표준 `asyncio` `datetime` `json` `os`

외부 `fastapi` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`postingOpen`	함수	34	`(app) -> bool`	공고 접수 마감 전인지	r.interview:resumeInterview · r.interview:startInterview
`precheck`	API	40	`(applicationId, body, db, user)`	`POST /applications/{applicationId}/precheck`	화면 components/Precheck.tsx
`questionProgress`	함수	68	`(db, app) -> dict`	질문 준비 진행(만든 수 / 10)	r.interview:interviewState
`interviewState`	API	74	`(applicationId, db, user)`	`GET /applications/{applicationId}/interview/state`	화면 exam/[applicationId] · 화면 exam/[applicationId]/resume
`requirePrecheck`	함수	89	`(app) -> None`	환경 점검 통과(15분)가 살아 있어야 시작. 아니면 409	r.interview:resumeInterview · r.interview:startInterview
`newSession`	함수	94	`(db, app, user) -> InterviewSession`	면접 세션 행을 새로 만든다(녹화 구간 번호 +1)	r.interview:resumeInterview · r.interview:startInterview
`sessionResponse`	함수	102	`(row, app, db) -> dict`	세션 시작 응답(WebSocket 주소, 질문 수, 녹화 구간 등)	r.interview:resumeInterview · r.interview:startInterview
`startInterview`	API	108	`(applicationId, db, user)`	`POST /applications/{applicationId}/interview/start`	화면 apply/[applicationId]/rules
`resumeInterview`	API	128	`(applicationId, db, user)`	`POST /applications/{applicationId}/interview/resume`	화면 exam/[applicationId]/resume
`sessionFor`	함수	149	`(db, sessionId, userId) -> tuple[InterviewSession, Applicati`	내 세션과 지원 건을 함께 찾는다. 남의 것이면 404	r.interview:codingApp · r.interview:recordingChunk
`beaconEvent`	API	157	`(sessionId, request, db)`	`POST /interview/{sessionId}/events` — sendBeacon 수신. Content-Type 과 상관없이 본문을 직접 풀고, 못 풀어도 204 (API 명세서 6장).	—
`recordingChunk`	API	177	`(sessionId, segment_no, chunk_no, phase, file, db, user)`	`POST /interview/{sessionId}/recording` — 카메라 녹화 10초 조각 (API 명세서 6장). 같은 조각을 다시 올리면 한 번만 저장한다.	화면 lib/recording.ts
`interviewSocket`	API	188	`(ws, sessionId)`	`WEBSOCKET /ws/interview/{sessionId}`	화면 exam/[applicationId] · 화면 lib/exam.ts
`codingApp`	함수	216	`(db, sessionId, user) -> Application`	코딩 단계인 지원 건만 꺼낸다	r.interview:codingAutosave · r.interview:codingRun · r.interview:codingStart · r.interview:codingSubmit · 외 1곳

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`autoSubmitIfOver`	함수	223	`(db, app) -> None`	코딩 시간이 0이면 자동 제출하고 409 TIME_OVER	r.interview:codingAutosave · r.interview:codingRun · r.interview:codingSubmit · r.interview:codingView
`codingView`	API	230	`(sessionId, db, user)`	`GET /coding/{sessionId}`	—
`requireCoding`	함수	241	`(app) -> None`	코딩 진행 중이 아니면 409	r.interview:codingAutosave · r.interview:codingRun · r.interview:codingStart
`codingStart`	API	247	`(sessionId, db, user)`	`POST /coding/{sessionId}/start`	화면 exam/[applicationId]
`codingRun`	API	258	`(sessionId, body, db, user)`	`POST /coding/{sessionId}/run`	화면 components/exam/Coding.tsx
`codingAutosave`	API	281	`(sessionId, body, db, user)`	`POST /coding/{sessionId}/autosave`	화면 components/exam/Coding.tsx
`codingSubmit`	API	296	`(sessionId, body, db, user)`	`POST /coding/{sessionId}/submit`	화면 components/exam/Coding.tsx
`questionAudio`	API	313	`(questionId, db, user)`	`GET /media/questions/{questionId}`	—

app/routers/me.py

역할: 마이페이지 API — 마이페이지 지원 내역·결과 (API 명세서 9장). 점수·근거는 어느 응답에도 넣지 않는다 (RPT-03, AC-16).

가져다 쓰는 모듈 · 우리 `db` `deps` `domain` `errors` `interview` `models` `timeutil` · 표준 `datetime` · 외부 `fastapi` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`releaseAt`	함수	20	`(app)`	이 건의 공개 시각. 결과가 늦게 나온 건은 따로 정해진다(EVL-08). 확정 전에는 공고의 공개 시각.	r.me:isReleased · r.me:myApplications
`isReleased`	함수	25	`(app) -> bool`	결과가 공개됐고 아직 지워지기 전인지	r.me:myApplications · r.me:myResult
`myApplications`	API	33	`(page, size, db, user)`	`GET /me/applications`	화면 mypage · 화면 mypage/results/[applicationId]
`myResult`	API	62	`(applicationId, db, user)`	`GET /me/results/{applicationId}` — 공개된 결과(public_view)만 준다. 점수·근거·이유는 넣지 않는다 (RPT-03·04). 공개 전·보관 만료는 404.	화면 mypage/results/[applicationId]
`notifications`	API	85	`(db, user)`	`GET /notifications` — 머리글 🔔: 최근 20건과 안 읽은 수 (API 명세서 9장).	화면 components/NotificationBell.tsx

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`readNotification`	API	95	`(notificationId, db, user)`	`POST /notifications/{notificationId}/read`	화면 components/NotificationBell.tsx
`hideNotification`	API	105	`(notificationId, db, user)`	`POST /notifications/{notificationId}/hide` — 🔔 목록에서 지운다(숨김). 안 읽은 알림이면 읽음으로도 표시한다.	화면 components/NotificationBell.tsx
`hideReadNotifications`	API	117	`(db, user)`	`POST /notifications/hide-read` — 읽은 알림을 모두 🔔 목록에서 지운다(숨김).	화면 components/NotificationBell.tsx

app/routers/postings.py

역할: 공고 조회 API(지원자용) — 공개 공고 API (API 명세서 4장 GET /postings · /postings/{id} · /job-codes).

가져다 쓰는 모듈 | 🏠 우리 `db` `deps` `errors` `models` `services.postings` `timeutil` | 📦 외부 `fastapi` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`jobCodes`	API	17	`(db)`	`GET /job-codes`	화면 admin/postings/ai · 화면 jobs · 화면 components/PostingEditor.tsx
`listPostings`	API	23	`(q, job_code, page, size, db, user)`	`GET /postings`	화면 jobs
`getPosting`	API	47	`(postingId, db, user)`	`GET /postings/{postingId}`	화면 jobs/[postingId]

A-5. services — 주방 (실제 기능)

app/services/admission.py

역할: 동시 면접 자리(Redis Lua) — 동시 면접 입장 관리 (TRD 5-3, ADR-08). Redis Lua 로 비교와 더하기를 한 번에 한다.

가져다 쓰는 모듈 | 🏠 우리 `config` `models` `timeutil` | 📦 외부 `redis` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`redisClient`	함수	36	`() -> redis.Redis`	Redis 연결(한 번 만들어 재사용)	r.auth:passwordResetConfirm · r.auth:sendResetLink · s.admission:forceAcquire · s.admission:markPending · 외 4곳 · 🧪 테스트 3개 파일
`tryAcquire`	함수	43	`(db, app, limit) -> bool`	Lua 로 면접 자리 하나 잡기(꽉 차면 False). 잡으면 60초 입장 중 표시	r.interview:startInterview · 🧪 테스트 1개 파일
`forceAcquire`	함수	56	`(db, app) -> None`	이어가기: N 을 다시 확인하지 않고 들어온다 (TRD 5-3).	r.interview:resumeInterview · 🧪 테스트 1개 파일

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`release`	함수	67	`(db, app) -> None`	자리 반환(Lua 로 −1)과 입장 중 표시 삭제	i.lifecycle:enterCoding · i.lifecycle:finish · jobs:releaseHeldSeats · s.postings:closePosting · 외 1곳
`reconcile`	함수	77	`(db) -> int`	Redis 숫자를 holds_seat 가 켜진 면접 수(커밋 전 입장 포함)로 다시 맞춘다. jobs 의 1분 주기 작업이 부른다.	jobs:runForever · main:recoverAfterRestart · 🧪 테스트 1개 파일
`markPending`	함수	86	`(app) -> None`	'입장 중' 표시를 60초 남긴다(E10)	s.admission:forceAcquire · s.admission:tryAcquire

app/services/coding.py

역할: 코딩테스트 진행 — 코딩테스트 제출·채점 (PRD COD-01~06, API 명세서 8장).

가져다 쓰는 모듈 | 🏠 우리 `db` `domain` `interview` `models` `timeutil` | 표준 `logging` | 외부 `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`scheduleGrade`	함수	23	`(applicationId) -> None`	숨겨진 테스트 채점을 worker-sandbox 에 넣는다. 같은 건은 5분 안에 다시 넣지 않는다.	jobs:regradePending · s.coding:finishCoding
`runExamples`	함수	29	`(code, cases) -> dict`	[실행]: worker-sandbox 에서 예시 입력으로 돌리고 결과를 기다린다. 못 받으면 인프라 오류 (오답 아님).	r.interview:codingRun · 🧪 테스트 1개 파일
`problemViews`	함수	50	`(db, app) -> list[dict]`	화면에 줄 문제 목록. 다시 연결해도 저장된 코드가 이어지도록 함께 준다 (COD-04).	i.engine:sendCodingPhase · r.interview:codingView · 🧪 테스트 2개 파일
`saveCodes`	함수	61	`(db, app, codes) -> None`	문항별 코드를 저장하고 저장 시각을 남긴다	s.coding:finishCoding
`finishCoding`	함수	70	`(db, app, codes) -> None`	제출 또는 시간 끝 자동 제출. 응시 완료로 바꾸고 숨겨진 테스트 채점을 뒤에서 돌린다.	i.engine:codingTimeOver · r.interview:autoSubmitIfOver · r.interview:codingSubmit
`gradeAll`	함수	77	`(applicationId) -> None`	worker-sandbox 안에서만 부른다.	worker:gradeCodingTask · 🧪 테스트 1개 파일

app/services/coding_bank.py

역할: 코딩 문제 목록 — 코딩테스트 문제 목록 (PRD COD-02: AI/AX 엔지니어 문제 목록에서 기초 1 · 응용 1, Python 3, 60분).

가져다 쓰는 모듈 | 표준 `hashlib`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`pickProblems`	함수	137	`(applicationId) -> list[str]`	지원 건마다 기초 1 · 응용 1 을 고른다. 같은 지원 건은 언제나 같은 문제다. 고른 결과는 coding_submissions 에 남는다.	i.lifecycle:enterCoding · 🧪 테스트 2개 파일

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`publicView`	함수	147	`(problemNo, key) -> dict`	지원자에게 보여 줄 문제 정보(정답·숨은 테스트 제외)	s.coding:problemViews

app/services/evaluation.py

역할: 자동 채점 전체 — 응시 1건 채점 → 1차 결과 확정 (PRD 5-7·5-8, TRD 8장, AI 아키텍처 v1.4 6.3).

가져다 쓰는 모듈 · 우리 `domain` `models` `timeutil` · 표준 `dataclasses` `datetime` `decimal` `hashlib` `json` `logging` `re` `typing` · 외부 `pydantic` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`AiUnavailable`	클래스	86	`(Exception)`	모델 서버가 내려가 있음 → 시도 횟수를 쓰지 않고 미룬다 (AI 아키텍처 7장).	s.evaluation:evaluate · s.evaluation:writeReason · worker:evaluateTask · 🧪 테스트 1개 파일
`EvalFailed`	클래스	90	`(Exception)`	모델이 답했지만 쓸 수 없음 → 시도 횟수를 쓴다.	s.evaluation:loadInput · s.evaluation:scoreQuestion · s.evaluation:writeReason · 🧪 테스트 1개 파일
`NeedsReview`	클래스	94	`(Exception)`	자동 채점을 멈추고 평가 지연으로 보낼 때 쓰는 예외(이유 포함)	s.evaluation:evaluate · s.evaluation:writeReason · worker:evaluateTask · 🧪 테스트 1개 파일
`NeedsReview.__init__`	메소드	95	`(self, reason)`	이유 저장	—
`QuestionInput`	클래스	103	``	채점할 질문 하나(번호, 종류, 질문, 답변, 평가 항목)	s.evaluation:loadQuestions · s.evaluation:scoreQuestion · s.evaluation:t5Prompt · 🧪 테스트 1개 파일
`EvalInput`	클래스	114	``	채점에 필요한 지원 건 전체 입력 (가중치, 질문들, 코딩 결과 등)	s.evaluation:codingWaiting · s.evaluation:computeScores · s.evaluation:loadInput · s.evaluation:pickHighlights · 외 7곳
`loadQuestions`	함수	130	`(db, app) -> list[QuestionInput]`	질문 번호 순서로 답변 (꼬리질문 답변은 본질문 답변 뒤에 붙인다).	r.admin:applicantDetail · r.admin:reprocess · s.evaluation:loadInput
`loadInput`	함수	148	`(db, app) -> EvalInput`	지원 때 잠근 기준 버전만 쓴다. 없으면 다른 버전(다른 합격 기준)으로 판정하지 않고 실패로 넘긴다	s.evaluation:evaluate
`fence`	함수	163	`(text) -> str`	답변·코드 안의 꺾쇠를 바꿔 <답변> 같은 구분 태그를 닫거나 새로 열지 못하게 한다 (프롬프트 인젝션).	i.engine:followupDecide · s.evaluation:checkT5 · s.evaluation:reviewCode · s.evaluation:t5Prompt · 외 3곳

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`redact`	함수	168	`(text, name, email, phone) -> str`	이름·연락처를 AI 입력에서 지운다 (PRD EVL-01, AI 아키텍처 8장).	s.evaluation:pickHighlights · s.evaluation:reviewCode · s.evaluation:scoreQuestion · s.evaluation:writeReason · 🧪 테스트 1개 파일
`instructionHits`	함수	180	`(questions) -> list[dict]`	채점을 바꾸려는 말로 보이는 구절 [{seq, phrase}] (관리자 확인용).	r.admin:applicantDetail · r.admin:reprocess · s.evaluation:evaluate
`hitsDigest`	함수	191	`(hits) -> str`	채점 근거 묶음의 지문(SHA-256). 같은 입력인지 비교용	r.admin:reprocess · s.evaluation:instructionAcknowledged
`instructionAcknowledged`	함수	195	`(db, applicationId, hits) -> bool`	관리자가 이 구절들을 확인하고 [채점하기]를 눌렀으면 그대로 채점한다. 구절이 바뀌면 다시 확인.	s.evaluation:evaluate
`codingWaiting`	함수	202	`(data) -> bool`	코딩 채점이 아직 안 끝났으면 기다린다 (AI 아키텍처 6.3 load_report_inputs).	s.evaluation:evaluate
`T5Item`	클래스	211	`(BaseModel)`	AI 문항 채점 한 칸(항목, 0~5점, 근거 구절)	s.evaluation:checkT5 · s.evaluation:scoreQuestion · 🧪 테스트 1개 파일
`T5Out`	클래스	218	`(BaseModel)`	AI 문항 채점 결과(항목 목록)	s.evaluation:checkT5 · s.evaluation:scoreQuestion · 🧪 테스트 1개 파일
`t5Prompt`	함수	230	`(q, answer, problems) -> str`	문항 채점 프롬프트(질문·평가 항목·루브릭·답변, 재시도면 지난 문제점)	s.evaluation:scoreQuestion · 🧪 테스트 1개 파일
`checkT5`	함수	237	`(out, items, answer) -> tuple[dict[str, T5Item], list[str]]`	쓸 수 있는 항목과 문제 목록. 점수 범위는 스키마가, 근거 위치는 여기서 본다 (TRD 8-2 ②).	s.evaluation:scoreQuestion · 🧪 테스트 1개 파일
`scoreQuestion`	async 함수	256	`(q, data) -> list[dict]`	한 질문 번호의 항목 점수. 답이 없으면 0점, 근거를 끝내 못 얻으면 1점 (PRD EVL-02).	s.evaluation:evaluate
`Scores`	클래스	293	``	계산된 점수 묶음(항목별, 면접, 코딩, 총점, 판정)	s.evaluation:computeScores · s.evaluation:evaluate · s.evaluation:save · s.evaluation:t10Prompt · 외 3곳
`computeScores`	함수	301	`(data, perQuestion) -> Scores`	AI 항목 점수를 모아 코드로 면접·코딩·총점과 합격 여부를 계산	s.evaluation:evaluate
`T10Raw`	클래스	315	`(BaseModel)`	모델이 내는 형식. 근거 문장은 글자로 옮겨 쓰지 않고 목록 번호로 고른다 (CPU 8B 토큰·옮겨 쓰기 오류 줄이기).	s.evaluation:checkT10 · s.evaluation:writeReason · 🧪 테스트 1개 파일
`T10Out`	클래스	321	`(BaseModel)`	저장하는 형식: 이유 + 번호를 실제 근거 문장으로 바꾼 것.	s.evaluation:evaluate · s.evaluation:save · s.evaluation:writeReason
`evidenceFor`	함수	334	`(perQuestion) -> list[dict]`	항목마다 가장 높은·낮은 근거 문장 (중복 없이 최대 MAX_EVIDENCE).	s.evaluation:writeReason
`t10Prompt`	함수	351	`(data, scores, evidence, problems) -> str`	결과 이유 프롬프트(정해진 판정·점수·근거만 주고 이유만 쓰게)	s.evaluation:writeReason

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`checkT10`	함수	362	`(out, decision, evidenceCount) -> str | None`	틀린 이유 코드. 맞으면 None (TRD 8-2 ⑤, API 명세서 needs_review.reason). 근거 목록은 T5 에서 답변 안에 있는지 확인한 문장이라, 번호가 목록 안이면 근거가 답변에 있는 것이다.	s.evaluation:writeReason · 🧪 테스트 1개 파일
`writeReason`	async 함수	380	`(data, scores, perQuestion) -> tuple[T10Out, str]`	(이유, decision_check). 두 번 다 틀리면 NeedsReview.	s.evaluation:evaluate
`T6Out`	클래스	408	`(BaseModel)`	요약·강점·개선점 결과 모양	s.evaluation:evaluate · s.evaluation:save · s.evaluation:writeSummary · 🧪 테스트 1개 파일
`T7Out`	클래스	414	`(BaseModel)`	강점·개선점 결과 모양	s.evaluation:writeFeedback · 🧪 테스트 1개 파일
`T8Out`	클래스	419	`(BaseModel)`	코드 리뷰 결과 모양	s.evaluation:reviewCode · 🧪 테스트 1개 파일
`gradeOf`	함수	426	`(points) -> str`	점수 → 상(70↑)/중(40↑)/하	s.evaluation:writeFeedback
`copiesAnswer`	함수	430	`(text, answers) -> bool`	피드백이 답변을 COPY_LIMIT 자 이상 그대로 옮겼는지 (띄어쓰기·문장부호는 무시).	s.evaluation:writeFeedback · 🧪 테스트 1개 파일
`fallbackFeedback`	함수	437	`(itemPoints) -> tuple[list[str], list[str]]`	T7 이 실패했을 때 항목 등급으로 만드는 규칙 문장.	s.evaluation:writeFeedback · s.evaluation:writeSummary
`writeSummary`	async 함수	445	`(data, perQuestion, scores) -> T6Out`	질문별 점수를 보고 요약 글을 만든다	s.evaluation:evaluate
`writeFeedback`	async 함수	470	`(data, scores) -> tuple[list[str], list[str]]`	지원자용 강점·개선점. 점수·인용·합불은 넣지도 쓰지도 않는다 (T7, RPT-03).	s.evaluation:evaluate
`reviewCode`	async 함수	487	`(problem, data) -> str | None`	T8. 숨겨진 테스트는 넣지 않는다. 실패하면 빈칸 (점수와 무관).	s.evaluation:evaluate
`locateQuote`	함수	507	`(words, quote) -> tuple[int, int] | None`	근거 문장을 STT 단어 시각과 맞춰 (시작, 끝) ms. 앞뒤 1초씩 늘린다.	s.evaluation:pickHighlights · 🧪 테스트 1개 파일
`pickHighlights`	함수	524	`(data, perQuestion) -> list[dict]`	질문마다 최고·최저 1개 → 잘함(4~5)·깎임(1~2) 각각 최대 3개, 점수 차가 큰 순서. 0점은 뺀다.	s.evaluation:save
`releaseSchedule`	함수	549	`(postingReleaseAt, decidedAt) -> tuple[object, str]`	(이 건의 공개 시각, release_status). 공고 공개일까지 못 나왔으면 나온 다음 날 10:00, 'delayed' (EVL-08).	s.evaluation:closeUnscored · s.evaluation:save · 🧪 테스트 1개 파일
`statusReason`	함수	559	`(attemptStatus) -> str | None`	응시 상태(취소·마감·종료)에 맞는 결과 사유 코드	s.evaluation:closeUnscored · s.evaluation:save
`evaluate`	async 함수	563	`(applicationId) -> str`	'decided' · 'waiting_coding' · 'skipped'. 실패는 AiUnavailable · EvalFailed · NeedsReview 로 알린다.	worker:evaluateTask · 🧪 테스트 1개 파일
`save`	함수	611	`(data, scores, perQuestion, reason, decisionCheck, summary,`	채점 결과를 한 트랜잭션으로 저장(행 잠금)	s.evaluation:evaluate
`reviewDeadline`	함수	663	`(app)`	평가 지연 결정 기한(지연 시작 + 3일)	r.admin:applicantDetail · worker:reminderRows

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`closeUnscored`	함수	667	`(db, app, decision, reason, check, statusReason, modelVersio`	점수 없이 결과를 확정한다. 커밋은 부르는 쪽에서 한다. 기준 버전이 아예 없으면 False.	s.evaluation:closeIncomplete · s.evaluation:decideManually
`decideManually`	함수	686	`(db, app, decision, reason, adminId) -> bool`	관리자 직접 판정을 저장하고 감사 기록(사유 글은 빼고)을 남긴다	r.admin:manualDecision
`closeIncomplete`	함수	694	`(db, app) -> bool`	평가 미완료 불합격으로 닫는다. 커밋은 부르는 쪽에서 한다.	worker:settleReviews
`markNeedsReview`	함수	703	`(applicationId, reason) -> bool`	확인 필요로 바꾼다. 이미 결과가 있으면 바꾸지 않는다. 바꿨으면 True.	worker:toNeedsReview · 🧪 테스트 2개 파일

app/services/llm.py

역할: LLM 호출 공통 — LLM 호출 (TRD 3장 구현 준수 사항, 기술 아키텍처 9-2).

가져다 쓰는 모듈 | 🏠 우리 `config` | 🐍 표준 `asyncio` `logging` | 📦 외부 `httpx` `langchain_core` (langchain-core) `langchain_openai` (langchain-openai) `pydantic`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`LlmError`	클래스	20	`(Exception)`	LLM 호출 실패	i.engine:followupDecide · main:warmUp · s.evaluation:reviewCode · s.evaluation:scoreQuestion · 외 9곳 · 🧪 테스트 3개 파일
`laneUrl`	함수	28	`(lane) -> str`	lane 이 eval 이면 8B 서버, 아니면 4B 서버 주소	s.llm:chatModel · s.llm:health
`chatModel`	함수	32	`(timeoutSec, maxTokens, temperature, lane) -> ChatOpenAI`	LangChain ChatOpenAI 객체를 만든다 (내부 서버, 시간·토큰·온도 지정)	s.llm:structured
`structured`	async 함수	44	`(schema, system, user, timeoutSec, maxTokens, temperature, l`	JSON 스키마 강제 → Pydantic 재검사. 시간 초과·형식 오류는 LlmError.	i.engine:followupDecide · main:warmUp · s.evaluation:reviewCode · s.evaluation:scoreQuestion · 외 8곳
`health`	async 함수	64	`(timeoutSec, lane) -> bool`	LLM 서버 /health 가 살아 있는지	r.interview:precheck · s.evaluation:evaluate · s.evaluation:writeReason · worker:aiReady

app/services/login_block.py

역할: 로그인 실패 IP 차단 — 로그인 실패 IP 차단. 계정과 무관하게 세어 계정 존재를 드러내지 않고, 남의 계정을 일부러 잠글 수도 없게 한다.

가져다 쓰는 모듈 | 우리 `config` `crypto` `errors` `models` `settings_store` `timeutil` | 표준 `datetime` `ipaddress` `logging` `secrets` `socket` | 외부 `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`isTrustedProxy`	함수	29	`(peer) -> bool`	요청을 넘겨 준 쪽이 믿는 프록시인지	s.login_block:clientIp
`normalizeIp`	함수	38	`(raw) -> str`	IP 글을 표준 모양으로(틀리면 unknown)	s.login_block:clientIp
`clientIp`	함수	46	`(request) -> str`	실제 접속 IP(믿는 프록시일 때만 X-Forwarded-For 사용)	s.login_block:guardKey · 🧪 테스트 1개 파일
`isExempt`	함수	54	`(ip) -> bool`	차단 예외 IP(대역)인지	s.login_block:guardKey · 🧪 테스트 1개 파일
`guardKey`	함수	67	`(request) -> str | None`	차단을 세는 키. 제외 대상이면 None.	r.auth:login
`isActive`	함수	73	`(row) -> bool`	차단이 아직 유효한지	s.login_block:activeBlock · s.login_block:recordFailure
`activeBlock`	함수	77	`(db, key) -> LoginBlock | None`	그 키의 유효한 차단 행	r.auth:login
`lockedRow`	함수	82	`(db, key) -> LoginBlock`	처음 보는 IP 의 동시 요청이 겹쳐도 한 행으로 센다.	s.login_block:recordFailure
`recordFailure`	함수	96	`(db, key) -> LoginBlock | None`	실패 한 번을 센다. 차단 중이면 그 행을 돌려준다. 커밋은 부르는 쪽에서 한다.	r.auth:login
`blockedError`	함수	116	`(db, row) -> ApiError`	429 LOGIN_BLOCKED 오류(풀리는 시각·문의 코드 포함)	r.auth:login
`activeBlocks`	함수	122	`(db) -> list[LoginBlock]`	지금 유효한 차단 목록(대시보드용)	r.admin_ops:opsStatus
`release`	함수	127	`(db, code, adminId) -> bool`	관리자가 문의 코드로 차단을 푼다(감사 기록)	r.admin_ops:releaseLoginBlock
`purgeExpired`	함수	135	`(db) -> int`	차단도 집계 기간도 끝난 행을 지운다. 커밋은 부르는 쪽에서 한다.	worker:purgeTask · 🧪 테스트 1개 파일

app/services/notify.py

역할: 알림과 메일 — 사이트 알림 + 메일 (TRD 10장, 테이블 정의서 4-22).

가져다 쓰는 모듈 | 우리 `config` `models` `timeutil` | 표준 `email` `logging` `smtplib` | 외부 `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`create`	함수	25	`(db, userId, kind, title, link, applicationId) -> Notificati`	알림 행을 만든다(커밋은 호출한 쪽). 메일은 커밋 뒤 queueMail 로 넣는다.	worker:notifyMany · worker:releaseDue
`exists`	함수	33	`(db, userId, kind, applicationId, title) -> bool`	같은 알림을 두 번 만들지 않기 위해(매시 작업이 같은 건을 다시 볼 수 있음).	worker:purgeNoticeTask · worker:reminderRows · worker:toNeedsReview
`adminIds`	함수	41	`(db) -> list[str]`	관리자 사용자 ID 목록	worker:purgeNoticeTask · worker:reminderRows · worker:sweepTask · worker:toNeedsReview

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`mailBody`	함수	45	`(row) -> str`	알림 메일 본문(제목 + 링크)	s.notify:sendNow
`MailRetry`	클래스	50	`(Exception)`	메일을 다시 보내야 할 때 쓰는 예외	s.notify:retryOrFail · s.notify:sendPlain · worker:sendMailTask · worker:sendResetMailTask
`sendNow`	함수	54	`(notificationId) -> str`	메일 1통. 'sent' · 'failed' · 'skipped'. 다시 보낼 오류는 MailRetry.	worker:sendMailTask
`sendPlain`	함수	96	`(to, subject, body) -> str`	알림 행 없이 보내는 메일 (비밀번호 재설정 — 링크에 토큰이 있어 DB 에 남기지 않는다). 다시 보낼 오류는 MailRetry.	worker:sendResetMailTask
`retryOrFail`	함수	117	`(db, row, reason) -> str`	재시도 횟수가 남았으면 다시, 아니면 실패로 기록	s.notify:sendNow

app/services/pdf_box.py

역할: PDF 읽기 격리 프로세스 — PDF 를 읽는 별도 프로세스. 믿을 수 없는 파일이라 api 프로세스 안에서 열지 않는다 (논의사항 P5).

가져다 쓰는 모듈 | 표준 `io` `json` `re` `resource` `sys` | 외부 `fitz` (pymupdf) `pdfplumber`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`isReadable`	함수	31	`(text) -> bool`	쪽 글자를 믿을 수 있는지: 의미 있는 글자가 충분하고, 깨진 글자 흔적이 적어야 한다.	s.pdf_box:readPages · 테스트 1개 파일
`Encrypted`	클래스	40	`(Exception)`	암호 걸린 PDF	s.pdf_box:main · s.pdf_box:readPages
`readPages`	함수	44	`(data) -> list[dict]`	PDF 쪽마다 글을 뽑는다(PyMuPDF 먼저, 모자라면 pdfplumber)	s.pdf_box:main · 테스트 1개 파일
`renderPage`	함수	75	`(data, pageNo) -> bytes`	PDF 한 쪽을 이미지로(OCR 용, 해상도 상한)	s.pdf_box:main
`limitResources`	함수	84	`() -> None`	자식 프로세스의 메모리·CPU 시간을 제한(리눅스)	s.pdf_box:main
`main`	함수	95	`(args) -> None`	별도 프로세스 입구: stdin 으로 PDF 를 받아 read/render 실행	—

app/services/posting_draft.py

역할: AI 공고 초안 — AI 공고 초안 (S-R02, JOB-02, G-06).

가져다 쓰는 모듈 | 우리 `db` `interview.checkpoint` `models` `llm` | 표준 `asyncio` `logging` `random` `re` `typing` | 외부 `langgraph` `pydantic`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`DraftText`	클래스	65	`(BaseModel)`	AI 공고 초안의 칸 모양(제목·소개·업무·자격…)	s.posting_draft:draft · s.posting_draft:fillGaps · s.posting_draft:isUsable · 테스트 2개 파일
`biasWarnings`	함수	77	`(*texts) -> list[dict]`	차별로 보일 수 있는 표현을 찾아 경고 목록으로	s.posting_draft:deliver · s.posting_draft:save · s.posting_draft:sectionProblems · s.posting_draft:status · 테스트 1개 파일
`buildPrompt`	함수	88	`(body, jobName) -> str`	관리자 답변으로 공고 초안 프롬프트를 만든다	r.admin:draftAi · 테스트 1개 파일
`isUsable`	함수	107	`(draft) -> bool`	부서 소개는 보지 않는다 (짧으면 fillGaps 가 맡을 일로 만든 소개로 채운다).	s.posting_draft:draft
`isDashOnly`	함수	112	`(text) -> bool`	'-' 만 있는 빈 목록인지	s.posting_draft:fillGaps · s.posting_draft:tidySection
`tidyDuty`	함수	148	`(duty) -> str`	업무 한 줄을 '~합니다' 꼴로 다듬는다	s.posting_draft:fallbackIntro · 테스트 1개 파일
`fallbackIntro`	함수	170	`(body, jobName, template) -> str`	메모를 그대로 내보내지 않고 관리자가 고른 맡을 일·기술로만 부서 소개를 만든다. 문구는 기본 문구 중 하나를 고른다.	r.admin:draftAi · s.posting_draft:generate · s.posting_draft:start · 테스트 1개 파일
`unifyBunLines`	함수	182	`(text) -> str`	자격 요건 줄 끝을 '~ 분'으로 맞춘다	s.posting_draft:fillGaps · s.posting_draft:tidySection · 테스트 1개 파일
`fillGaps`	함수	194	`(draft, fallback) -> DraftText`	작은 모델이 부서 소개를 짧게 쓰거나 비우면 실패시키지 않고 fallbackIntro 로 채운다. 자격 요건·우대 사항 끝맺음도 맞춘다.	s.posting_draft:draft · 테스트 1개 파일
`SectionText`	클래스	222	`(BaseModel)`	한 칸만 다시 쓴 결과 모양(text)	s.posting_draft:rewrite · 테스트 1개 파일
`DraftState`	클래스	226	`(TypedDict)`	AI 초안 그래프의 상태(체크포인트에 남음). 공고 문장과 관리자 입력만 있고 지원자 개인정보는 없다	s.posting_draft:afterCheck · s.posting_draft:afterDraft · s.posting_draft:buildDraftGraph · s.posting_draft:check · 외 7곳
`threadId`	함수	242	`(postingId) -> str`	공고별 그래프 기록 이름 'posting-draft:{공고 id}'	r.admin:deletePosting · r.admin:draftAi · r.admin:publish · s.posting_draft:graphConfig
`graphConfig`	함수	246	`(postingId) -> dict`	LangGraph 실행 설정(thread_id)	s.posting_draft:generate · s.posting_draft:hasDraft · s.posting_draft:revise

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`setState`	함수	250	`(postingId, **values) -> None`	화면이 보는 진행 상태(메모리)를 고친다: status · stage · 경고 · 처리 기록 · revision	s.posting_draft:check · s.posting_draft:deliver · s.posting_draft:draft · s.posting_draft:fail · 외 6곳
`referenceText`	함수	255	`(p) -> str`	참고 공고 한 건을 프롬프트에 넣을 글로(제목·업무·자격, 길이 제한)	s.posting_draft:findReferences
`findReferences`	함수	259	`(db, posting) -> list[str]`	① 같은 직무의 게시·마감 공고 최근 2건을 찾는다 (벡터 검색 없이 DB 조회)	s.posting_draft:retrieve
`fenceText`	함수	267	`(text) -> str`	참고 글 속 지시문을 따르지 않게 글을 울타리로 감싼다	s.posting_draft:rewritePrompt · s.posting_draft:withReferences
`withReferences`	함수	272	`(prompt, references) -> str`	초안 프롬프트에 참고 공고를 '용어·문체만 참고, 그대로 옮기지 말 것'으로 붙인다	s.posting_draft:draft · s.posting_draft:rewritePrompt
`normalized`	함수	280	`(text) -> str`	띄어쓰기·문장부호를 뺀 글 (베끼기 비교용)	s.posting_draft:copiesReference
`copiesReference`	함수	284	`(text, references) -> bool`	참고 공고 문장을 40자 넘게 그대로 옮겼는가	s.posting_draft:sectionProblems
`sectionProblems`	함수	290	`(text, references) -> list[str]`	③ 한 칸의 고칠 점 목록: 차별로 보일 수 있는 표현, 예전 공고 문장 베끼기	s.posting_draft:check
`tidySection`	함수	298	`(section, text) -> str | None`	다시 쓴 칸을 그 칸 형식에 맞게 다듬고, 안 맞으면 None(앞의 글을 둔다)	s.posting_draft:rewrite
`rewritePrompt`	함수	311	`(state, section) -> str`	② 한 칸 다시 쓰기 프롬프트(고칠 점 또는 관리자 요청 + 지금 초안)	s.posting_draft:rewrite
`rewriteLimit`	함수	325	`(state, section) -> int`	그 칸을 다시 쓸 수 있는 횟수 (자동 1번, 관리자 요청이면 1번 더)	s.posting_draft:pendingSections · s.posting_draft:rewrite
`pendingSections`	함수	330	`(state) -> list[str]`	아직 다시 써야 하는 칸 목록 (고칠 점이 있고 횟수가 남은 칸, 관리자가 고른 칸)	s.posting_draft:afterCheck · s.posting_draft:rewrite
`buildDraftGraph`	함수	340	`()`	AI 공고 초안 LangGraph: retrieve → draft → check → rewrite ↺ → save(처음) / deliver(한 칸 다시 쓰기) / fail	s.posting_draft:generate · s.posting_draft:hasDraft · s.posting_draft:revise
`buildDraftGraph.retrieve`	안쪽 함수	341	`(state) -> dict`	노드 ① 참고 공고 찾기, 이번 흐름 값 비우기	—
`buildDraftGraph.draft`	안쪽 함수	350	`(state) -> dict`	노드 초안 작성(LLM). 못 쓰면 시도 횟수만 올린다	—
`buildDraftGraph.check`	안쪽 함수	362	`(state) -> dict`	노드 ③ 칸별 검사 (한 칸 다시 쓰기면 그 칸만)	—
`buildDraftGraph.rewrite`	안쪽 함수	368	`(state) -> dict`	노드 ② 고칠 칸만 다시 쓰고 처리 기록(notes)을 남긴다. 형식이 틀리면 앞의 글을 둔다	—
`buildDraftGraph.save`	안쪽 함수	393	`(state) -> dict`	노드 처음 초안을 공고에 저장하고 차별 표현 경고를 붙인다 (임시저장 공고만)	—

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`buildDraftGraph.deliver`	안쪽 함수	408	`(state) -> dict`	노드 ④ 다시 쓴 칸을 DB 에 쓰지 않고 화면에 준다(revision)	—
`buildDraftGraph.fail`	안쪽 함수	420	`(state) -> dict`	노드 실패로 표시	—
`buildDraftGraph.routeStart`	안쪽 함수	424	`(state) -> str`	처음 초안이면 retrieve, 한 칸 다시 쓰기면 rewrite 에서 시작	—
`buildDraftGraph.afterDraft`	안쪽 함수	427	`(state) -> str`	초안이 있으면 check, 없으면 다시 (2번까지) 또는 fail	—
`buildDraftGraph.afterCheck`	안쪽 함수	432	`(state) -> str`	다시 쓸 칸이 남으면 rewrite, 아니면 save / deliver	—
`generate`	async 함수	451	`(postingId, prompt, fallbackIntro) -> None`	새 초안: 그래프를 처음부터 돌린다. 예상 못 한 오류도 '실패'로 보여 다시 시도하게 한다	s.posting_draft:start · 🧪 테스트 2개 파일
`hasDraft`	async 함수	461	`(postingId) -> bool`	다시 쓰기에 쓸 초안 기록(체크포인트)이 있는가	r.admin:draftAiRevise · 🧪 테스트 1개 파일
`revise`	async 함수	467	`(postingId, section, note, current) -> None`	④ 관리자가 고른 칸만 다시 쓴다. 처음 입력·참고 공고는 체크포인트에서 이어 쓴다	s.posting_draft:startRevision · 🧪 테스트 1개 파일
`runInBackground`	함수	480	`(coro) -> None`	작업을 뒤에서 돌리고 끝날 때까지 참조를 잡아 둔다	s.posting_draft:start · s.posting_draft:startRevision
`start`	함수	486	`(postingId, prompt, fallbackIntro) -> None`	새 초안 만들기를 뒤에서 시작하고 진행 상태를 '참고 공고 찾기'로 둔다	r.admin:draftAi
`startRevision`	함수	491	`(postingId, section, note, current) -> None`	한 칸 다시 쓰기를 뒤에서 시작한다	r.admin:draftAiRevise
`status`	함수	496	`(postingId, posting) -> dict`	초안 진행 상태(stage·경고·처리 기록·revision). 기록이 없으면(재시작) 내용 유무로 완료/실패	r.admin:draftAiStatus

app/services/posting_questions.py

역할: 공고 기반 질문 후보 — 게시한 공고로 직무·회사 질문 후보를 만들어 '검토 대기'로 넣고, 인터넷 참고 후보 파일을 불러온다 (CONFLICTS C-17)

가져다 쓰는 모듈

- 우리: `db` `domain.question_text` `domain.scoring` `models` `evaluation` `llm` `question_bank` `question_compose` `rag`
- 표준: `asyncio` `json` `logging`
- 외부: `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`schedule`	함수	38	`(postingId) -> None`	공고 기반 질문 후보 만들기를 뒤에서 시작한다 (게시 응답을 기다리게 하지 않음)	r.admin:publish
`saveCandidate`	함수	44	`(db, posting, kind, text, vector, category) -> None`	후보 1개를 적용 범위·출처를 붙여 '검토 대기'로 저장하고 임베딩을 단다	s.posting_questions:generateForPosting · s.posting_questions:loadWebCandidates

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`isKnown`	함수	54	`(db, text) -> bool`	같은 글(띄어쓰기·문장부호 무시)이 질문 DB 에 이미 있는가	s.posting_questions:loadWebCandidates · s.posting_questions:makeOne
`examplesFor`	async 함수	58	`(db, posting, kind, topic) -> list[rag.Example]`	그 자리의 예시 질문(RAG)을 찾는다. 임베딩 서버가 없으면 빈 목록	s.posting_questions:makeOne
`makeOne`	async 함수	66	`(db, posting, kind, topic, made)`	질문 1개를 만들어 검사한다(글 검사·다른 회사 이름·중복·질문 DB 와 0.8 이상). 2번까지, 실패하면 (None, None)	s.posting_questions:generateForPosting
`generateForPosting`	async 함수	99	`(postingId) -> dict`	공고 하나로 직무 질문(줄마다 1개, 최대 6) + 회사 질문 3개(초점이 서로 다름)를 만들어 저장하고 감사 기록을 남긴다. 실패해도 오류를 던지지 않는다	s.posting_questions:schedule · 🧪 테스트 1개 파일
`loadWebCandidates`	함수	123	`(db, path, postingId) -> int`	인터넷 참고 후보 파일에서 verdict=new 이고 아직 없는 질문을 검토 대기로 넣는다(임베딩 포함)	manage:loadWebQuestions · 🧪 테스트 1개 파일

app/services/postings.py

역할: 공고 규칙 — 공고·평가 기준 규칙 (PRD JOB-01~07, 화면정의서 §10.3, API 명세서 4장).

가져다 쓰는 모듈 | 🏠 우리 `domain.scoring` `domain` `errors` `models` `timeutil` | 📦 외부 `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`seedJobCodes`	함수	31	`(db) -> None`	직무 코드 기본값을 DB 에 넣는다	main:lifespan · manage:init · 🧪 테스트 1개 파일
`currentCriteria`	함수	38	`(posting) -> EvaluationCriteria | None`	공고의 최신 평가 기준	i.engine:beginQuestion · i.engine:finishInterview · r.admin:applicants · r.admin:draftAi · 외 7곳
`hasStartedAttempt`	함수	42	`(db, posting) -> bool`	응시를 시작한 지원자가 있는지(기준 잠금 판단)	r.admin:getPosting · s.postings:updatePosting
`criteriaView`	함수	46	`(c) -> dict | None`	평가 기준을 화면용 dict 로	s.postings:postingView
`postingView`	함수	53	`(p, withCriteria) -> dict`	공고를 화면용 dict 로(기준 포함 선택)	r.admin:applicantDetail · r.admin:applicants · r.admin:createPosting · r.admin:getPosting · 외 5곳
`revisionSummary`	함수	70	`(field, before, after) -> str`	지원자에게 보여 줄 수정 사항 한 줄(테이블 정의서 4-6 summary). 예: 근무지: 서울 강남구 → 서울 서초구	s.postings:revisionsView · s.postings:updatePosting
`revisionsView`	함수	78	`(db, postingId) -> list[dict]`	공고 수정 이력 목록	r.admin:getPosting · r.postings:getPosting
`isAcceptingApplications`	함수	85	`(p) -> bool`	게시 중이고 접수 기간 안인지	r.applications:createApplication · r.applications:submitResume

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`displayValue`	함수	90	`(field, value) -> str`	수정 이력에 보일 값(고용 형태 이름, '명' 등)	s.postings:applyCriteria · s.postings:updatePosting
`applyCriteria`	함수	100	`(db, posting, body, locked) -> list[tuple[str, str, str]]`	평가 기준 반영. 잠기면 409. 게시 중 코딩 사용 여부가 바뀌면 수정 사항으로 남길 값을 돌려준다.	s.postings:updatePosting
`updatePosting`	함수	124	`(db, posting, body) -> JobPosting`	공고 수정(마감 공고 거부, 게시 후 잠긴 칸 검사, 이력 기록)	r.admin:createPosting · r.admin:draftAi · r.admin:patchPosting
`validateForPublish`	함수	164	`(db, posting) -> None`	게시 전 필수 칸·평가 기준·직무 코드 검사	r.admin:publish
`closePosting`	함수	177	`(db, posting) -> dict`	마감: 시작 전 → 미응시(마감), 진행 중·장애 중단 → 응시 종료(마감) (INT-10, D-04).	jobs:closeExpiredPostings · r.admin:close
`newDraft`	함수	211	`(db, adminEmail) -> JobPosting`	빈 임시 공고와 기본 평가 기준을 만든다	r.admin:createPosting

app/services/question_bank.py

역할: 질문은행 엑셀 읽기 — 질문 DB 불러오기 (TRD 7-4 ①, PRD 5-5). manage.py load_questions 가 부른다.

가져다 쓰는 모듈 | 우리 `domain.scoring` `models` | 표준 `re` | 외부 `openpyxl` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`excludedReason`	함수	72	`(text) -> str | None`	다른 회사 이름·관계없는 질문이면 제외 이유	s.posting_questions:makeOne · s.question_bank:excludeOtherCompanies · s.question_bank:loadItQuestions · s.question_bank:loadPersonalityQuestions · 외 1곳 · 테스트 1개 파일
`excludeOtherCompanies`	함수	80	`(db) -> int`	이미 넣은 엑셀 질문 중 걸리는 것을 'excluded'로 바꾼다 (load_questions 때마다 다시 확인).	s.question_bank:loadAll · 테스트 1개 파일
`pickDuplicates`	함수	91	`(ids, pairs) -> dict[int, tuple[int, float]]`	id 순서로 보며 앞에서 남긴 질문과 겹치면 뺀다. {뺄 id: (가장 비슷한 남긴 id, 유사도)}.	s.question_bank:dedupeExcel · 테스트 1개 파일
`dedupeExcel`	함수	107	`(db) -> dict[str, int]`	사용 중인 엑셀 질문끼리 뜻이 겹치면 앞선 것만 남기고 'excluded'로 바꾼다 (CONFLICTS). 임베딩을 만든 뒤에 돌린다.	manage:dedupeQuestions · 테스트 1개 파일

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`normalizeText`	함수	124	`(text) -> str`	비교용: 공백·기호 지우고 소문자	s.posting_questions:isKnown · s.posting_questions:saveCandidate · s.question_bank:loadItQuestions · s.question_bank:loadPersonalityQuestions · 외 4곳 · 🧪 테스트 1개 파일
`cell`	함수	128	`(row, header, name)`	엑셀 행에서 머리글 이름으로 칸 값 꺼내기	s.question_bank:loadItQuestions · s.question_bank:loadPersonalityQuestions
`existingKeys`	함수	135	`(db) -> set[tuple[str, int]]`	이미 넣은 (시트, 행 번호) 목록	s.question_bank:loadAll
`loadItQuestions`	함수	139	`(db, path, keys) -> dict`	IT 질문 엑셀을 읽어 넣는다(코딩·빈 행 건너뜀)	s.question_bank:loadAll
`loadPersonalityQuestions`	함수	172	`(db, path, keys) -> dict`	인성 질문 엑셀을 읽어 인성/기업으로 나눠 넣는다	s.question_bank:loadAll
`loadReserveQuestions`	함수	197	`(db) -> int`	코드에 적힌 예비 질문을 넣는다	s.question_bank:loadAll
`loadAll`	함수	210	`(db, itPath, personalityPath) -> dict`	위 세 가지를 모두 넣고 통계를 돌려준다	manage:loadQuestions

app/services/question_compose.py

역할: 면접 질문 10개 만들기(RAG) — 지원자별 질문 10개 구성 (PRD 5-5, QST-01~05, TRD 7-4 ②~⑥).

가져다 쓰는 모듈

- 🏠 우리 `db` `domain.question_text` `domain.scoring` `models` `evaluation` `llm` `question_bank` `rag` `speech` `storage` `settings_store`
- 🐍 표준 `asyncio` `hashlib` `logging` `re` `time` `typing`
- 📦 외부 `langgraph` `pydantic` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`RagQuestion`	클래스	44	`(BaseModel)`	AI 가 만든 질문 모양(질문, 이력서 근거, 요구 일 수)	s.posting_questions:makeOne · s.question_compose:checkGenerated · s.question_compose:generate · 🧪 테스트 1개 파일
`stableOrder`	함수	91	`(applicationId, questionId) -> str`	지원 건마다 다르지만 늘 같은 순서를 주는 정렬 키	s.question_compose:candidates
`resumeText`	함수	95	`(resume) -> str`	LLM 입력용 이력서 요약. 이름·연락처는 넣지 않는다 (TRD 8-2 ①, 기술 아키텍처 10장).	i.engine:followupDecide · s.question_compose:compose · s.question_compose:experienceText · s.question_compose:isPersonal · 🧪 테스트 1개 파일

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`experienceText`	함수	116	`(resume) -> str`	이력서 질문의 재료이자 근거 검사 범위: 경력·프로젝트·자기소개 줄만. 학력·기술·자격은 '경험'이 아니다 (실측: 기술 이름 'Python, pandas' 를 근거로 신입에게 없는 'pandas 데이터 분석 프로젝트'를 물었다).	s.question_compose:compose · 🧪 테스트 1개 파일
`experienceAnchors`	함수	122	`(resume) -> list[str]`	이력서 질문이 짚어야 할 이름: 프로젝트 이름·경력 회사 이름.	s.question_compose:compose · 🧪 테스트 1개 파일
`mentionsAnchor`	함수	128	`(question, anchors) -> bool`	질문에 이름 하나가 (조각이라도) 들어 있나. 이름이 없으면(자기소개만) 보지 않는다.	s.question_compose:checkGenerated
`groundedLength`	함수	141	`(evidence, experience) -> int`	근거 구절 앞부분이 이력서 경험 글과 몇 글자까지 같은가. 뒤에 붙은 찌꺼기('}·✅...')는 세지 않는다.	s.question_compose:checkGenerated
`companyText`	함수	150	`(posting, brief) -> str`	회사의 상황 = 공고 본문 (회사 소개·주요 업무·자격 요건·우대 사항). brief: 직무 질문용 — 회사 소개만 (업무 줄을 다 넣으면 자리별 기술 주제보다 업무 문구로 쏠림, 실측).	s.posting_questions:makeOne · s.question_compose:compose
`jobTopics`	함수	160	`(posting, resume) -> list[str]`	직무 질문 자리마다 다른 기술 주제 (실측: 같은 찾을 글을 쓰면 4자리가 모두 같은 주제로 쏠림). 공고의 주요 업무·자격 요건·우대 사항 줄을 번갈아 고르고, 모자라면 지원자 기술을 붙인다.	s.posting_questions:generateForPosting · s.question_compose:compose · 🧪 테스트 1개 파일
`queryText`	함수	172	`(kind, posting, resume, topic) -> str`	비슷한 질문을 찾을 글 (TRD 7-4 ③-1). 이력서 기반은 이력서 프로젝트를, 직무는 자리별 기술 주제를, 인성·회사는 공고 글을 중심으로 한다.	s.posting_questions:examplesFor · s.question_compose:examples
`looksLikeQuestion`	함수	183	`(text) -> bool`	질문 모양 검사(8~220자, 한 줄)	s.question_compose:checkGenerated
`candidates`	함수	188	`(db, app, kind, usedGroups, usedIds) -> list[Question]`	질문은행 대체 후보: 같은 종류의 사용 중 질문 중 적용 범위가 맞는 것(자기 공고·같은 직무·공통)을 덜 쓴 순으로	s.question_compose:bankFallback · 🧪 테스트 2개 파일
`reserveFor`	함수	203	`(db, kind, usedGroups, usedIds, chosen) -> Question | None`	묶음(duplicate_group)이 없는 예비 질문도 이미 쓴 것이면 건너뛴다. 묶음만 보면 같은 질문이 여러 자리에 들어갔다 (E11)	s.question_compose:bank · 🧪 테스트 1개 파일
`skillsText`	함수	211	`(resume) -> str`	직무 질문에 넣는 이력서: 기술만 (프로젝트 내용은 넣지 않아 직무 질문이 이력서 질문이 되지 않게).	s.question_compose:compose
`ragPrompt`	함수	219	`(kind, company, resume, examples, avoid, problem, topic) ->`	질문 1개를 만드는 프롬프트: 종류 안내·꼴 예·말끝 안내 + 기술 주제(직무)나 초점(회사) + 공고·이력서·예시 질문·이미 낸 질문 + 이전 답의 문제	s.posting_questions:makeOne · s.question_compose:generate · 🧪 테스트 1개 파일
`checkGenerated`	함수	245	`(kind, out, experience, anchors) -> str | None`	만든 질문 확인: 형식·요구하는 일 1개·글 검사, 직무는 경험 질문 금지, 회사는 입사 전 지원자에게 회사 안 경험 묻기 금지, 이력서는 근거·이름 확인. 문제 코드, 맞으면 None	s.posting_questions:makeOne · s.question_compose:check · 🧪 테스트 1개 파일

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`sameText`	함수	283	`(a, b) -> bool`	띄어쓰기·문장부호만 다른 질문은 같은 질문으로 본다.	s.question_compose:isRepeat · 🧪 테스트 1개 파일
`sameText.norm`	안쪽 함수	285	`(t) -> str`	비교용 정규화	—
`isRepeat`	함수	290	`(text, chosen) -> bool`	이미 고른 질문과 같은 문장인지	s.posting_questions:makeOne · s.question_compose:bankFallback · s.question_compose:check · s.question_compose:lastResort · 외 1곳 · 🧪 테스트 1개 파일
`chosenVectors`	함수	294	`(chosen) -> list[list[float]]`	고른 질문들의 벡터	s.question_compose:bankFallback · s.question_compose:check
`bankFallback`	함수	298	`(db, app, kind, chosen, ctx) -> Question | None`	질문은행(확인된 DB 질문)에서 고른다. 이미 고른 질문과 뜻이 0.9 이상 비슷하면 건너뛴다 (TRD 7-4 ②).	s.question_compose:bank · 🧪 테스트 1개 파일
`SlotState`	클래스	324	`(TypedDict)`	질문 만들기 그래프의 상태	s.question_compose:accept · s.question_compose:afterAccept · s.question_compose:afterCheck · s.question_compose:afterExamples · 외 8곳
`buildComposeGraph`	함수	337	`(db, app, ctx, deadline, settings)`	질문 10칸 채우기 순서도: 고르기 → 예시 → 생성 → 검사 → (실패 시) 은행·예비	s.question_compose:composeAll
`buildComposeGraph.remaining`	안쪽 함수	340	`() -> float`	남은 시간(초)	—
`buildComposeGraph.accept`	안쪽 함수	343	`(state, text, source, question, substituted) -> dict`	질문 하나를 확정해 chosen 에 넣는다	—
`buildComposeGraph.pick`	안쪽 함수	357	`(state) -> dict`	노드: 다음 칸(종류)을 정한다	—
`buildComposeGraph.examples`	안쪽 함수	370	`(state) -> dict`	① 찾기: 공고·이력서로 같은 종류 기존 질문 예시를 찾는다.	—
`buildComposeGraph.generate`	안쪽 함수	397	`(state) -> dict`	② 만들기: 예시 + 이력서 요약 + 회사 상황으로 LLM 이 질문 1개.	—
`buildComposeGraph.check`	안쪽 함수	419	`(state) -> dict`	③ 확인: 형식·요구하는 일 수·이력서 근거 → 예시 베낌·고른 질문과 중복.	—
`buildComposeGraph.bank`	안쪽 함수	451	`(state) -> dict`	질문은행(확인된 엑셀 질문) → 예비 질문. 질문 바꾸기는 감점·장애가 아니다.	—
`buildComposeGraph.afterPick`	안쪽 함수	463	`(state) -> str`	갈림길: RAG 가능하면 examples, 아니면 bank	—
`buildComposeGraph.afterExamples`	안쪽 함수	466	`(state) -> str`	갈림길: 예시가 있으면 generate, 없으면 bank	—

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`buildComposeGraph.afterGenerate`	안쪽 함수	469	`(state) -> str`	갈림길: 시간 초과면 bank, 아니면 check	—
`buildComposeGraph.afterCheck`	안쪽 함수	472	`(state) -> str`	갈림길: 통과면 다음 칸, 실패면 재생성 또는 bank	—
`buildComposeGraph.afterAccept`	안쪽 함수	479	`(state) -> str`	갈림길: 10칸이 다 차면 끝	—
`composeAll`	async 함수	494	`(db, app, ctx, deadline, settings) -> list[dict]`	그래프를 돌려 10개를 만든다	s.question_compose:compose
`lastResort`	함수	515	`(chosen) -> str`	고정 문장 중 아직 안 쓴 것	s.question_compose:bank · 🧪 테스트 1개 파일
`synthesizeAll`	async 함수	522	`(db, app) -> None`	기본 질문 음성을 미리 만든다. 실패한 질문은 자막만으로 낸다 (INT-01).	s.question_compose:compose
`compose`	async 함수	538	`(applicationId, budgetSec) -> None`	지원 건의 질문 10개를 만들고 저장 (TTS 포함), 상태를 ready/failed 로	s.question_compose:ensureComposed · s.question_compose:scheduleCompose · 🧪 테스트 1개 파일
`countUsage`	함수	588	`(db, postingId, questionIds) -> None`	공고별 사용 횟수 +1 (TRD 7-4 ②). 동시에 두 지원자가 같은 질문을 골라도 충돌하지 않게 INSERT … ON CONFLICT.	s.question_compose:compose · 🧪 테스트 1개 파일
`personalTerms`	함수	611	`(app) -> set[str]`	지원자를 가리킬 수 있는 말: 이름·학교·회사·프로젝트·자격 이름, 그리고 공고의 회사명 (U1 회사명 표기 안 함).	s.question_compose:savePending · 🧪 테스트 1개 파일
`isPersonal`	함수	622	`(text, terms, app) -> bool`	질문에 지원자 고유의 말이나 이력서 문구가 들어 있으면 True (질문 DB 에 넣지 않는다).	s.question_compose:savePending · 🧪 테스트 1개 파일
`scopeFor`	함수	631	`(kind, posting) -> dict`	질문 적용 범위: 회사 질문 → 그 공고 (posting_id), 직무 질문 → 그 직무 (job_code), 인성 → 공통	s.posting_questions:saveCandidate · s.question_compose:savePending
`savePending`	함수	640	`(db, app, chosen) -> None`	면접 때 AI 가 만든 인성·회사·직무 질문 중 개인 정보가 없고 질문 DB 와 겹치지 않는 것을 적용 범위를 붙여 '검토 대기'로 넣는다	s.question_compose:compose · 🧪 테스트 2개 파일
`scheduleCompose`	함수	669	`(applicationId) -> None`	질문 만들기를 백그라운드로 예약(중복 방지)	main:recoverAfterRestart · r.applications:submitResume · r.interview:startInterview
`cancelCompose`	함수	675	`(applicationId) -> None`	지운 지원의 질문을 계속 만들면 LLM 을 쓰느라 다른 지원자의 질문 준비가 밀린다.	s.withdrawal:withdrawApplication
`ensureComposed`	async 함수	682	`(applicationId, waitSec) -> None`	면접 시작 시: 아직이면 최대 waitSec 기다리고, 넘으면 LLM 없이 남은 자리를 채운다 (TRD 7-4 ⑥).	i.engine:prepareQuestions

app/services/rag.py

역할: 임베딩과 벡터 검색 — 질문 DB 임베딩과 비슷한 질문 찾기 (TRD 7-4 ①②③, 테이블 정의서 4-16 questions.embedding).

가져다 쓰는 모듈 | 우리 `config` | 표준 `dataclasses` `logging` `math` | 외부 `httpx` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`EmbedUnavailable`	클래스	32	`(Exception)`	임베딩 서버를 쓸 수 없음	s.posting_questions:examplesFor · s.posting_questions:makeOne · s.question_compose:check · s.question_compose:examples · 외 3곳 · 🧪 테스트 1개 파일
`Example`	클래스	37	``	비슷한 기존 질문 하나(문장, 유사도, 벡터)	s.posting_questions:examplesFor · s.question_compose:ragPrompt · s.rag:similar · 🧪 테스트 1개 파일
`supported`	함수	44	`(db) -> bool`	벡터 검색 가능(PostgreSQL)한지	s.posting_questions:loadWebCandidates · s.posting_questions:makeOne · s.question_bank:dedupeExcel · s.question_compose:compose · 외 4곳
`toLiteral`	함수	48	`(vector) -> str`	벡터를 pgvector 글 형식으로	s.rag:indexQuestions · s.rag:maxStoredSimilarity · s.rag:similar · s.rag:storeVector
`parseVector`	함수	52	`(raw) -> list[float]`	pgvector 글을 숫자 목록으로	s.rag:similar · s.rag:vectorsFor
`cosine`	함수	58	`(a, b) -> float`	두 벡터의 코사인 유사도	s.rag:maxSimilarity
`maxSimilarity`	함수	64	`(vector, others) -> float`	여러 벡터 중 가장 높은 유사도	s.posting_questions:makeOne · s.question_compose:bankFallback · s.question_compose:check · 🧪 테스트 1개 파일
`parseResponse`	함수	68	`(res, count) -> list[list[float]]`	임베딩 서버 응답 검사(개수·차원 1024)	s.rag:embed · s.rag:embedSync

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`embed`	async 함수	77	`(texts, timeoutSec) -> list[list[float]]`	글 목록을 임베딩(비동기)	s.posting_questions:examplesFor · s.posting_questions:makeOne · s.question_compose:check · s.question_compose:examples
`embedSync`	함수	86	`(texts, timeoutSec) -> list[list[float]]`	글 목록을 임베딩(동기, 관리 명령용)	s.posting_questions:loadWebCandidates · s.rag:indexQuestions
`health`	async 함수	94	`(timeoutSec) -> bool`	임베딩 서버가 살아 있는지	s.question_compose:compose
`indexedCount`	함수	102	`(db) -> int`	벡터가 저장된 질문 수	manage:embedQuestions · s.question_compose:compose
`indexQuestions`	함수	108	`(db) -> int`	임베딩이 없는 질문을 모두 숫자 목록으로 바꿔 넣는다 (manage.py embed_questions). 다시 돌려도 남은 것만 한다.	manage:embedQuestions
`similar`	함수	128	`(db, slotKind, vector, count, excludeIds) -> list[Example]`	같은 종류의 쓰는 질문(엑셀) 중 뜻이 가장 가까운 것 count 개 (RAG 예시, TRD 7-4 ③-2).	s.posting_questions:examplesFor · s.question_compose:examples
`maxStoredSimilarity`	함수	139	`(db, kind, vector) -> float`	같은 종류 질문 DB 전체(상태 무관 — 거절한 질문도 다시 들어오지 않게)에서 가장 비슷한 정도.	s.posting_questions:makeOne · s.question_compose:savePending
`similarPairs`	함수	147	`(db, kind, threshold) -> list[tuple[int, int, float]]`	같은 종류의 사용 중 엑셀 질문 중 threshold 이상 비슷한 쌍 (앞 id, 뒤 id, 유사도).	s.question_bank:dedupeExcel
`storeVector`	함수	156	`(db, questionId, vector) -> None`	질문 하나의 벡터 저장	s.posting_questions:saveCandidate · s.question_compose:savePending
`vectorsFor`	함수	160	`(db, ids) -> dict[int, list[float]]`	질문 ID 들의 벡터 읽기	s.question_compose:accept · s.question_compose:bankFallback

app/services/recording.py

역할: 카메라 녹화 조각 저장 — 카메라 녹화 조각 저장·재생·보관 기한 (API 명세서 6장·2-6, 테이블 정의서 4-10). 녹화는 평가에 쓰지 않는다.

가져다 쓰는 모듈 | 우리 `domain` `errors` `models` `timeutil` | 표준 `datetime` `logging` | 외부 `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`chunkKey`	함수	24	`(fileKey, chunkNo) -> str`	녹화 조각 파일 이름(번호 5자리)	s.recording:saveChunk

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`baseExpiry`	함수	28	`(app)`	응시가 끝났으면 끝난 시각, 아직이면 마감 시각(가장 늦게 끝날 수 있는 시각) + 3일.	s.recording:expiresAt · s.recording:saveChunk
`expiresAt`	함수	34	`(rec, app)`	기록된 기한과 '끝난 시각 + 3일' 중 이른 쪽. 조각이 더 오지 않아 기록된 기한이 마감 기준으로 남아 있어도 3일에 지운다.	r.admin:applicantDetail · r.admin:recordingVideo · s.retention:purgeRecordings
`acceptsUploads`	함수	39	`(app) -> bool`	녹화 조각을 받을 수 있는 상태인지 (진행 중 또는 끝난 직후 잠깐)	s.recording:saveChunk
`saveChunk`	함수	46	`(db, session, app, segmentNo, chunkNo, data) -> None`	녹화 조각 저장(상태·구간 번호 검사)	r.interview:recordingChunk
`uploadFailureReason`	함수	83	`(db, app) -> str`	화면이 '녹화 업로드 실패'를 알렸을 때의 장애 사유. 화면이 보낸 원인은 믿지 않고(이어가기 횟수 우회 방지) 서버가 최근에 저장소 오류를 기록했을 때만 서버 쪽 장애(server_error, 이어가기 횟수에 안 셈)로 본다.	i.engine:onText · 🧪 테스트 1개 파일
`streamSegment`	함수	91	`(fileKey)`	녹화 구간 조각들을 순서대로 이어 내보낸다	r.admin:recordingVideo

app/services/resume_extract.py

역할: 이력서 읽기(PDF·OCR·LLM) — 이력서 파일 → 항목별 정리 (TRD 7-6, API 명세서 resume/upload).

가져다 쓰는 모듈
🏠 우리 `config` `models` `pdf_box` `settings_store` `timeutil`
🐍 표준 `asyncio` `dataclasses` `json` `logging` `os` `re` `sys` `time`
📦 외부 `httpx` `pydantic` `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`detectType`	함수	51	`(fileName, head) -> str | None`	확장자와 파일 앞부분의 실제 형식이 같을 때만 받는다 (이름만 바꾼 파일 거르기).	r.applications:acceptUpload
`PageText`	클래스	66	``	쪽 하나의 글과 읽은 방법 (pymupdf/ocr 등)	s.resume_extract:extractUpload · s.resume_extract:joined · s.resume_extract:ocrPages · s.resume_extract:readPages
`BoxError`	클래스	73	`(Exception)`	code 는 업로드 응답의 오류 코드가 된다. 메시지는 내부 기록용.	r.applications:acceptUpload · s.resume_extract:ocrPages · s.resume_extract:runBox · 🧪 테스트 1개 파일
`BoxError.__init__`	메소드	76	`(self, message, code)`	오류 코드 저장	—
`boxEnv`	함수	84	`() -> dict[str, str]`	비밀값을 물려주지 않는다.	s.resume_extract:runBox

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`runBox`	async 함수	89	`(args, data) -> bytes`	pdf_box 를 별도 프로세스로 돌린다. 시간을 넘기거나 비정상 종료하면 BoxError.	s.resume_extract:readPages · s.resume_extract:renderPage
`readPages`	async 함수	105	`(data) -> list[PageText]`	격리 프로세스(pdf_box)로 PDF 글 읽기	s.resume_extract:countPages · s.resume_extract:extractUpload · 🧪 테스트 2개 파일
`countPages`	async 함수	109	`(data, contentType) -> tuple[int, int, str]`	(글자 있는 쪽 수, OCR 이 필요한 쪽 수, 리더로 읽은 글자). 업로드 직후 예상 시간 계산에 쓴다.	r.applications:acceptUpload · 🧪 테스트 1개 파일
`estimateSeconds`	함수	118	`(textPages, scannedPages, ocrSecPerPage, llmSec) -> int`	예상 시간 = 글 쪽×0.5 + 스캔 쪽 ×OCR 시간 + LLM 시간	r.applications:acceptUpload
`renderPage`	async 함수	124	`(data, pageNo) -> bytes`	격리 프로세스로 PDF 쪽을 이미지로	s.resume_extract:ocrPages · 🧪 테스트 1개 파일
`OcrError`	클래스	128	`(Exception)`	OCR 실패	s.resume_extract:extractUpload · s.resume_extract:ocrImage · s.resume_extract:ocrPages · 🧪 테스트 2개 파일
`ocrImage`	async 함수	132	`(image) -> dict`	OCR 서비스 호출 → {"text", "lines": [{text, conf}], "confidence"}.	s.resume_extract:ocrPages
`ocrPages`	async 함수	144	`(data, contentType, pageNos) -> list[tuple[PageText, list[di`	스캔 쪽을 OCR 로 읽는다(이미지면 그대로, PDF 면 렌더 후)	s.resume_extract:extractUpload · 🧪 테스트 1개 파일
`Education`	클래스	158	`(BaseModel)`	학력 칸	—
`Career`	클래스	166	`(BaseModel)`	경력 칸	—
`Project`	클래스	174	`(BaseModel)`	프로젝트 칸	—
`Certificate`	클래스	181	`(BaseModel)`	자격증 칸	—
`StructuredResume`	클래스	186	`(BaseModel)`	정리된 이력서 전체 모양	s.resume_extract:structure
`repairDateRanges`	함수	211	`(lines) -> list[str]`	표 칸이 좁아 '2012년 3월 ~' / '2025년 8월' 처럼 두 줄로 갈린 기간을 한 줄로 붙인다 (뒤 두 줄까지 본다).	s.resume_extract:tidyText
`tidyText`	함수	229	`(text) -> str`	표 배치 때문에 생긴 공백·빈 줄을 줄이고 갈린 기간을 붙인다.	s.resume_extract:structure · 🧪 테스트 1개 파일
`splitCoverLetter`	함수	235	`(text) -> tuple[str, str]`	'자기소개서' 제목 뒤는 원문 그대로 자기소개서 칸에 넣는다 (LLM 이 다시 쓰지 않게: 빠르고 잘리지 않음).	s.resume_extract:structure · 🧪 테스트 1개 파일
`cleanResult`	함수	246	`(resume) -> dict`	LLM 결과 뒷정리: 띄어 쓴 이름 붙이기, 채우지 않은 양식 프로젝트·'졸업'만 든 전공 지우기.	s.resume_extract:structure · 🧪 테스트 1개 파일
`structure`	async 함수	259	`(text, timeoutSec) -> dict`	이력서 글을 LLM 으로 칸에 나눠 담는다	s.resume_extract:extractUpload
`missingFields`	함수	275	`(resume) -> list[str]`	지원에 꼭 필요한 칸(성명, 학력 또는 경력 1건 — APP-05) 중 빈 것.	s.resume_extract:extractUpload · 🧪 테스트 1개 파일
`fillBlanks`	함수	285	`(base, extra) -> dict`	base 의 빈 칸만 extra 값으로 채운다. 이미 읽은 값은 덮어쓰지 않는다.	s.resume_extract:extractUpload · 🧪 테스트 1개 파일

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`lowConfidenceFields`	함수	295	`(resume, ocrLines) -> list[str]`	OCR 신뢰도가 LOW_CONF 미만인 줄에서 온 값의 위치 (예: career[0].company).	s.resume_extract:extractUpload · 🧪 테스트 1개 파일
`lowConfidenceFields.visit`	안쪽 함수	302	`(value, path) -> None`	OCR 신뢰도 낮은 줄에서 온 칸을 찾는 재귀 탐색	—
`setStage`	함수	319	`(db, upload, stage, **fields) -> None`	업로드 진행 단계를 바꾸고 저장	s.resume_extract:extractUpload
`joined`	함수	326	`(pages) -> str`	쪽 글을 순서대로 합친다	s.resume_extract:extractUpload
`failUnfinished`	함수	330	`(db) -> int`	추출은 api 프로세스 안의 작업이라 재시작하면 사라진다. 기동 때 불러 화면이 '처리 중'에 남지 않게 실패로 바꾼다.	main:recoverAfterRestart
`runExtraction`	async 함수	341	`(uploadId) -> None`	동시 추출 수를 제한하며 extractUpload 실행	r.applications:acceptUpload · 🧪 테스트 1개 파일
`extractUpload`	async 함수	346	`(uploadId) -> None`	업로드 하나 추출 전체: 읽기 → OCR → LLM 정리 → 결과 저장	s.resume_extract:runExtraction
`splitResult`	함수	426	`(stored) -> tuple[dict | None, dict]`	저장된 결과에서 화면용 이력서와 OCR 정보(_ocr)를 나눈다.	r.applications:uploadStatus · 🧪 테스트 1개 파일
`resumeLlmSec`	함수	434	`(db) -> int`	LLM 정리 예상 시간 설정값	—

app/services/retention.py

역할: 보관 기간 지난 데이터 삭제 — 보관 기간이 끝난 면접 데이터 삭제 (PRD 10-2, TRD 4-2·10장, 테이블 정의서 1-6·4-13).

가져다 쓰는 모듈 | 🏠 우리 `domain` `interview` `models` `timeutil` | 🐍 표준 `collections` `datetime` `decimal` `logging` | 📦 외부 `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`filePrefixes`	함수	50	`(db, app) -> list[str]`	지원 건 하나의 파일 경로 앞부분들 (답변·질문 음성·녹화·이력서)	s.retention:purgeApplication
`archiveStats`	함수	56	`(db, app) -> None`	지우기 전에 통계 숫자만 StatsArchive에 더한다 (CONFLICTS F1). 관리자 통계(/admin/stats)와 같은 기준으로 센다.	s.retention:purgeApplication
`purgeApplication`	함수	93	`(db, app, redisClient, cause) -> dict`	한 지원 건의 면접 데이터를 지우고 삭제 기록을 남긴다. 파일을 먼저 지워 DB 만 지워지고 파일이 남는 일이 없게 한다.	s.retention:purgeDue · s.withdrawal:withdrawApplication
`purgeRecordings`	함수	113	`(db, at) -> int`	보관 기한(응시 끝 + 3일)이 지난 녹화 파일·행을 지운다. 지원 건은 남긴다. 실패한 건은 다음 날 다시.	worker:purgeRecordingsTask · 🧪 테스트 1개 파일

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`purgeDue`	함수	139	`(db, redisClient, at) -> list[str]`	purge_at 이 지난 지원 건을 지운다. 한 건이 실패해도 다음 건은 계속 지운다 (다음 날 다시 시도). '확인 필요' 건은 purge_at 이 없어 지우지 않는다 — 결과가 정해질 때(최대 7일, CONFLICTS U8) 삭제일도 정해진다.	worker:purgeTask · 🧪 테스트 2개 파일
`adminNoticeRows`	함수	161	`(db, adminIds, at) -> list[tuple]`	삭제 2일 전 관리자 안내: 그날 삭제될 건을 공고별로 묶어 한 통씩 (TRD 10장, 매일 09:00).	worker:purgeNoticeTask · 🧪 테스트 1개 파일

app/services/sandbox_runner.py

역할: 코드 격리 실행 — 코드 실행 상자 (TRD 9장, ADR-09). 고정 이미지·고정 옵션만 쓰도록 이 한 곳에 모은다 (R-06).

가져다 쓰는 모듈 | 표준 `ast` `base64` `json` `secrets` `threading` | 외부 `docker`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`SandboxBusy`	클래스	65	`(Exception)`	실행 자리가 없음	r.interview:codingRun · s.coding:gradeAll · s.coding:runExamples · s.sandbox_runner:run · 외 1곳
`SandboxInfraError`	클래스	69	`(Exception)`	인프라 오류는 오답이 아니다 (기술 아키텍처 6-6).	r.interview:codingRun · s.coding:gradeAll · s.coding:runExamples · s.sandbox_runner:run · 외 1곳 · 🧪 테스트 1개 파일
`expectedRepr`	함수	73	`(expected) -> str`	기대 출력 글을 비교용 repr 로	s.sandbox_runner:judge
`judge`	함수	77	`(raw, cases) -> dict`	상자가 돌려준 실행 결과를 기대값과 비교한다. 형식이 틀리면 실행 오류로 본다.	s.sandbox_runner:run · 🧪 테스트 1개 파일
`readLogs`	함수	105	`(container) -> str`	무한 출력으로 api 메모리가 차지 않도록 앞쪽 상한까지만 읽고, 표식이 있는 끝부분은 따로 읽는다	s.sandbox_runner:run
`run`	함수	116	`(code, cases) -> dict`	격리 컨테이너를 띄워 코드를 테스트 케이스로 돌리고 결과를 모은다	s.coding:gradeAll · worker:sandboxRunTask

app/services/speech.py

역할: STT·TTS 호출 — STT · TTS 내부 서비스 호출. 시간 제한 필수 (기술 아키텍처 4-3).

가져다 쓰는 모듈 | 우리 `config` | 외부 `httpx`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`SpeechError`	클래스	8	`(Exception)`	STT/TTS 실패	i.engine:createFollowup · i.engine:transcribeWithRetry · s.question_compose:synthesizeAll · s.speech:synthesize · 외 1곳
`transcribe`	async 함수	12	`(pcm, timeoutSec) -> dict`	16kHz mono PCM16 → {text, words, elapsed_sec}.	i.engine:transcribeWithRetry
`synthesize`	async 함수	24	`(text, timeoutSec) -> tuple[bytes, float]`	한국어 문장 → (WAV 바이트, 길이 초).	i.engine:createFollowup · s.question_compose:synthesizeAll
`serviceHealthy`	async 함수	36	`(url, timeoutSec) -> bool`	음성 서버 /health 가 살아 있는지	r.interview:precheck

app/services/storage.py

역할: 파일 저장소(RustFS, 암호화) — 파일 저장소 (S3 호환 RustFS). 모든 객체는 앱에서 AES-256-GCM 으로 암호화해 넣는다 (TRD 4-2).

가져다 쓰는 모듈 | 우리 `crypto` | 표준 `os` | 외부 `boto3` `botocore` (boto3) `cryptography`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`client`	함수	20	`()`	RustFS(S3 호환) 연결	s.storage:deletePrefix · s.storage:ensureBucket · s.storage:getObject · s.storage:listKeys · 외 1곳
`ensureBucket`	함수	34	`() -> None`	버킷이 없으면 만든다	main:lifespan
`putObject`	함수	41	`(key, data, contentType) -> None`	파일을 AES-GCM 으로 암호화해 올린다(키 이름을 AAD 로 묶음)	i.engine:createFollowup · i.engine:saveAnswer · r.applications:acceptUpload · s.question_compose:synthesizeAll · 외 1곳
`getObject`	함수	47	`(key) -> tuple[bytes, str]`	파일을 내려받아 복호화한다	r.admin:answerAudio · r.applications:resumeFile · r.interview:questionAudio · s.recording:streamSegment · 외 1곳
`listKeys`	함수	57	`(prefix) -> list[str]`	경로 앞부분으로 파일 목록	s.recording:saveChunk · s.recording:streamSegment

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`deletePrefix`	함수	64	`(prefix) -> int`	경로 앞부분 아래 파일을 모두 지운다	r.applications:cancelUpload · s.retention:purgeApplication · s.retention:purgeRecordings · s.withdrawal:withdrawAccount

app/services/withdrawal.py

역할: 동의 철회·회원 탈퇴 — 동의 철회 (APP-03) · 회원 탈퇴 (S-A13). 진행 중 응시는 취소하고 자리를 돌려준 뒤 바로 지운다.

가져다 쓰는 모듈 | 우리 `domain` `interview` `models` `timeutil` | 외부 `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`withdrawApplication`	async 함수	16	`(db, app, cause) -> None`	cause: consent(동의 철회) · withdraw(탈퇴).	r.applications:withdrawConsent · s.withdrawal:withdrawAccount · 테스트 1개 파일
`withdrawAccount`	async 함수	28	`(db, user) -> None`	회원 탈퇴: 지원 건마다 철회 처리, 이력서 파일·계정 삭제	r.auth:withdrawMe

A-6. interview — 면접 엔진

app/interview/checkpoint.py

역할: LangGraph 체크포인트 저장소 — LangGraph 체크포인트 저장소 (ADR-07, TRD 7-5, 테이블 정의서 LangGraph 체크포인트 표 3개).

가져다 쓰는 모듈 | 우리 `config` | 표준 `logging`
외부 `langgraph` `psycopg` `psycopg_pool` (psycopg-pool) `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`conninfo`	함수	26	`() -> str | None`	LangGraph 체크포인트용 PostgreSQL 주소. SQLite 면 None	i.checkpoint:openSaver
`openSaver`	async 함수	33	`() -> None`	api 시작 때 한 번. 체크포인트 표가 없으면 만든다.	main:lifespan
`closeSaver`	async 함수	52	`() -> None`	체크포인트 연결 풀을 닫는다(앱 종료 때)	main:lifespan
`saver`	함수	60	`() -> BaseCheckpointSaver`	체크포인트 저장소를 돌려준다. 준비 전이면 메모리 저장소(테스트용)	i.engine:runInterview · s.posting_draft:buildDraftGraph

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`deleteThread`	함수	67	`(db, threadId) -> int`	지원 1건의 체크포인트를 같은 트랜잭션에서 지운다 (보관 삭제·동의 철회·탈퇴). 지운 체크포인트 수.	r.admin:deletePosting · r.admin:draftAi · r.admin:publish · s.retention:purgeApplication

app/interview/engine.py

역할: 면접 진행 엔진(WebSocket 한 건 = LiveSession) — 면접 진행 (TRD 5-2 · 7-2 · 7-3 · 7-5, API 명세서 7장, 기술 아키텍처 6-3 · 6-4).

가져다 쓰는 모듈

- 우리 `config` `db` `domain.question_text` `domain.scoring` `domain.status` `checkpoint` `graph` `lifecycle` `registry` `models` `services.coding` `services.evaluation` `services.llm` `services.postings` `services.question_compose` `services.recording` `services.speech` `services.storage` `settings_store` `timeutil`
- 표준 `asyncio` `contextlib` `datetime` `json` `logging` `time` `typing`
- 외부 `fastapi` `langgraph` `pydantic` `redis`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`SessionEnded`	클래스	51	`(Exception)`	세션이 이미 끝났음을 알리는 내부 신호	i.engine:collectAnswer · i.engine:endWith · i.engine:enter · i.engine:guard · 외 3곳
`FollowupDecision`	클래스	55	`(BaseModel)`	이유를 먼저 고르게 하고, 꼬리질문 여부는 이유에서 정한다.	i.engine:followupDecide · 🧪 테스트 1개 파일
`FollowupDecision.need`	메소드	62	`(self) -> bool`	꼬리질문이 필요한지(reason_code 가 none 이 아닌지)	—
`yesNoFollowup`	함수	85	`(questionText, transcript) -> str | None`	예/아니오 질문에 30자보다 짧게 답했으면 LLM 없이 '그렇게 답하신 이유를 구체적으로 말씀해 주세요.'를 꼬리질문으로 정한다	i.engine:followupDecide · 🧪 테스트 1개 파일
`followupPrompt`	함수	92	`(questionText, fencedAnswer, resume) -> str`	꼬리질문 판단 프롬프트. 본질문이 예/아니오 질문이면 '이유가 없으면 이유를 물어라'를 덧붙인다	i.engine:followupDecide · 🧪 테스트 1개 파일
`dbSession`	함수	103	`()`	with 문으로 쓰는 짧은 DB 세션	i.engine:**init** · i.engine:advance · i.engine:beginQuestion · i.engine:codingTimeOver · 외 21곳
`isEmptyAnswer`	함수	111	`(text) -> bool`	답변이 사실상 비었는지(4글자 미만, '모르겠습니다' 류)	i.engine:followupPreflight · i.engine:saveAnswer
`followupLimitCause`	함수	116	`(db, app, base, totalLimit) -> str | None`	꼬리질문 상한: 전체 · 질문당 1개 · 질문 종류별 1개. 앞 질문이 상한을 다 써서 직무 질문이 빠지지 않게 종류별로 나눈다 (INT-06).	i.engine:followupPreflight · 🧪 테스트 1개 파일
`LiveSession`	클래스	128	``	WebSocket 연결 하나 = 면접 한 건을 맡는 객체 (질문 보내기, 음성 받기, 끊김·이탈 판정)	r.interview:interviewSocket · 🧪 테스트 1개 파일

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`LiveSession.__init__`	메소드	129	`(self, ws, applicationId, sessionId)`	연결·지원 ID·세션 ID·상태값 초기화	—
`LiveSession.send`	메소드	158	`(self, message) -> None`	화면으로 JSON 메시지를 보낸다 (동시에 보내지 않게 잠금). 끝난 뒤엔 종료·중단 메시지만	i.engine:captionRelay · i.engine:endWith · i.engine:finishAnswer · i.engine:judgeConnectionLoss · 외 8곳
`LiveSession.closeWs`	메소드	167	`(self) -> None`	WebSocket 을 정상 종료(1000)로 닫는다	i.engine:endWith · i.engine:pauseForFault · i.engine:terminateFromOutside · i.engine:watchdog
`LiveSession.serve`	메소드	175	`(self) -> None`	세션 시작점. 같은 지원 건의 이전 세션을 끝내고 등록한 뒤 메시지를 받는다	—
`LiveSession.shutdown`	메소드	206	`(self) -> None`	상태를 ended 로 바꾸고 돌던 작업을 모두 멈춘다	i.engine:serve
`LiveSession.guard`	메소드	225	`(self, coro) -> None`	작업 중 SessionEnded 가 나면 조용히 끝낸다	i.engine:serve
`LiveSession.judgeAfterDisconnect`	메소드	236	`(self) -> None`	끊긴 지 heartbeat 판정 시간(15초)이 되는 순간 판단한다 (TRD 7-2, D-09).	i.engine:serve
`LiveSession.judgeConnectionLoss`	메소드	244	`(self, reason, lookbackExtra) -> None`	끊긴 뒤 '나가요' 기록이 있는지 보고 응시 종료 / 장애 중단을 정한다	i.engine:judgeAfterDisconnect · i.engine:watchdog
`LiveSession.endWith`	메소드	268	`(self, target, reason, detail, message) -> None`	응시를 정해진 최종 상태로 끝내고 기록·자리 반환·채점 예약을 한다	i.engine:finishInterview · i.engine:judgeDeviceEnded · i.engine:onAway · i.engine:onDeviceError · 외 2곳
`LiveSession.pauseForFault`	메소드	281	`(self, reason) -> None`	장애 중단으로 바꾸고 화면에 paused(남은 이어가기 횟수 포함)를 보낸다	i.engine:guard · i.engine:judgeDeviceEnded · i.engine:onDeviceError · i.engine:onText · 외 1곳
`LiveSession.terminateFromOutside`	메소드	300	`(self, reason, detail) -> None`	상태는 부른 쪽이 이미 바꿨다. 화면에 알리고 연결을 닫는다.	i.engine:codingTimeOver
`LiveSession.onAudio`	메소드	313	`(self, chunk) -> None`	경고창이 떠 있는 동안 들어온 음성은 버린다 (기술 아키텍처 6-4, API 명세서 7-6)	i.engine:serve
`LiveSession.onText`	메소드	329	`(self, text) -> None`	화면에서 온 글 메시지를 종류별로 나눠 처리한다(heartbeat·away·답변 완료 등)	i.engine:serve
`LiveSession.enter`	메소드	370	`(self, state) -> None`	러너 단계 전환. 연결이 끊겨 판정을 기다리는 중이면 더 진행하지 않는다.	i.engine:beginQuestion · i.engine:finishAnswer · i.engine:prepareQuestions
`LiveSession.runSafely`	메소드	376	`(self, coro) -> None`	작업을 돌리다 세션이 끝나면 남은 작업을 정리한다	i.engine:onAway · i.engine:onDeviceError · i.engine:onText · i.engine:watchdog
`LiveSession.onAway`	메소드	383	`(self, kind, message) -> None`	화면 이탈 신호 처리: 3초 안 신호는 합치고, 1번째 경고·2번째 응시 취소	i.engine:onText
`LiveSession.onWarnAck`	메소드	426	`(self) -> None`	경고 확인을 받으면 멈췄던 시간을 이어서 흐르게 한다	i.engine:onText

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`LiveSession.onPermissionRevoked`	메소드	448	`(self, device) -> None`	권한 직접 해제 → 장애 아님 → 응시 종료 (TRD 7-2, OV-07).	i.engine:onText
`LiveSession.onDeviceError`	메소드	456	`(self, message) -> None`	어느 장치가 어떻게 멈췄는지 남긴다 (장애 중단 기록에는 'device_error' 만 남아 원인을 알 수 없었다, 2026-11-03)	i.engine:onText
`LiveSession.judgeDeviceEnded`	메소드	474	`(self) -> None`	장치가 꺼진 뒤 3초 기다렸다가 '나가요' 기록을 보고 판정한다(E18)	i.engine:onDeviceError
`LiveSession.watchdog`	메소드	487	`(self) -> None`	1초마다 heartbeat 가 끊겼는지 본다 (간격×허용 횟수 초과면 끊김)	i.engine:serve
`LiveSession.codingTimeOver`	메소드	514	`(self) -> bool`	60분이 끝나면 서버가 마지막 저장 코드로 자동 제출한다 (COD-04).	i.engine:watchdog
`LiveSession.captionRelay`	메소드	526	`(self) -> None`	STT 실시간 자막을 Redis 에서 받아 화면으로 넘긴다	i.engine:serve
`LiveSession.prepareQuestions`	메소드	542	`(self) -> None`	질문 10개가 준비 안 됐으면 지금 만든다(시간 제한 안에서)	—
`LiveSession.baseRow`	메소드	560	`(self, db, seq) -> InterviewQuestion`	seq 번째 기본 질문 행을 찾는다	i.engine:locate
`LiveSession.followupRow`	메소드	563	`(self, db, seq) -> InterviewQuestion | None`	seq 번째 질문에 달린 꼬리질문 행 (없으면 None)	i.engine:locate
`LiveSession.isAnswered`	메소드	566	`(self, db, row) -> bool`	그 질문에 답변이 저장됐는지	i.engine:locate
`LiveSession.runInterview`	메소드	569	`(self) -> None`	면접 진행 그래프를 돌린다 (graph.py). 답변 기다리기에서 그래프가 멈추면 여기서 답변을 받고 잇는다.	i.engine:serve
`LiveSession.locate`	메소드	578	`(self) -> dict`	원장(applications · interview_questions · answers)에서 지금 낼 질문을 찾는다.	—
`LiveSession.advance`	메소드	593	`(self, seq) -> None`	현재 질문 번호를 다음으로 올린다	—
`LiveSession.finishInterview`	메소드	599	`(self) -> None`	질문을 다 마쳤을 때: 코딩테스트가 있으면 코딩 단계로, 없으면 응시 완료	—
`LiveSession.sendCodingPhase`	메소드	613	`(self) -> None`	코딩 타이머를 다시 켜고 코딩 문제·남은 시간을 화면에 보낸다	i.engine:finishInterview · i.engine:runCoding
`LiveSession.runCoding`	메소드	624	`(self) -> None`	코딩테스트는 HTTP API 로 진행하고, 이 연결은 heartbeat·이탈 감시만 한다.	i.engine:finishInterview · i.engine:serve
`LiveSession.sendQuestion`	메소드	631	`(self, row, resend) -> None`	질문(음성 주소·자막)을 보내고 답변 마감 시각을 계산한다	i.engine:beginQuestion · i.engine:onWarnAck
`LiveSession.questionRow`	메소드	642	`(self, rowId) -> InterviewQuestion`	질문 행 하나를 ID로 읽는다	i.engine:finishAnswer
`LiveSession.beginQuestion`	메소드	646	`(self, rowId) -> None`	질문 내기: 응시 횟수·기준 잠금, 경고창이 닫히기를 기다린 뒤 질문을 보낸다.	—
`LiveSession.collectAnswer`	메소드	663	`(self) -> str`	답변 기다리기 (그래프가 interrupt 로 멈춘 동안). 끝난 이유: done · timeout.	i.engine:runInterview
`LiveSession.finishAnswer`	메소드	684	`(self, rowId, endReason) -> dict`	글자로 바꾸기(STT) → 답변 저장 → 최종 자막.	—

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`LiveSession.transcribeWithRetry`	메소드	696	`(self, pcm) -> tuple[str, list, float]`	답변 음성을 STT 로 바꾼다. 0.3초 미만은 무음, 실패하면 설정 횟수만큼 재시도	i.engine:finishAnswer
`LiveSession.saveAnswer`	메소드	710	`(self, row, pcm, transcript, words, sttSec, endReason) -> No`	답변 음성(wav)을 저장소에, 글·시간을 DB 에 저장한다	i.engine:finishAnswer
`LiveSession.followupBudget`	메소드	726	`(self, answeredAt) -> float`	꼬리질문 판단 시간 = 꼬리질문 목표 시간 − 지난 시간 − TTS 평균 (TRD 7-5, 목표 시간은 CONFLICTS U7).	i.engine:followupDecide · i.engine:followupPreflight
`LiveSession.answerText`	메소드	730	`(self, db, rowId) -> str`	저장된 답변 글을 읽는다	i.engine:followupDecide · i.engine:followupPreflight
`LiveSession.followupPreflight`	메소드	734	`(self, rowId, answeredAt) -> str | None`	규칙으로 정할 수 있으면 LLM 을 부르지 않는다 (AI 아키텍처 P4). 건너뛸 이유, 아니면 None.	—
`LiveSession.followupDecide`	메소드	748	`(self, rowId, answeredAt) -> dict`	T4 꼬리질문 판단 (RT 레인, 남은 시간을 timeout 으로). 이력서 기반 질문은 이력서 요약을 근거로 함께 준다.	—
`LiveSession.createFollowup`	메소드	770	`(self, rowId, reasonCode, question, answeredAt) -> str`	꼬리질문 저장 → 같은 질문 줄에서 TTS. 늦으면 자막만으로 낸다 (INT-01).	—

app/interview/graph.py

역할: 면접·꼬리질문 순서도(LangGraph) — 면접 진행 그래프 (ADR-07, TRD 7-5, AI 아키텍처 6.1·6.2).

가져다 쓰는 모듈 · 우리 `domain.question_text` `services.posting_draft` · 표준 `logging` `re` `typing` · 외부 `langgraph`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`Runner`	클래스	37	`(Protocol)`	LangGraph 노드가 부를 실제 동작의 약속(인터페이스). LiveSession 이 구현한다	i.graph:buildFollowupGraph · i.graph:buildInterviewGraph
`Runner.prepareQuestions`	메소드	40	`(self) -> None`	약속: 질문 준비	—
`Runner.locate`	메소드	41	`(self) -> dict`	약속: 지금 낼 질문 찾기	—
`Runner.advance`	메소드	42	`(self, seq) -> None`	약속: 다음 질문으로	—
`Runner.beginQuestion`	메소드	43	`(self, rowId) -> None`	약속: 질문 보내고 답변 받기 시작	—
`Runner.finishAnswer`	메소드	44	`(self, rowId, endReason) -> dict`	약속: 답변 마무리(STT·저장)	—
`Runner.followupPreflight`	메소드	45	`(self, rowId, answeredAt) -> str | None`	약속: 꼬리질문을 물을 수 있는 상황인지 먼저 확인	—
`Runner.followupDecide`	메소드	46	`(self, rowId, answeredAt) -> dict`	약속: AI 에게 꼬리질문 필요 여부 묻기	—
`Runner.createFollowup`	메소드	47	`(self, rowId, reasonCode, question, answeredAt) -> str`	약속: 꼬리질문 행 만들기	—

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`Runner.finishInterview`	메소드	48	`(self) -> None`	약속: 면접 끝내기	—
`InterviewState`	클래스	51	`(TypedDict)`	면접 그래프가 들고 다니는 상태(질문 번호, 질문 ID, 종류, 끝난 이유)	i.graph:advance · i.graph:afterFollowup · i.graph:afterLocate · i.graph:afterTranscribe · 외 8곳
`FollowupState`	클래스	61	`(TypedDict)`	꼬리질문 그래프의 상태	i.graph:buildFollowupGraph · i.graph:create · i.graph:decide · i.graph:preflight · 외 2곳
`validateFollowup`	함수	71	`(reasonCode, question) -> str | None`	꼬리질문 검사: 길이·한국어·일본어/한자·차별/사적 주제, 그리고 본질문과 같은 글 검사(question_text). 문제 코드, 맞으면 None	i.graph:validate · 🧪 테스트 2개 파일
`buildFollowupGraph`	함수	87	`(runner)`	꼬리질문 순서도: 사전 확인 → AI 판단 → 검사 → 만들기 (중간에 skip 가능)	i.graph:buildInterviewGraph
`buildFollowupGraph.preflight`	안쪽 함수	88	`(state) -> dict`	노드: 물어볼 수 있는지 확인	—
`buildFollowupGraph.decide`	안쪽 함수	91	`(state) -> dict`	노드: AI 판단	—
`buildFollowupGraph.validate`	안쪽 함수	96	`(state) -> dict`	노드: AI 가 낸 꼬리질문 검사	—
`buildFollowupGraph.create`	안쪽 함수	99	`(state) -> dict`	노드: 꼬리질문 저장	—
`buildFollowupGraph.skip`	안쪽 함수	104	`(state) -> dict`	사유만 남긴다 (답변·질문 글은 로그에 넣지 않음)	—
`buildFollowupGraph.nextOr`	안쪽 함수	109	`(target)`	skipCause 가 있으면 skip, 없으면 다음 노드로	—
`buildInterviewGraph`	함수	127	`(runner, checkpointer)`	면접 순서도: 준비 → 찾기 → 질문 → 답변 → 꼬리질문 → 다음 … → 끝	i.engine:runInterview · 🧪 테스트 1개 파일
`buildInterviewGraph.prepare`	안쪽 함수	128	`(_state) -> dict`	노드: 질문 준비	—
`buildInterviewGraph.locate`	안쪽 함수	132	`(_state) -> dict`	노드: 지금 질문 찾기	—
`buildInterviewGraph.ask`	안쪽 함수	135	`(state) -> dict`	노드: 질문 보내기	—
`buildInterviewGraph.wait`	안쪽 함수	139	`(state) -> dict`	Python 3.11 이상이 필요하다: 3.10 에서는 interrupt 가 실행 문맥을 찾지 못한다 (LangGraph 제약, 이미지는 3.11)	—
`buildInterviewGraph.transcribe`	안쪽 함수	145	`(state) -> dict`	노드: 답변 받기·STT	—
`buildInterviewGraph.queueFollowup`	안쪽 함수	149	`(state) -> dict`	노드: 만든 꼬리질문을 다음 질문으로 세운다	—
`buildInterviewGraph.advance`	안쪽 함수	152	`(state) -> dict`	노드: 다음 번호로	—
`buildInterviewGraph.finish`	안쪽 함수	156	`(_state) -> dict`	노드: 면접 끝	—

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`buildInterviewGraph.afterLocate`	안쪽 함수	160	`(state) -> str`	갈림길: 10번 넘으면 finish, 질문 있으면 ask, 없으면 advance	—
`buildInterviewGraph.afterTranscribe`	안쪽 함수	165	`(state) -> str`	갈림길: 기본 질문이면 꼬리질문 판단, 꼬리질문이면 다음	—
`buildInterviewGraph.afterFollowup`	안쪽 함수	168	`(state) -> str`	갈림길: 꼬리질문이 만들어졌으면 그것을, 아니면 다음	—
`graphConfig`	함수	194	`(applicationId) -> dict`	그래프 실행 설정(지원 ID = 체크포인트 이름, 반복 상한)	i.engine:runInterview · 🧪 테스트 1개 파일

app/interview/lifecycle.py

역할: 응시 기록·이어가기 횟수·코딩 타이머 — 응시 상태를 바꾸는 공통 동작. 면접 세션(WebSocket)·HTTP API·주기 작업이 같은 함수를 쓴다.

가져다 쓰는 모듈 | 🏠 우리 `domain` `models` `services` `settings_store` `timeutil` | 🐍 표준 `datetime` `io` `wave` | 📦 외부 `sqlalchemy`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`logEvent`	함수	19	`(db, app, type_, detail, judged, sessionId, seq) -> None`	응시 기록(session_events)에 사건 한 줄을 남긴다	i.engine:judgeConnectionLoss · i.engine:onAway · i.engine:onDeviceError · i.engine:onPermissionRevoked · 외 3곳 · 🧪 테스트 1개 파일
`recentEventTypes`	함수	25	`(db, app, lookbackSec) -> set[str]`	최근 N초 안에 남은 사건 종류들	i.engine:judgeConnectionLoss · i.engine:judgeDeviceEnded · i.engine:onDeviceError
`codingStarted`	함수	31	`(db, app) -> bool`	안내 화면을 지나 시간을 쓰기 시작했는가. 멈춘 동안에는 coding_started_at 이 비어도 남은 시간이 줄어 있다.	i.lifecycle:resumeCodingTimer · r.interview:codingView · 🧪 테스트 1개 파일
`resumeCodingTimer`	함수	36	`(db, app) -> bool`	이미 시작한 코딩테스트로 다시 연결되면 안내 화면 없이 남은 시간부터 잰다 (COD-04). 시작했는지를 돌려준다.	i.engine:sendCodingPhase · 🧪 테스트 1개 파일
`pauseCodingTimer`	함수	45	`(app) -> None`	코딩 타이머를 멈추고 남은 초를 저장한다	i.engine:onAway · i.lifecycle:finish · i.lifecycle:markInterrupted
`codingRemaining`	함수	52	`(app) -> int | None`	코딩 남은 시간(초) 계산	i.engine:codingTimeOver · i.engine:onAway · i.engine:onWarnAck · i.engine:sendCodingPhase · 외 3곳 · 🧪 테스트 1개 파일
`applicantInterruptions`	함수	64	`(db, app) -> int`	지원자 쪽 원인 장애 횟수(서버 원인 제외)	i.lifecycle:resumesLeft

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`resumesLeft`	함수	69	`(db, app) -> int`	남은 이어가기 횟수 = 상한 − 지원자 쪽 장애 횟수	i.engine:judgeConnectionLoss · i.engine:pauseForFault · i.lifecycle:markInterrupted · r.interview:interviewState · 🧪 테스트 1개 파일
`markInterrupted`	함수	73	`(db, app, reason) -> bool`	장애 중단: 시간을 멈추고 자리는 disconnect_seat_hold_min 동안 맡아 둔다 (TRD 7-2). 지원자 쪽 끊김으로 이어가기를 resume_limit 번 다 썼으면 응시를 종료한다 (CONFLICTS D1). 중단이면 True, 종료면 False.	i.engine:judgeConnectionLoss · i.engine:pauseForFault · jobs:sweepOrphans · main:recoverAfterRestart · 🧪 테스트 1개 파일
`finish`	함수	90	`(db, app, target, detail) -> None`	응시 완료·종료·취소·마감 종료. 자리 반환까지 한다.	i.engine:endWith · i.engine:judgeConnectionLoss · i.lifecycle:markInterrupted · s.coding:finishCoding
`queueEvaluation`	함수	103	`(applicationId) -> None`	채점 작업을 커밋 뒤에 넣는다 (2단계 Celery). 빠지면 worker 의 5분 점검이 다시 넣는다.	i.lifecycle:finish
`enterCoding`	함수	110	`(db, app) -> None`	AI 면접이 끝나 코딩테스트로: 자리를 돌려주고(코딩은 GPU 를 안 씀) 문제를 배정한다 (TRD 5-3).	i.engine:finishInterview · 🧪 테스트 2개 파일
`pcmToWav`	함수	125	`(pcm, sampleRate) -> bytes`	녹음 원본(PCM)을 wav 파일 바이트로 바꾼다	i.engine:saveAnswer

app/interview/registry.py

역할: 살아 있는 면접 세션 명부 — 지금 연결된 면접 세션 목록. 마감·관리자 조작이 진행 중 면접을 끝낼 때 쓴다 (1단계: api 프로세스 1개 기준).

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`claimSession`	함수	8	`(sessionId) -> bool`	세션 ID 로 연결은 한 번만 받는다(중복 접속 막기)	r.interview:interviewSocket
`releaseSession`	함수	15	`(sessionId) -> None`	세션이 끝나 ended_at 이 저장된 뒤 부른다. 그 뒤로는 ended_at 이 다시 연결을 막으므로 집합에 남길 필요가 없다 (E15).	i.engine:shutdown
`get`	함수	20	`(applicationId)`	지원 건의 살아 있는 LiveSession 을 찾는다	i.engine:serve · jobs:sweepOrphans · r.interview:codingStart · r.interview:codingSubmit · 외 1곳
`register`	함수	24	`(applicationId, session) -> None`	살아 있는 세션으로 등록	i.engine:serve
`unregister`	함수	28	`(applicationId, session) -> None`	등록 해제(같은 객체일 때만)	i.engine:shutdown

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`terminateMany`	async 함수	33	`(applicationIds, reason, detail) -> None`	여러 지원 건의 세션을 밖에서 끝낸다 (공고 마감 등)	jobs:closeExpiredPostings · r.admin:close · s.withdrawal:withdrawApplication

A-7. worker · jobs — 뒷주방과 시간표

app/jobs.py

역할: api 안에서 1분마다 도는 정리 작업 — 1분마다 도는 정리 작업 (1단계: api 안 asyncio 작업, 2단계에서 Celery beat 로 옮긴다).

가져다 쓰는 모듈 | 우리 `db` `domain` `interview` `models` `services.postings` `services` `settings_store` `timeutil` | 표준 `asyncio` `datetime` `logging`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`releaseHeldSeats`	함수	26	`(db) -> None`	장애 중단 뒤 N분 지난 건의 면접 자리를 반환한다	jobs:runForever
`sweepOrphans`	함수	33	`(db) -> None`	진행 중인데 붙어 있는 면접 세션이 없는 지원 (재시작·연결 한 번도 안 됨) → 장애 중단으로 돌린다.	jobs:runForever · 테스트 1개 파일
`regradePending`	함수	49	`() -> None`	끝난 응시인데 채점되지 않은(마감·종료로 끝났거나 인프라 오류) 코딩 답안을 다시 채점한다.	jobs:runForever
`closeExpiredPostings`	async 함수	63	`(db) -> None`	마감 시각이 지난 게시 공고를 자동 마감하고 진행 중 세션을 끝낸다	jobs:runForever
`runForever`	async 함수	70	`() -> None`	api 프로세스 안에서 1분마다 정리 작업을 도는 반복문	main:lifespan

app/worker.py

역할: Celery 작업자(채점·메일·시간표) — Celery worker · beat (TRD 3장 Celery 5 + Redis, AI 아키텍처 v1.4 7장).

가져다 쓰는 모듈 | 우리 `config` `domain` `models` `services` `timeutil` | 표준 `asyncio` `datetime` `logging` `uuid` | 외부 `celery` `redis`

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`quietTaskResults`	함수	78	`(**_kwargs) -> None`	'작업 성공: <결과>' 줄에 지원자 코드 실행 결과가 찍히지 않게 한다 (TRD 11장: 로그에 답변 원문 금지).	—
`enqueueEvaluation`	함수	83	`(applicationId, attempt, countdown) -> None`	DB 커밋 뒤에 부른다. 브로커가 안 되면 기록만 남기고 5분 점검이 다시 넣는다.	i.lifecycle:queueEvaluation · s.postings:closePosting · worker:requestReprocess · worker:reschedule · 외 1곳

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`queueMail`	함수	92	`(notificationId, countdown) -> None`	메일 보내기 작업을 큐에 넣는다 (실패해도 경고만)	worker:notifyMany · worker:sendMailTask · worker:sweepTask
`queueResetMail`	함수	99	`(email, link, attempt) -> None`	비밀번호 재설정 메일. 링크(토큰)는 로그에 남기지 않는다.	r.auth:sendResetLink · worker:sendResetMailTask
`notifyMany`	함수	109	`(rows) -> None`	[(user_id, kind, title, link, application_id)] 알림을 만들고 커밋 뒤 메일을 넣는다.	worker:purgeNoticeTask · worker:remindersTask · worker:sweepTask · worker:toNeedsReview
`lockKey`	함수	123	`(applicationId) -> str`	채점 잠금 Redis 키 이름	worker:evaluateTask · worker:findUnscored · worker:releaseLock
`sinceKey`	함수	127	`(applicationId) -> str`	채점 시작 시각 Redis 키 이름	worker:evaluationStartedAt · worker:requestReprocess
`queuedKey`	함수	131	`(applicationId) -> str`	다시 넣어 둔 작업이 기다리는 중 — 5분 점검이 같은 건을 또 넣지 않게.	worker:evaluateTask · worker:findUnscored · worker:requestReprocess · worker:reschedule
`releaseLock`	함수	136	`(applicationId, token) -> None`	내가 건 잠금만 푼다 (다른 작업의 잠금을 지우지 않게).	worker:evaluateTask
`evaluationStartedAt`	함수	143	`(applicationId, completedAt) -> object`	30분 한도를 재는 기준 시각: 다시 돌리기를 눌렀으면 그때, 아니면 응시 끝난 때.	worker:evaluateTask
`toNeedsReview`	함수	151	`(applicationId, reason) -> None`	평가 지연으로 바꾸고 관리자에게 알림	worker:evaluateTask · 🧪 테스트 2개 파일
`evaluateTask`	⏱ 작업	165	`(self, applicationId, attempt) -> str`	Celery 작업	🧪 테스트 1개 파일
`reschedule`	함수	207	`(applicationId, attempt, delaySec) -> None`	채점을 몇 초 뒤 다시 예약	worker:evaluateTask · 🧪 테스트 1개 파일
`sendMailTask`	⏱ 작업	213	`(notificationId) -> str`	Celery 작업	—
`sendResetMailTask`	⏱ 작업	222	`(email, link, attempt) -> str`	Celery 작업	—
`findUnscored`	함수	235	`(db) -> list[str]`	응시는 끝났는데 결과가 없고 작업도 돌고 있지 않은 건 (AC-17).	worker:sweepTask · 🧪 테스트 1개 파일
`releaseDue`	함수	243	`(db) -> list[str]`	공개 시각이 지난 확정 결과를 공개로 바꾸고, 같은 트랜잭션에서 지원자 알림을 만든다 (RPT-02). 돌려주는 값은 메일을 보낼 알림 id.	worker:sweepTask · 🧪 테스트 1개 파일
`noteSweep`	함수	257	`() -> bool`	이번 점검 시각을 남긴다. 서비스가 멈췄다 돌아온 지 24시간 안이면 True.	worker:settleReviews
`aiReady`	함수	267	`() -> bool`	LLM 두 서버(4B·8B)가 모두 살아 있는지	worker:settleReviews
`settleReviews`	함수	271	`(db) -> tuple[list[str], list[str]]`	평가 지연 건을 재채점하거나 닫는다. (닫은 건, 다시 넣을 건). 재채점 건의 사유·시각은 남겨 기한이 늘어나지 않게 한다.	worker:sweepTask
`sweepTask`	⏱ 작업	304	`() -> dict`	Celery 작업	🧪 테스트 1개 파일

이름	종류	줄	인자 → 반환	하는 일	부르는 곳
`reminderRows`	함수	324	`(db) -> list[tuple]`	마감 임박 공고 안내 대상	worker:remindersTask · 🧪 테스트 1개 파일
`remindersTask`	⏱ 작업	356	`() -> int`	Celery 작업	—
`gradeQueuedKey`	함수	366	`(applicationId) -> str`	문항 채점 대기 Redis 키 이름	worker:enqueueGrade
`enqueueGrade`	함수	370	`(applicationId) -> None`	코딩 숨겨진 테스트 채점. 5분 안에 같은 건을 또 넣지 않는다 (인프라 오류면 5분 뒤 점검이 다시 넣음).	s.coding:scheduleGrade · 🧪 테스트 1개 파일
`sandboxRunTask`	⏱ 작업	381	`(code, cases) -> dict`	Celery 작업	—
`gradeCodingTask`	⏱ 작업	391	`(applicationId) -> str`	Celery 작업	—
`purgeTask`	⏱ 작업	406	`() -> int`	Celery 작업	—
`purgeRecordingsTask`	⏱ 작업	419	`() -> int`	Celery 작업	—
`purgeNoticeTask`	⏱ 작업	431	`() -> int`	Celery 작업	—
`requestReprocess`	함수	442	`(applicationId) -> None`	관리자 '다시 돌리기': 30분 한도를 지금부터 다시 잰다.	r.admin:reprocess · worker:sweepTask

부록 B. 프론트엔드 함수 전체 표 (`huge/frontend`)

화면 주소와 파일은 1:1 입니다: `app/mypage/page.tsx` = `/mypage`. `[applicationId]` 같은 대괄호 폴더는 주소의 바뀌는 부분입니다. 각 `page.tsx` 의 `...Page` 함수는 Next.js 가 부르는 **입구**이고, 실제 내용은 같은 파일의 다른 컴포넌트에 있습니다.

B-0. 폴더 지도

파일	줄	화면 주소 / 역할
`app/admin/applicants/[applicationId]/page.tsx`	567	/admin/applicants/[applicationId]
`app/admin/applicants/page.tsx`	166	/admin/applicants
`app/admin/compare/page.tsx`	98	/admin/compare
`app/admin/dashboard/page.tsx`	204	/admin/dashboard
`app/admin/next-stage/page.tsx`	93	/admin/next-stage
`app/admin/postings/[postingId]/edit/page.tsx`	32	/admin/postings/[postingId]/edit
`app/admin/postings/ai/page.tsx`	251	/admin/postings/ai
`app/admin/postings/new/page.tsx`	35	/admin/postings/new
`app/admin/postings/page.tsx`	155	/admin/postings
`app/admin/questions/page.tsx`	116	/admin/questions

파일	줄	화면 주소 / 역할
app/admin/settings/page.tsx	101	/admin/settings
app/apply/[applicationId]/consent/page.tsx	82	/apply/[applicationId]/consent
app/apply/[applicationId]/precheck/page.tsx	39	/apply/[applicationId]/precheck
app/apply/[applicationId]/resume/confirm/page.tsx	125	/apply/[applicationId]/resume/confirm
app/apply/[applicationId]/resume/page.tsx	231	/apply/[applicationId]/resume
app/apply/[applicationId]/rules/page.tsx	119	/apply/[applicationId]/rules
app/exam/[applicationId]/page.tsx	601	/exam/[applicationId]
app/exam/[applicationId]/resume/page.tsx	124	/exam/[applicationId]/resume
app/jobs/[postingId]/page.tsx	133	/jobs/[postingId]
app/jobs/page.tsx	154	/jobs
app/layout.tsx	24	/
app/login/page.tsx	95	/login
app/mypage/page.tsx	356	/mypage
app/mypage/results/[applicationId]/page.tsx	81	/mypage/results/[applicationId]
app/not-found.tsx	17	/
app/page.tsx	5	/
app/password/change/page.tsx	65	/password/change
app/password/forgot/page.tsx	51	/password/forgot
app/password/reset/page.tsx	73	/password/reset
app/signup/page.tsx	113	/signup
components/AdminShell.tsx	63	
components/ApplyFlow.tsx	161	
components/Brand.tsx	31	eslint-disable @next/next/no-img-element -- 브랜드 이미지는 public/
components/exam/Coding.tsx	279	
components/exam/Overlays.tsx	146	
components/NotificationBell.tsx	120	
components/PasswordChecklist.tsx	23	비밀번호 규칙 4줄 체크리스트 (PRD 5-1, 화면정의서 §5.2). 서버도 같은 규칙으로 검사한다
components/PostingBody.tsx	87	
components/PostingEditor.tsx	392	

파일	줄	화면 주소 / 역할
components/Precheck.tsx	216	
components/Providers.tsx	95	
components/ResumeForm.tsx	144	
components/Shell.tsx	125	
components/Ui.tsx	134	
lib/admin.ts	19	관리자 공통 형식
lib/api.ts	47	서버 요청 공통
lib/exam.ts	28	면접 세션 정보 보관과 '나가요' 쪽지(sendBeacon)
lib/format.ts	59	시각·시간 표시
lib/media.ts	107	카메라·마이크·스피커 도우미
lib/pcm.ts	59	답변 녹음기
lib/recording.ts	92	카메라 녹화 조각 올리기
lib/resumeDraft.ts	7	이력서 임시값
lib/status.ts	82	상태 이름·색
proxy.ts	14	127.0.0.1 로 주소 맞추기

B-1. app — 화면

app/admin/applicants/[applicationId]/page.tsx · /admin/applicants/[applicationId]

가져다 쓰는 모듈

- 우리 components/AdminShell components/Providers components/ResumeForm components/Ui lib/api lib/admin lib/format lib/status
- 외부 next/link next/navigation react

이름	종류	줄	하는 일	쓰는 곳
text	함수	55	값이 글/숫자면 글로, 아니면 빈 글	이 파일 안
list	함수	59	값이 배열이면 객체 항목만 남긴다	이 파일 안
ResumeEntry	컴포넌트	65	이력서 항목 한 줄: 제목(학교·회사 등) · 기간 · 설명. 비어 있는 칸은 빼고 보여 준다	이 파일 안
span	함수	79	기간을 '시작 ~ 끝' 글로	이 파일 안
ResumeSection	컴포넌트	85	이력서 한 묶음(학력·경력…) 제목과 내용. 비면 숨김	이 파일 안
ResumeView	컴포넌트	97	지원자가 제출한 이력서를 지원자 화면과 같은 이름(학력·경력…)으로 보여 준다. 글자로만 보여 준다 (HTML 해석 안 함)	이 파일 안
ScoreBar	컴포넌트	143	0~100 점수 막대	이 파일 안

이름	종류	줄	하는 일	쓰는 곳
`ManualDecision`	컴포넌트	153	자동 채점을 끝내지 못한 건만 관리자가 정한다 (CONFLICTS U8). 자동 채점 결과는 여기서 바꿀 수 없다	이 파일 안
`DecisionCard`	컴포넌트	176	판정 카드: 결과·총점·기준·이유. 자동 채점 결과를 바꾸는 버튼은 없다 (EVL-05, AC-14). 확인 필요면 [다시 돌리기]와 직접 결정	이 파일 안
`Report`	컴포넌트	226	리포트 탭: 항목별 점수·근거·결과 이유	이 파일 안
`Recordings`	컴포넌트	251	카메라 녹화 다시 보기 (응시 끝 + 3일). 부정행위 확인용이며 평가에는 쓰지 않는다	이 파일 안
`Memos`	컴포넌트	277	내부 메모 (ADM-05). 지원자에게는 보이지 않는다	이 파일 안
`ApplicantDetail`	컴포넌트	359	S-R06 지원자 상세. 판정·점수를 바꾸는 버튼은 두지 않는다 (EVL-05, AC-14)	이 파일 안
`playFrom`	함수	402	하이라이트: 그 구간부터 재생 (ADM-04)	이 파일 안
`ApplicantDetailPage`	컴포넌트 · 내보냄	564	화면 입구 (Next.js 가 `/admin/applicants/[applicationId]` 에서 부름)	이 파일 안

app/admin/applicants/page.tsx · /admin/applicants

가져다 쓰는 모듈

우리 `components/AdminShell` `components/Ui` `lib/api` `lib/admin` `lib/format` `lib/status`

외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`Applicants`	컴포넌트	25	지원자 목록: 공고·상태·결과·이름으로 거르기, 2~3명 골라 비교	이 파일 안
`togglePick`	함수	42	비교는 같은 공고 안에서 2~3명 (S-R07)	이 파일 안
`ApplicantsPage`	컴포넌트 · 내보냄	163	화면 입구 (Next.js 가 `/admin/applicants` 에서 부름)	이 파일 안

app/admin/compare/page.tsx · /admin/compare

가져다 쓰는 모듈

우리 `components/AdminShell` `components/Ui` `lib/api` `lib/status`

외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`Radar`	컴포넌트	19	항목 점수 레이더 그림(SVG)	이 파일 안
`point`	함수	23	레이더 꼭짓점 좌표 계산	이 파일 안
`Compare`	컴포넌트	43	고른 지원자들을 나란히 비교	이 파일 안
`ComparePage`	컴포넌트 · 내보냄	95	화면 입구 (Next.js 가 `/admin/compare` 에서 부름)	이 파일 안

app/admin/dashboard/page.tsx · /admin/dashboard

가져다 쓰는 모듈

- 우리 `components/AdminShell` `components/Providers` `components/Ui` `lib/api` `lib/format` `lib/status`
- 외부 `next/link` `react`

이름	종류	줄	하는 일	쓰는 곳
`Bars`	컴포넌트	27	막대그래프 + 같은 값의 표(접근성)	이 파일 안
`Dashboard`	컴포넌트	48	통계와 운영 현황(평가 지연·메일 실패·로그인 차단)	이 파일 안
`DashboardPage`	컴포넌트 · 내보냄	201	화면 입구 (Next.js 가 `/admin/dashboard` 에서 부름)	이 파일 안

app/admin/next-stage/page.tsx · /admin/next-stage

가져다 쓰는 모듈

- 우리 `components/AdminShell` `components/Ui` `lib/api` `lib/admin` `lib/format`
- 외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`NextStage`	컴포넌트	18	공고별 1차 합격자 연락처	이 파일 안
`NextStagePage`	컴포넌트 · 내보냄	90	화면 입구 (Next.js 가 `/admin/next-stage` 에서 부름)	이 파일 안

app/admin/postings/[postingId]/edit/page.tsx · /admin/postings/[postingId]/edit

가져다 쓰는 모듈

- 우리 `components/AdminShell` `components/PostingEditor` `components/Ui` `lib/api` `lib/admin`
- 외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`EditPostingPage`	컴포넌트 · 내보냄	12	화면 입구 (Next.js 가 `/admin/postings/[postingId]/edit` 에서 부름)	이 파일 안

app/admin/postings/ai/page.tsx · /admin/postings/ai

가져다 쓰는 모듈

- 우리 `components/AdminShell` `components/Brand` `components/Providers` `components/Ui` `lib/api` `lib/format` `lib/status`
- 외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`Chips`	컴포넌트	31	고르기 칩(직접 추가 가능)	이 파일 안
`toggle`	함수	35	칩 하나 켜기/끄기	이 파일 안
`add`	함수	36	직접 쓴 칩 추가(50자까지)	이 파일 안
`AiPostingPage`	컴포넌트 · 내보냄	59	S-R02 AI 공고 자동 만들기. 회사명은 묻지 않는다 (단일 기업 서비스)	이 파일 안

app/admin/postings/new/page.tsx · /admin/postings/new

가져다 쓰는 모듈

- 우리 `components/AdminShell` `components/Brand` `components/PostingEditor`
- 외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`NewPosting`	컴포넌트	12	새 공고: 작성 방식 선택 (S-R01 모달 내용을 한 화면으로) → AI 초안(S-R02) 또는 직접 작성(S-R03)	이 파일 안
`NewPostingPage`	컴포넌트 · 내보냄	32	화면 입구 (Next.js 가 `/admin/postings/new` 에서 부름)	이 파일 안

app/admin/postings/page.tsx · /admin/postings

가져다 쓰는 모듈

- 우리 `components/AdminShell` `components/Providers` `components/Ui` `lib/api` `lib/admin` `lib/format` `lib/status`
- 외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`AdminPostingsPage`	컴포넌트 · 내보냄	16	화면 입구 (Next.js 가 `/admin/postings` 에서 부름)	이 파일 안

app/admin/questions/page.tsx · /admin/questions

가져다 쓰는 모듈

- 우리 `components/AdminShell` `components/Ui` `lib/api`
- 외부 `react`

이름	종류	줄	하는 일	쓰는 곳
`Questions`	컴포넌트	23	질문 관리: 검토 대기/사용 중/제외 탭, 종류 거르기	이 파일 안
`QuestionsPage`	컴포넌트 · 내보냄	113	화면 입구 (Next.js 가 `/admin/questions` 에서 부름)	이 파일 안

app/admin/settings/page.tsx · /admin/settings

가져다 쓰는 모듈

- 우리 `components/AdminShell` `components/Providers` `components/Ui` `lib/api` `lib/format`
- 외부 `react`

이름	종류	줄	하는 일	쓰는 곳
`SettingsBody`	컴포넌트	15	운영 설정(동시 면접 상한) 바꾸기와 변경 이력	이 파일 안
`SettingsPage`	컴포넌트 · 내보냄	98	화면 입구 (Next.js 가 `/admin/settings` 에서 부름)	이 파일 안

app/apply/[applicationId]/consent/page.tsx · /apply/[applicationId]/consent

가져다 쓰는 모듈

- 우리 `components/ApplyFlow` `components/Providers` `components/Ui` `lib/api`
- 외부 `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`Consent`	컴포넌트	26	동의 5가지 체크와 제출	이 파일 안
`ConsentPage`	컴포넌트 · 내보냄	79	화면 입구 (Next.js 가 `/apply/[applicationId]/consent` 에서 부름)	이 파일 안

app/apply/[applicationId]/precheck/page.tsx · /apply/[applicationId]/precheck

가져다 쓰는 모듈

- 우리 `components/ApplyFlow` `components/Precheck` `components/Providers` `components/Ui` `lib/format`
- 외부 `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`PrecheckStep`	컴포넌트	11	환경 점검 단계(Precheck 를 감싸 통과하면 다음으로)	이 파일 안
`PrecheckPage`	컴포넌트 · 내보냄	36	화면 입구 (Next.js 가 `/apply/[applicationId]/precheck` 에서 부름)	이 파일 안

app/apply/[applicationId]/resume/confirm/page.tsx · /apply/[applicationId]/resume/confirm

가져다 쓰는 모듈

- 우리 `components/ApplyFlow` `components/Providers` `components/ResumeForm` `components/Ui` `lib/api` `lib/resumeDraft`
- 외부 `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`Confirm`	컴포넌트	12	추출한 이력서를 확인·수정해 제출	이 파일 안
`clearLowConf`	함수	42	지원자가 고치거나 [확인]을 누른 칸은 확인 필요 표시를 지운다 (화면정의서 S-A05)	이 파일 안
`ConfirmPage`	컴포넌트 · 내보냄	122	화면 입구 (Next.js 가 `/apply/[applicationId]/resume/confirm` 에서 부름)	이 파일 안

app/apply/[applicationId]/resume/page.tsx · /apply/[applicationId]/resume

가져다 쓰는 모듈

- 우리 `components/ApplyFlow` `components/Ui` `lib/api` `lib/format` `lib/resumeDraft`
- 외부 `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`ResumeMethod`	컴포넌트	38	이력서 방법 고르기(파일·저장본·직접) 와 업로드 진행	이 파일 안
`ResumePage`	컴포넌트 · 내보냄	228	화면 입구 (Next.js 가 `/apply/[applicationId]/resume` 에서 부름)	이 파일 안

app/apply/[applicationId]/rules/page.tsx · /apply/[applicationId]/rules

가져다 쓰는 모듈

- 우리 `components/ApplyFlow` `components/Providers` `components/Ui` `lib/api` `lib/exam` `lib/format`
- 외부 `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`Rules`	컴포넌트	14	응시 규칙 확인 후 면접 시작(자리 없으면 안내)	이 파일 안
`RulesPage`	컴포넌트 · 내보냄	116	화면 입구 (Next.js 가 `/apply/[applicationId]/rules` 에서 부름)	이 파일 안

app/exam/[applicationId]/page.tsx · /exam/[applicationId]

가져다 쓰는 모듈

- 우리 `components/Brand` `components/exam/Coding` `components/exam/Overlays` `components/Providers` `components/Shell` `components/Ui` `lib/api` `lib/exam` `lib/format` `lib/media` `lib/pcm` `lib/recording`
- 외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`Exam`	컴포넌트	39	면접 화면 본체: WebSocket·녹음·녹화·이탈 감지·오버레이	이 파일 안
`ExamPage`	컴포넌트 · 내보냄	597	화면 입구 (Next.js 가 `/exam/[applicationId]` 에서 부름)	이 파일 안

app/exam/[applicationId]/resume/page.tsx · /exam/[applicationId]/resume

가져다 쓰는 모듈

- 우리 `components/Precheck` `components/Providers` `components/Shell` `components/Ui` `lib/api` `lib/exam` `lib/format`
- 외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`reasonText`	함수	28	모르는 코드는 영문 그대로 보이지 않게 한다 (E13)	이 파일 안
`Resume`	컴포넌트	33	S-A15 면접 이어가기: 서버 판정 조회 → 재점검 → 이어서 시작	이 파일 안
`ResumePage`	컴포넌트 · 내보냄	114	화면 입구 (Next.js 가 `/exam/[applicationId]/resume` 에서 부름)	이 파일 안

app/jobs/[postingId]/page.tsx · /jobs/[postingId]

가져다 쓰는 모듈

- 우리 `components/PostingBody` `components/Providers` `components/Shell` `components/Ui` `lib/api` `lib/format` `lib/status`
- 외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`JobDetail`	컴포넌트	16	공고 상세와 [지원하기]	이 파일 안
`JobDetailPage`	컴포넌트 · 내보냄	130	화면 입구 (Next.js 가 `/jobs/[postingId]` 에서 부름)	이 파일 안

app/jobs/page.tsx · /jobs

가져다 쓰는 모듈

우리 `components/Brand` `components/Shell` `components/Ui` `lib/api` `lib/format` `lib/status`

외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`Skeleton`	컴포넌트	20	불러오는 동안 보이는 빈 카드	이 파일 안
`JobList`	컴포넌트	24	공고 목록(검색·직무 거르기)	이 파일 안
`JobsPage`	컴포넌트 · 내보냄	151	화면 입구 (Next.js 가 `/jobs` 에서 부름)	이 파일 안

app/layout.tsx · /

가져다 쓰는 모듈

우리 `./globals.css` `components/Providers`

외부 `pretendard/dist/web/variable/pretendardvariable-dynamic-subset.css` `next` `react`

이름	종류	줄	하는 일	쓰는 곳
`RootLayout`	컴포넌트 · 내보냄	15	화면 입구 (Next.js 가 `/` 에서 부름)	이 파일 안

app/login/page.tsx · /login

가져다 쓰는 모듈

우리 `components/Brand` `components/Providers` `components/Ui` `lib/api`

외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`LoginForm`	컴포넌트	13	로그인 폼(성공하면 next 주소로)	이 파일 안
`LoginPage`	컴포넌트 · 내보냄	92	화면 입구 (Next.js 가 `/login` 에서 부름)	이 파일 안

app/mypage/page.tsx · /mypage

가져다 쓰는 모듈

우리 `components/ApplyFlow` `components/Brand` `components/Providers` `components/ResumeForm` `components/Shell` `components/Ui` `lib/api` `lib/format` `lib/status`

외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`CardAction`	컴포넌트	42	지원 카드의 다음 행동 버튼(이어 쓰기·점검·이어가기…)	이 파일 안
`Applications`	컴포넌트	55	지원 내역 탭	이 파일 안
`Results`	컴포넌트	75	결과 탭(공개 예정일·지연 안내)	이 파일 안
`SavedResume`	컴포넌트	101	저장 이력서 탭	이 파일 안
`Consents`	컴포넌트	169	지원별 동의 철회 (APP-03). 철회하면 진행 중 응시가 취소되고 그 지원의 데이터가 바로 지워진다	이 파일 안

이름	종류	줄	하는 일	쓰는 곳
`Withdraw`	컴포넌트	215	회원 탈퇴: '탈퇴'를 입력해야 버튼이 켜진다. 진행 중 응시 취소, 응시 데이터·계정 즉시 삭제	이 파일 안
`Account`	컴포넌트	263	회원 정보 탭(이름·전화·비밀번호·탈퇴)	이 파일 안
`MyPage`	컴포넌트	302	마이페이지 탭 묶음	이 파일 안
`MyPageRoute`	컴포넌트 · 내보냄	353	화면 입구 (Next.js 가 `/mypage` 에서 부름)	이 파일 안

app/mypage/results/[applicationId]/page.tsx · /mypage/results/[applicationId]

가져다 쓰는 모듈

우리 `components/Brand` `components/Shell` `components/Ui` `lib/api` `lib/format` `lib/status`

외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`ResultDetail`	컴포넌트	29	S-A14 결과 상세. 점수·근거는 보여 주지 않는다 (RPT-03, AC-16)	이 파일 안
`ResultPage`	컴포넌트 · 내보냄	78	화면 입구 (Next.js 가 `/mypage/results/[applicationId]` 에서 부름)	이 파일 안

app/not-found.tsx · /

가져다 쓰는 모듈

우리 `components/Brand` `components/Shell`

외부 `next/link`

이름	종류	줄	하는 일	쓰는 곳
`NotFound`	컴포넌트 · 내보냄	5	화면 입구 (Next.js 가 `/` 에서 부름)	이 파일 안

app/page.tsx · /

가져다 쓰는 모듈

외부 `next/navigation`

이름	종류	줄	하는 일	쓰는 곳
`Home`	컴포넌트 · 내보냄	2	화면 입구 (Next.js 가 `/` 에서 부름)	이 파일 안

app/password/change/page.tsx · /password/change

가져다 쓰는 모듈

우리 `components/Brand` `components/PasswordChecklist` `components/Providers` `components/Ui` `lib/api`

외부 `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`PasswordChangePage`	컴포넌트 · 내보냄	13	S-C04 관리자 초기 비밀번호 변경. 건너뛸 수 없다	이 파일 안

app/password/forgot/page.tsx · /password/forgot

가져다 쓰는 모듈 우리 `components/Brand` `components/Ui` `lib/api` 외부 `next/link` `react`

이름	종류	줄	하는 일	쓰는 곳
`PasswordForgotPage`	컴포넌트 · 내보냄	11	S-C03 ① 재설정 링크 요청. 가입 여부와 상관없이 같은 안내를 보여 준다	이 파일 안

app/password/reset/page.tsx · /password/reset

가져다 쓰는 모듈

우리 `components/Brand` `components/PasswordChecklist` `components/Providers` `components/Ui` `lib/api`

외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`ResetForm`	컴포넌트	14	S-C03 ② 새 비밀번호 정하기. 링크는 30분, 한 번만 쓸 수 있다	이 파일 안
`PasswordResetPage`	컴포넌트 · 내보냄	70	화면 입구 (Next.js 가 `/password/reset` 에서 부름)	이 파일 안

app/signup/page.tsx · /signup

가져다 쓰는 모듈 우리 `components/Brand` `components/PasswordChecklist` `components/Ui` `lib/api`

외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`formatPhone`	함수	14	전화번호를 010-0000-0000 꼴로	이 파일 안
`SignupForm`	컴포넌트	22	회원가입 폼(약관 동의 포함)	이 파일 안
`SignupPage`	컴포넌트 · 내보냄	110	화면 입구 (Next.js 가 `/signup` 에서 부름)	이 파일 안

B-2. components — 여러 화면이 같이 쓰는 조각

components/AdminShell.tsx

가져다 쓰는 모듈

우리 `components/Brand` `components/NotificationBell` `components/Providers` `components/Shell` `components/Ui` `lib/api`

외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`AdminShell`	컴포넌트 · 내보냄	22	관리자 화면 틀(왼쪽 메뉴·제목·로그아웃). 관리자만	`app/admin/applicants/[applicationId]/page.tsx`, `app/admin/applicants/page.tsx`, `app/admin/compare/page.tsx`, `app/admin/dashboard/page.tsx` 외 7

components/ApplyFlow.tsx

가져다 쓰는 모듈 우리 `components/Providers` `components/Shell` `components/Ui` `lib/api` `lib/format` `lib/status`
외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`useApplication`	훅 · 내보냄	42	지원 단계 화면들이 지원 건 정보를 꺼내는 훅	`app/apply/[applicationId]/consent/page.tsx`, `app/apply/[applicationId]/precheck/page.tsx`, `app/apply/[applicationId]/resume/confirm/page.tsx`, `app/apply/[applicationId]/resume/page.tsx` 외 1
`emptyResume`	함수 · 내보냄	48	빈 이력서 값	`app/apply/[applicationId]/resume/page.tsx`, `app/mypage/page.tsx`
`redirectFor`	함수	54	응시 상태로 들어갈 수 있는 단계를 정한다 (§2.3 상태별 가드). 허용이면 null, 아니면 보낼 경로	이 파일 안
`Stepper`	컴포넌트 · 내보냄	77	지원 단계 표시줄(① 동의 … ④ 시작)	이 파일 안
`ApplyFlow`	컴포넌트 · 내보냄	93	지원 단계 공통 틀: 지원 건을 불러오고 상태에 맞지 않는 단계면 돌려보낸다	`app/apply/[applicationId]/consent/page.tsx`, `app/apply/[applicationId]/precheck/page.tsx`, `app/apply/[applicationId]/resume/confirm/page.tsx`, `app/apply/[applicationId]/resume/page.tsx` 외 1
`BottomBar`	컴포넌트 · 내보냄	158	아래 고정 버튼 줄	`app/apply/[applicationId]/consent/page.tsx`, `app/apply/[applicationId]/precheck/page.tsx`, `app/apply/[applicationId]/resume/confirm/page.tsx`, `app/apply/[applicationId]/resume/page.tsx` 외 1

components/Brand.tsx

역할: eslint-disable @next/next/no-img-element -- 브랜드 이미지는 public/brand 의 고정 크기 webp 라 next/image 최적화가 필요 없다 Huge 로고 (프로그램 이름, 2026-10-30 사용자 확정)

가져다 쓰는 모듈 외부 `next/link`

이름	종류	줄	하는 일	쓰는 곳
`HugeLogo`	컴포넌트 · 내보냄	5	Huge 로고 (프로그램 이름, 2026-10-30 사용자 확정)	`app/exam/[applicationId]/page.tsx`, `app/login/page.tsx`, `app/password/change/page.tsx`, `app/password/forgot/page.tsx` 외 4
`TeamCredit`	컴포넌트 · 내보냄	19	'Team AIIT' 표시 (개발 팀 크레딧)	`app/login/page.tsx`, `components/AdminShell.tsx`, `components/Shell.tsx`
`Mascot`	컴포넌트 · 내보냄	28	마스코트 그림	`app/admin/postings/ai/page.tsx`, `app/admin/postings/new/page.tsx`, `app/exam/[applicationId]/page.tsx`, `app/jobs/page.tsx` 외 4

components/exam/Coding.tsx

가져다 쓰는 모듈 우리 `components/Providers` `components/Ui` `lib/api` `lib/format`
외부 `@monaco-editor/react` `react`

이름	종류	줄	하는 일	쓰는 곳
`CodingIntro`	컴포넌트 · 내보냄	27	S-A10 안내: [시작] 또는 60초 뒤 자동 시작 (G-03)	`app/exam/[applicationId]/page.tsx`
`CodingTest`	컴포넌트 · 내보냄	65	S-A11 코딩테스트. 시간은 서버가 재고, 화면은 받은 남은 시간으로 센다	`app/exam/[applicationId]/page.tsx`

components/exam/Overlays.tsx

가져다 쓰는 모듈 우리 `components/Ui` `lib/format` 외부 `next/link` `react`

이름	종류	줄	하는 일	쓰는 곳
`Shell`	컴포넌트	18	면접 중 덮개 창 틀(ESC 로 안 닫힘, 뒤 화면 막음)	이 파일 안
`useOnline`	훅	39	브라우저 온라인 여부 훅	이 파일 안
`ToMypage`	컴포넌트	51	마이페이지로 가기 버튼	이 파일 안
`ExamOverlay`	컴포넌트 · 내보냄	60	면접 상태별 덮개(확인 중·경고·장애 중단·종료…)	`app/exam/[applicationId]/page.tsx`

components/NotificationBell.tsx

가져다 쓰는 모듈 우리 `lib/api` `lib/format` 외부 `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`NotificationBell`	컴포넌트 · 내보냄	15	머리글 🔔 사이트 알림 최근 20건 (API 명세서 9장 GET /notifications)	`components/AdminShell.tsx`, `components/Shell.tsx`
`hide`	함수	54	지우기는 목록에서 숨기는 것이다 (메일 발송 기록은 남는다)	이 파일 안

components/PasswordChecklist.tsx

역할: 비밀번호 규칙 4줄 체크리스트 (PRD 5-1, 화면정의서 §5.2). 서버도 같은 규칙으로 검사한다

이름	종류	줄	하는 일	쓰는 곳
`passwordRules`	함수 · 내보냄	2	비밀번호 규칙 4줄 체크리스트 (PRD 5-1, 화면정의서 §5.2). 서버도 같은 규칙으로 검사한다	`app/password/change/page.tsx`, `app/password/reset/page.tsx`, `app/signup/page.tsx`
`PasswordChecklist`	컴포넌트 · 내보냄	12	비밀번호 규칙 4줄 체크 표시	`app/password/change/page.tsx`, `app/password/reset/page.tsx`, `app/signup/page.tsx`

components/PostingBody.tsx

가져다 쓰는 모듈 우리 `lib/format` `lib/status` 외부 `react`

이름	종류	줄	하는 일	쓰는 곳
`Section`	컴포넌트	12	공고 본문 한 칸(글로만 표시)	이 파일 안
`PostingBody`	컴포넌트 · 내보냄	25	S-A02 본문 + 요약. 관리자 미리보기(S-R03 3단계)도 같은 모양을 쓴다	`app/jobs/[postingId]/page.tsx`, `components/PostingEditor.tsx`

components/PostingEditor.tsx

가져다 쓰는 모듈

우리 `components/PostingBody` `components/Providers` `components/Ui` `lib/api` `lib/admin` `lib/format` `lib/status`

외부 `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`toForm`	함수	36	공고 데이터를 편집 폼 값으로	이 파일 안
`step1Errors`	함수	47	1단계 필수 칸·인원·기간 검사	이 파일 안
`weightError`	함수	58	가중치 5% 단위·합계 100% 검사	이 파일 안
`criteriaError`	함수	66	2단계(평가 기준)에서 다음·저장을 막는 사유. 합격 기준 칸을 비우면 0점이 아니라 입력 요청	이 파일 안
`AiRewrite`	컴포넌트	75	AI 초안 공고 1단계의 'AI에게 한 칸 다시 쓰게 하기' 상자: 칸·요청을 골라 보내고 결과를 그 칸에 넣는다(저장은 [임시저장])	이 파일 안
`PostingEditor`	컴포넌트 · 내보냄	143	S-R03 공고 작성·수정 3단계 (새 공고는 posting 이 null)	`app/admin/postings/[postingId]/edit/page.tsx`, `app/admin/postings/new/page.tsx`
`locked`	함수	174	게시 뒤 잠긴 칸인지	이 파일 안

components/Precheck.tsx

가져다 쓰는 모듈 우리 `lib/api` `lib/recording` `lib/media` 외부 `react`

이름	종류	줄	하는 일	쓰는 곳
`Precheck`	컴포넌트 · 내보냄	28	S-A06 환경 점검. S-A15 이어가기 재점검에서도 그대로 쓴다	`app/apply/[applicationId]/precheck/page.tsx`, `app/exam/[applicationId]/resume/page.tsx`

components/Providers.tsx

가져다 쓰는 모듈 우리 `lib/api` 외부 `react`

이름	종류	줄	하는 일	쓰는 곳
`Providers`	컴포넌트 · 내보냄	22	앱 전체 공통 상태(로그인 사용자·토스트)와 401 처리	`app/layout.tsx`

이름	종류	줄	하는 일	쓰는 곳
`useApp`	훅 · 내보냄	90	공통 상태를 꺼내는 훅	`app/admin/applicants/[applicationId]/page.tsx`, `app/admin/dashboard/page.tsx`, `app/admin/postings/ai/page.tsx`, `app/admin/postings/page.tsx` 외 17

components/ResumeForm.tsx

가져다 쓰는 모듈 | 우리 `components/ApplyFlow` | 외부 `react`

이름	종류	줄	하는 일	쓰는 곳
`resumeRequirement`	함수 · 내보냄	34	이력서 필수(성명 + 학력 또는 경력) 충족 여부	`app/apply/[applicationId]/resume/confirm/page.tsx`
`LowConfLabel`	컴포넌트	45	OCR 신뢰도가 낮은 칸 표시: 이름 앞 🔍 + [확인] 버튼	이 파일 안
`ResumeForm`	컴포넌트 · 내보냄	57	이력서 편집 폼(낮은 신뢰도 칸 강조)	`app/apply/[applicationId]/resume/confirm/page.tsx`, `app/mypage/page.tsx`
`flagged`	함수	62	OCR 신뢰도 낮은 칸이면 강조 스타일	이 파일 안
`edit`	함수	64	칸 수정 알림 + 값 바꾸기	이 파일 안

components/Shell.tsx

가져다 쓰는 모듈

우리 `components/Brand` `components/NotificationBell` `components/Providers` `components/Ui` `lib/api`

외부 `next/link` `next/navigation` `react`

이름	종류	줄	하는 일	쓰는 곳
`Gnb`	컴포넌트 · 내보냄	12	위쪽 메뉴줄	`app/exam/[applicationId]/resume/page.tsx`, `components/ApplyFlow.tsx`
`Footer`	컴포넌트 · 내보냄	50	아래쪽 안내줄	`app/exam/[applicationId]/resume/page.tsx`, `components/ApplyFlow.tsx`
`SiteShell`	컴포넌트 · 내보냄	61	지원자 화면 틀(메뉴·모바일 안내·본문·바닥)	`app/jobs/[postingId]/page.tsx`, `app/jobs/page.tsx`, `app/mypage/page.tsx`, `app/mypage/results/[applicationId]/page.tsx` 외 1
`useGuard`	훅 · 내보냄	74	라우트 가드 (§2.3). 역할이 다르면 그 역할 홈으로, 로그인이 없으면 /login?next=	`app/exam/[applicationId]/page.tsx`, `app/exam/[applicationId]/resume/page.tsx`, `components/AdminShell.tsx`, `components/ApplyFlow.tsx`
`Guarded`	컴포넌트 · 내보냄	93	역할이 맞을 때만 내용 표시	`app/mypage/page.tsx`, `app/mypage/results/[applicationId]/page.tsx`
`DesktopOnly`	컴포넌트 · 내보냄	100	지원 절차·응시·관리자 화면은 PC 전용 (§4.8, G-12)	`app/exam/[applicationId]/page.tsx`, `app/exam/[applicationId]/resume/page.tsx`, `components/AdminShell.tsx`, `components/ApplyFlow.tsx`

이름	종류	줄	하는 일	쓰는 곳
`MobileBanner`	컴포넌트 · 내보냄	123	모바일에서만 보이는 닫을 수 없는 안내 (§4.8)	이 파일 안

components/Ui.tsx

가져다 쓰는 모듈 | 우리 `lib/status` | 외부 `react`

이름	종류	줄	하는 일	쓰는 곳
`StatusBadge`	컴포넌트 · 내보냄	6	응시 상태 배지	`app/admin/applicants/[applicationId]/page.tsx`, `app/admin/applicants/page.tsx`, `app/jobs/page.tsx`, `app/mypage/page.tsx` 외 1
`Spinner`	컴포넌트 · 내보냄	10	돌아가는 처리 중 표시	`app/admin/postings/ai/page.tsx`, `app/exam/[applicationId]/page.tsx`, `app/exam/[applicationId]/resume/page.tsx`
`Button`	컴포넌트 · 내보냄	16	요청 중에는 비활성 + 스피너 (중복 제출 방지, §4.3). 비활성 사유는 버튼 아래 한 줄	`app/admin/applicants/[applicationId]/page.tsx`, `app/admin/applicants/page.tsx`, `app/admin/dashboard/page.tsx`, `app/admin/postings/ai/page.tsx` 외 13
`useFocusTrap`	훅 · 내보냄	41	모달: 포커스 가두기, ESC 닫기. 응시 중 모달은 escClose=false (§4.5) 대화상자 안에 포커스를 가두고, 닫히면 원래 자리로 돌려준다	`components/exam/Overlays.tsx`
`Modal`	컴포넌트 · 내보냄	74	모달 창(포커스 가두기, ESC 선택)	`app/admin/applicants/[applicationId]/page.tsx`, `app/admin/dashboard/page.tsx`, `app/admin/postings/page.tsx`, `app/admin/settings/page.tsx` 외 7
`Field`	컴포넌트 · 내보냄	90	입력 칸 틀(이름·필수·오류·도움말)	`app/admin/postings/ai/page.tsx`, `app/login/page.tsx`, `app/mypage/page.tsx`, `app/password/change/page.tsx` 외 4
`Empty`	컴포넌트 · 내보냄	96	내용이 없을 때 안내 카드	`app/admin/applicants/[applicationId]/page.tsx`, `app/admin/applicants/page.tsx`, `app/admin/next-stage/page.tsx`, `app/admin/postings/page.tsx` 외 3
`Loading`	컴포넌트 · 내보냄	105	불러오는 중 표시	`app/admin/applicants/[applicationId]/page.tsx`, `app/admin/applicants/page.tsx`, `app/admin/compare/page.tsx`, `app/admin/dashboard/page.tsx` 외 15
`ErrorBox`	컴포넌트 · 내보냄	109	오류 안내와 [다시 시도]	`app/admin/applicants/[applicationId]/page.tsx`, `app/admin/applicants/page.tsx`, `app/admin/compare/page.tsx`, `app/admin/dashboard/page.tsx` 외 10

이름	종류	줄	하는 일	쓰는 곳
`FieldBody`	컴포넌트	122	입력이 하나면 라벨·오류·필수를 그 입력에 이어 붙인다. 여러 요소를 감싼 경우는 label 로 감싼다	이 파일 안

B-3. lib — 도우미

lib/admin.ts

역할: 관리자 공통 형식 — 관리자 화면 공통 형식

이름	종류	줄	하는 일	쓰는 곳
`Criteria · AdminPosting`	type		관리자 화면의 평가 기준·공고 모양	
`DraftStatus`	type		AI 초안 진행 상태 모양: status · stage(참고 공고 찾기 → 초안 → 점검 → 다시 쓰기) · 경고 · 처리 기록(notes) · 한 칸 다시 쓰기 결과(revision)	`app/admin/postings/ai/page.tsx`, `components/PostingEditor.tsx`
`DEFAULT_WEIGHTS`	const		기본 가중치 30/25/20/10/15	

lib/api.ts

역할: 서버 요청 공통 — API 호출. 같은 출처 /api → FastAPI (next.config rewrites). 오류는 {code, message} 한 모양 (API 명세서 2장)

이름	종류	줄	하는 일	쓰는 곳
`api`	함수 · 내보냄	17	모든 서버 요청의 입구: /api 로 보내고, 오류는 {code,message} 로 던지고, 401 이면 로그인 끝 신호	`app/admin/applicants/[applicationId]/page.tsx`, `app/admin/applicants/page.tsx`, `app/admin/compare/page.tsx`, `app/admin/dashboard/page.tsx` 외 30
`loginPath`	함수 · 내보냄	43	지금 주소를 next 로 붙인 로그인 주소	`app/password/change/page.tsx`, `components/Shell.tsx`

lib/exam.ts

역할: 면접 세션 정보 보관과 '나가요' 쪽지(sendBeacon) — 면접 세션 정보 (start · resume 응답). 응시 창(sessionStorage)에만 둔다

이름	종류	줄	하는 일	쓰는 곳
`examSessionKey`	함수 · 내보냄	12	면접 세션 정보 sessionStorage 키	`app/apply/[applicationId]/rules/page.tsx`, `app/exam/[applicationId]/resume/page.tsx`
`loadExamSession`	함수 · 내보냄	14	저장된 면접 세션 정보 읽기	`app/exam/[applicationId]/page.tsx`
`sendUnloadBeacon`	함수 · 내보냄	25	창 닫기·새로고침 신호. Blob(JSON)으로 보내 사전 요청 없이 같은 출처로 간다 (API 명세서 6장)	`app/exam/[applicationId]/page.tsx`

lib/format.ts

역할: 시각·시간 표시 — 모든 시각은 서버가 준 Asia/Seoul 시각으로 보여 준다 (기술 아키텍처 11장)

이름	종류	줄	하는 일	쓰는 곳
`parts`	함수	4	시각을 서울 기준 연·월·일·시·분 조각으로	이 파일 안
`longDateTime`	함수 · 내보냄	16	11월 12일(목) 10:00	`app/admin/applicants/[applicationId]/page.tsx`, `app/admin/next-stage/page.tsx`, `app/admin/postings/page.tsx`, `app/apply/[applicationId]/rules/page.tsx` 외 7
`shortDateTime`	함수 · 내보냄	23	11.12 10:00	`app/admin/applicants/page.tsx`, `app/admin/dashboard/page.tsx`, `app/admin/postings/page.tsx`, `app/admin/settings/page.tsx` 외 7
`dateOnly`	함수 · 내보냄	30	2026.11.12	`app/admin/applicants/[applicationId]/page.tsx`, `app/admin/next-stage/page.tsx`, `app/apply/[applicationId]/resume/page.tsx`, `app/mypage/page.tsx` 외 1
`toLocalInput`	함수 · 내보냄	37	datetime-local 입력값 ↔ ISO(+09:00)	`components/PostingEditor.tsx`
`fromLocalInput`	함수 · 내보냄	42	입력 칸 시각에 +09:00 을 붙인다	`app/admin/postings/ai/page.tsx`, `components/PostingEditor.tsx`
`mmss`	함수 · 내보냄	46	초를 MM:SS 로	`app/admin/applicants/[applicationId]/page.tsx`, `app/exam/[applicationId]/page.tsx`, `components/exam/Coding.tsx`
`untilText`	함수 · 내보냄	51	마감까지 남은 시간 글(D-n, n시간…)	`app/admin/postings/page.tsx`, `app/jobs/[postingId]/page.tsx`, `app/jobs/page.tsx`

lib/media.ts

역할: 카메라·마이크·스피커 도우미 — 장치·브라우저 도우미. 점검(S-A06)에서 고른 장치를 면접에서도 쓴다

이름	종류	줄	하는 일	쓰는 곳
`loadDeviceChoice`	함수 · 내보냄	5	점검에서 고른 장치 읽기	`app/exam/[applicationId]/page.tsx`, `components/Precheck.tsx`
`saveDeviceChoice`	함수 · 내보냄	13	고른 장치 저장(sessionStorage)	`components/Precheck.tsx`
`detectBrowser`	함수 · 내보냄	17	브라우저 이름·버전 알아내기(Edge/Chrome)	`components/Precheck.tsx`
`isSupportedBrowser`	함수 · 내보냄	29	Chrome·Edge 120 이상인지	`components/Precheck.tsx`
`openStream`	함수 · 내보냄	35	고른 장치를 먼저 쓰고, 그 장치를 뺐으면 다른 장치로 연다 (exact 로 요구하면 장치를 뺀 뒤 열기에 실패한다)	`app/exam/[applicationId]/page.tsx`, `components/Precheck.tsx`
`routeToSpeaker`	함수 · 내보냄	43	환경 점검에서 고른 스피커로 소리를 낸다. 고를 수 없는 브라우저나 장치를 뺀 경우는 기본 출력 그대로	`app/exam/[applicationId]/page.tsx`
`stopStream`	함수 · 내보냄	47	카메라·마이크 끄기	`app/exam/[applicationId]/page.tsx`, `components/Precheck.tsx`

이름	종류	줄	하는 일	쓰는 곳
`levelMeter`	함수 · 내보냄	53	마이크 입력 크기(0~1)를 콜백으로 알려 주는 측정기	`components/Precheck.tsx`
`loop`	함수	61	마이크 크기를 계속 재는 반복	이 파일 안
`playTestTone`	함수 · 내보냄	71	고른 스피커로 테스트음 재생	`components/Precheck.tsx`
`measureMbps`	함수 · 내보냄	97	대략의 내려받기 속도(Mbps). 같은 서버의 정적 파일로 잰다	`components/Precheck.tsx`

lib/pcm.ts

역할: 답변 녹음기 — 답변 녹음기: 켜져 있는 동안만 PCM 조각을 onChunk 로 넘긴다. 마이크 레벨도 함께 잰다

이름	종류	줄	하는 일	쓰는 곳
`PcmRecorder`	class		답변 녹음기: 켜져 있는 동안만 16kHz PCM 조각을 onChunk 로 넘기고 마이크 크기도 잰다	

lib/recording.ts

역할: 카메라 녹화 조각 올리기 — 카메라 녹화: MediaRecorder 10초 조각을 순서대로 올린다 (API 명세서 6장 POST /interview/{sid}/recording). 녹화는 부정행위 확인용이며 평가에 쓰지 않는다. 조각은 1·2·4초 간격으로 3번(upload_re

가져다 쓰는 모듈 🏠 우리 `lib/api`

이름	종류	줄	하는 일	쓰는 곳
`mimeType`	함수	10	브라우저가 지원하는 녹화 형식	이 파일 안
`createRecorder`	함수	14	MediaRecorder 만들기(비트레이트 지정)	이 파일 안
`canRecord`	함수 · 내보냄	20	환경 점검용: 이 브라우저에서 실제로 녹화를 시작할 수 있는지 잠깐 켜 본다	`components/Precheck.tsx`

lib/resumeDraft.ts

역할: 이력서 임시값 — S-A04 → S-A05 로 넘기는 이력서 임시값. 이 창(sessionStorage)에만 두고 제출하면 지운다 low_conf_fields: OCR 신뢰도가 낮아 확인이 필요한 칸 (예: career[0].company)

가져다 쓰는 모듈 🏠 우리 `components/ApplyFlow`

이름	종류	줄	하는 일	쓰는 곳
`draftKey`	함수 · 내보냄	6	이력서 임시값 sessionStorage 키	`app/apply/[applicationId]/resume/confirm/page.tsx`, `app/apply/[applicationId]/resume/page.tsx`

`lib/status.ts`

역할: 상태 이름·색 — 응시 상태 이름·배지 색 (PRD 6-2, 화면정의서 §3.4 · §4.4)

이름	종류	줄	하는 일	쓰는 곳
`ATTEMPT_LABEL / ATTEMPT_BADGE`	const		응시 상태 → 한글 이름 / 배지 색	
`POSTING_BUTTON`	const		응시 상태별 공고 화면 버튼 글·활성 여부	
`EVAL_LABEL · EVENT_LABEL · JUDGED_LABEL · REVIEW_REASON`	const		평가 상태·기록 종류·판정·지연 사유의 한글 이름	

B-4. 기타

`proxy.ts`

역할: 127.0.0.1 로 주소 맞추기 — localhost 와 127.0.0.1 은 다른 출처라 쿠키·WebSocket Origin 검사가 어긋난다 → 127.0.0.1 로 맞춘다 (API 명세서 2-5)

가져다 쓰는 모듈 | 📦 외부 `next/server`

이름	종류	줄	하는 일	쓰는 곳
`proxy`	함수 · 내보냄	4	localhost 와 127.0.0.1 은 다른 출처라 쿠키·WebSocket Origin 검사가 어긋난다 → 127.0.0.1 로 맞춘다 (API 명세서 2-5)	이 파일 안

부록 C. 테스트 지도 (`huge/backend/tests` , `huge/e2e/tests`)

함수 하나가 맞게 동작하는지 궁금하면, 그 함수 이름으로 테스트 폴더를 검색하세요. 테스트는 '이 함수는 이렇게 동작해야 한다'는 살아 있는 설명서입니다.

파일	테스트 수	무엇을 확인하나
`backend/tests/test_admin_ops.py`	10	관리자 2단계-B: 비교 · 다음 전형 · 통계 · 운영 현황 · 운영 설정.
`backend/tests/test_admission.py`	2	동시 면접 입장 (E10). 입장은 시작 API 의 한 트랜잭션 안에서 정해져야 하고, 숫자 맞추기가 커밋 전 입장을 놓치면 안 된다.
`backend/tests/test_api_apply.py`	14	
`backend/tests/test_api_auth.py`	8	
`backend/tests/test_api_postings.py`	9	
`backend/tests/test_coding_bank.py`	10	코딩테스트 문제 목록 (COD-02): 문제 자체가 맞는지, 지원 건마다 기초 1 · 응용 1 을 고르는지 확인한다.

파일	테스트 수	무엇을 확인하나
`backend/tests/test_coding_resume.py`	4	코딩테스트 이어가기 (COD-04, TC-085): 다시 연결되면 안내 화면 없이 저장된 코드와 남은 시간부터 이어 간다.
`backend/tests/test_compose_isolation.py`	4	AI 상자는 밖으로 나가는 망에 붙지 않고, Redis 는 비밀번호 없이 받지 않는다 (논의사항 P3, AC-19).
`backend/tests/test_crypto_key.py`	3	암호화 키가 없거나 틀리면 개발 환경에서도 켜지지 않는다 (논의사항 P4).
`backend/tests/test_device_end.py`	5	장치가 꺼졌을 때 판정 (E18, TC-064). 창을 닫으면 카메라·마이크가 먼저 꺼지고 unload·연결 끊김이 뒤따른다.
`backend/tests/test_devtools.py`	8	개발자 확인 명령 (python -m app.devtools): DB 의 암호화 칸을 서버 안에서만 풀어 본다 (2026-11-02 사용자 요청).
`backend/tests/test_evaluation.py`	33	2단계 채점·결과 확정 (PRD 5-7·5-8, TRD 8장). 모델은 가짜로 바꿔 정해진 답을 돌려준다.
`backend/tests/test_followup.py`	3	
`backend/tests/test_followup_limits.py`	4	꼬리질문 상한 (INT-06): 질문당 1개, 질문 종류별 1개, 전체 4개. 앞 질문이 상한을 다 써서 직무 질문이 빠지는 일을 막는다.
`backend/tests/test_interview_graph.py`	5	면접 진행 그래프의 순서·갈림길 (ADR-07). 실제 일은 가짜 러너가 기록만 한다.
`backend/tests/test_login_block.py`	18	로그인 실패 IP 차단: 계정과 무관하게 같은 IP 5회 실패면 막고, 관리자 해제 또는 24시간 뒤 풀린다.
`backend/tests/test_memos.py`	4	관리자 메모 (ADM-05, API 명세서 10장 메모, 테이블 정의서 4-21).
`backend/tests/test_notifications.py`	3	머리글 🔔 알림: 목록 · 읽음 · 숨기기 (2026-11-02 사용자 결정 A: 지우지 않고 숨긴다. 메일 발송 기록은 남긴다).
`backend/tests/test_password_policy.py`	2	
`backend/tests/test_password_reset.py`	6	비밀번호 찾기·재설정 (화면정의서 S-C03, API 명세서 3장, 테이블 정의서 5장 pw_reset).
`backend/tests/test_pdf_box.py`	21	PDF 는 별도 프로세스에서 읽는다. 멈추거나 죽어도 api 는 영향이 없다 (논의사항 P5).
`backend/tests/test_posting_draft.py`	22	AI 공고 초안 (S-R02): 정해진 값은 그대로 복사, 문장은 AI, 회사명 없이 게시 가능.
`backend/tests/test_posting_graph.py`	15	AI 공고 초안 그래프 (2026-11-10 사용자 결정): 참고 공고 찾기 → 초안 → 칸별 검사 → 문제 칸만 다시 쓰기 → 저장, 그리고 같은 thread 로 이어서 관리자가 고른 칸만 다시 쓰기.
`backend/tests/test_posting_questions.py`	9	질문 적용 범위와 공고 기반 질문 후보 (2026-11-10 사용자 결정, CONFLICTS C-17).
`backend/tests/test_question_dedup.py`	7	질문 10개 안에 같은 질문이 두 번 나오지 않게 하는 비교.
`backend/tests/test_question_rag.py`	32	질문 10개 RAG 구성 (TRD 7-4, 사용자 결정 U5): RAG 로 먼저 만들고, 못 만들면 질문은행 → 예비 질문.
`backend/tests/test_question_text.py`	15	AI 가 만든 질문 글 검사 (본질문·꼬리질문 공통, 2026-11-10 사용자 결정): 찌꺼기·답변 방식 지시·한 번에 여러 개 묻기.
`backend/tests/test_recording.py`	8	카메라 녹화 (API 명세서 6장 POST /interview/{sid}/recording · 2-6 미디어, 테이블 정의서 4-10, 녹화 3일 삭제).
`backend/tests/test_reserve_fallback.py`	2	예비 질문이 바닥났을 때 같은 문장이 여러 자리에 들어가지 않는다 (E11).

파일	테스트 수	무엇을 확인하나
`backend/tests/test_resume_extract.py`	18	
`backend/tests/test_resume_limit.py`	2	장애 이어가기 상한 (CONFLICTS D1, 사용자 결정 2026-10-30: 3회). 지원자 쪽 끊김만 센다.
`backend/tests/test_retention.py`	10	보관 기간 삭제·코딩 실행 worker 이전 (2단계-C).
`backend/tests/test_review_deadline.py`	16	'평가 지연' 건: 6시간마다 자동 재채점, 3일 안에 관리자 결정, 없으면 마지막 재채점까지 실패한 뒤 평가 미완료 불합격,
`backend/tests/test_review_fixes.py`	3	코드 리뷰 반영 회귀 테스트: 비밀번호 변경 시 다른 로그인 끊기 · 붙은 세션 없는 진행 중 응시 정리.
`backend/tests/test_sandbox_judge.py`	5	상자 밖 채점 비교 (채점 위조 방지).
`backend/tests/test_schema_alignment.py`	8	테이블 정의서에 맞춘 덧붙이기 (CONFLICTS.md T 절, 2026-11-02 사용자 결정): CHECK 제약 · 빠진 시각 칸 · 수정 요약.
`backend/tests/test_scoring.py`	8	
`backend/tests/test_server_error.py`	1	처리되지 않은 서버 오류도 {code, message} 한 모양으로 답한다 (API 명세서 2장·11장, 논의사항 E9).
`backend/tests/test_status.py`	4	
`backend/tests/test_stt_server.py`	2	STT 서버: 변환이 도는 동안에도 /health 가 답해야 한다 (논의사항 E2).
`backend/tests/test_withdraw.py`	6	동의 철회 (APP-03)·회원 탈퇴 (화면정의서 S-A13 계정 탭, API 명세서 3장·5장, 테이블 정의서 1-6·1-7).
`e2e/tests/access.spec.ts`	3	로그인하지 않으면 전용 화면은 로그인으로 보내고 API 는 401 이다 외
`e2e/tests/auth.spec.ts`	2	회원가입 → 로그인 → 비밀번호 찾기 메일 → 새 비밀번호로 로그인 → 탈퇴 외
`e2e/tests/capacity.spec.ts`	1	자리가 없으면 안내 모달만 뜨고, 자리가 나면 다시 시도로 시작한다
`e2e/tests/closing.spec.ts`	2	마감하면 진행 중 면접은 바로 끝나고, 시작 전 지원자는 시작할 수 없다 외
`e2e/tests/coding-timeout.spec.ts`	1	끊긴 코딩테스트는 저장된 코드부터 이어 가고, 시간이 끝나면 자동 제출된다
`e2e/tests/device-failure.spec.ts`	5	지원자: 마이크 녹음기를 열 수 없으면 장애 중단 외
`e2e/tests/exit.spec.ts`	2	뒤로가기는 화면을 그대로 두고, 새로고침하면 이어가기 없이 응시 종료다 외
`e2e/tests/full.spec.ts`	2	코딩테스트 없는 공고: 마지막 질문 뒤 바로 응시 완료 → 채점 → 결과 공개 외
`e2e/tests/interview.spec.ts`	3	지원자: 지원부터 면접 중 화면 이탈 2회 취소까지 외
`e2e/tests/login-block.spec.ts`	2	지원자: 같은 IP 에서 5번 틀리면 차단되고, 관리자가 해제하면 다시 로그인한다 외
`e2e/tests/outage.spec.ts`	1	지원자: 면접 중 연결이 끊기면 장애 중단, 마이페이지에서 이어간다
`e2e/tests/resume-upload.spec.ts`	3	지원자: 이력서 PDF 를 별도 프로세스로 읽어 정리하고, 깨진 PDF 는 거부한다 외

파일	테스트 수	무엇을 확인하나
`e2e/tests/review-decision.spec.ts`	1	관리자: 평가 지연 건을 직접 결정하고, 지원자는 담당자 결정임을 본다